

—— // ZEMACS // ——

zemacs — Reference

v0.4.21

THE REFERENCE

every key · every command · every setting

2026-07-13

MenkeTechnologies

—— MENKETECHNOLOGIES ——

Contents

About This Reference	2
Keymap	3
Keymap	3
Normal mode	3
Insert mode	13
Select / extend mode	14
Picker.....	14
Prompt.....	14
Configuration and Settings	16
Editor	16
Typable Commands	35
Static Commands and Keybindings	74
Language Support	99

About This Reference

This is the zemacs reference: every keybinding, every configuration setting, and every command the editor ships, generated from the source. It is the companion to the narrative book — where the book explains *why*, this enumerates *what*.

The sections are: the **keymap** (organized by mode and prefix), the **configuration schema** ([\[editor\]](#) and its tables), the full list of **typable : commands**, the full list of **static commands with their default keybindings** by mode, and the **language-support** matrix.

Keymap

Keymap

- Normal mode
 - Movement
 - Changes
 - * Shell
 - Selection manipulation
 - Search
 - Minor modes
 - * View mode
 - * Goto mode
 - * Match mode
 - * Window mode
 - * Space mode
 - Popup
 - Completion Menu
 - Signature-help Popup
 - * Unimpaired
- Insert mode
- Select / extend mode
- Picker
- Prompt

□ Mappings marked (**LSP**) require an active language server for the file.

□ Mappings marked (**TS**) require a tree-sitter grammar for the file type.

⚠ Some terminals' default key mappings conflict with zemacs's. If any of the mappings described on this page do not work as expected, check your terminal's mappings to ensure they do not conflict. See the [wiki](#) for known conflicts.

Normal mode

Normal mode is the default mode when you launch zemacs. You can return to it from other modes by pressing the **Escape** key.

Movement

NOTE: Unlike Vim, **f**, **F**, **t** and **T** are not confined to the current line.

Hereafter, **<n>** represents an integer by typing a sequence of digits.

Key	Description	Command
h , Left	Move left	<code>move_char_left</code>
j , Down	Move down	<code>move_visual_line_down</code>
k , Up	Move up	<code>move_visual_line_up</code>
l , Right	Move right	<code>move_char_right</code>
w	Move next word start	<code>move_next_word_start</code>
b	Move previous word start	<code>move_prev_word_start</code>
e	Move next word end	<code>move_next_word_end</code>

Key	Description	Command
W	Move next WORD start	<code>move_next_long_word_start</code>
B	Move previous WORD start	<code>move_prev_long_word_start</code>
E	Move next WORD end	<code>move_next_long_word_end</code>
t	Find till next char	<code>find_till_char</code>
f	Find next char	<code>find_next_char</code>
T	Find till previous char	<code>till_prev_char</code>
F	Find previous char	<code>find_prev_char</code>
<n>G, <n>gg	Go to line number <n>	<code>goto_line</code>
Alt-.	Repeat last motion (f, t, m, [or])	<code>repeat_last_motion</code>
Home	Move to the start of the line	<code>goto_line_start</code>
End	Move to the end of the line	<code>goto_line_end</code>
Ctrl-b, PageUp	Move page up	<code>page_up</code>
Ctrl-f, PageDown	Move page down	<code>page_down</code>
Ctrl-u	Move cursor and page half page up	<code>page_cursor_half_up</code>
Ctrl-d	Move cursor and page half page down	<code>page_cursor_half_down</code>
Ctrl-i	Jump forward on the jumplist	<code>jump_forward</code>
Ctrl-o	Jump backward on the jumplist	<code>jump_backward</code>
Ctrl-s	Save the current selection to the jumplist	<code>save_selection</code>

Changes

Key	Description	Command
r	Replace with a character	<code>replace</code>
R	Replace with yanked text	<code>replace_with_yanked</code>
~	Switch case of the selected text	<code>switch_case</code>
`	Set the selected text to lower case	<code>switch_to_lowercase</code>
Alt-`	Set the selected text to upper case	<code>switch_to_uppercase</code>
i	Insert before selection	<code>insert_mode</code>
a	Insert after selection (append)	<code>append_mode</code>
I	Insert at the start of the line	<code>insert_at_line_start</code>
A	Insert at the end of the line	<code>insert_at_line_end</code>
o	Open new line below selection	<code>open_below</code>
O	Open new line above selection	<code>open_above</code>
.	Repeat last insert	N/A
u	Undo change	<code>undo</code>

Key	Description	Command
<code>U</code>	Redo change	<code>redo</code>
<code>Alt-u</code>	Move backward in history	<code>earlier</code>
<code>Alt-U</code>	Move forward in history	<code>later</code>
<code>y</code>	Yank selection	<code>yank</code>
<code>P</code>	Paste after selection	<code>paste_after</code>
<code>p</code>	Paste before selection	<code>paste_before</code>
<code>" <reg></code>	Select a register to yank to or paste from	<code>select_register</code>
<code>></code>	Indent selection	<code>indent</code>
<code><</code>	Unindent selection	<code>unindent</code>
<code>=</code>	Format selection (LSP)	<code>format_selections</code>
<code>d</code>	Delete selection	<code>delete_selection</code>
<code>Alt-d</code>	Delete selection, without yanking	<code>delete_selection_noyank</code>
<code>c</code>	Change selection (delete and enter insert mode)	<code>change_selection</code>
<code>Alt-c</code>	Change selection (delete and enter insert mode, without yanking)	<code>change_selection_noyank</code>
<code>[ctrl-a</code>	Increment object (number) under cursor	<code>increment</code>
<code>[ctrl-x</code>	Decrement object (number) under cursor	<code>decrement</code>
<code>Q</code>	Start/stop macro recording to the selected register (experimental)	<code>record_macro</code>
<code>q</code>	Play back a recorded macro from the selected register (experimental)	<code>replay_macro</code>
<code>[ctrl-z</code>	Suspend Zemacs and return to the shell (resume with <code>fg</code>)	<code>suspend</code>

Shell

Key	Description	Command
<code> </code>	Pipe each selection through shell command, replacing with output	<code>shell_pipe</code>
<code>Alt- </code>	Pipe each selection into shell command, ignoring output	<code>shell_pipe_to</code>
<code>!</code>	Run shell command, inserting output before each selection	<code>shell_insert_output</code>
<code>Alt-!</code>	Run shell command, appending output after each selection	<code>shell_append_output</code>
<code>\$</code>	Pipe each selection into shell command, keep selections where command returned 0	<code>shell_keep_pipe</code>

Selection manipulation

Key	Description	Command
<code>s</code>	Select all regex matches inside selections	<code>select_regex</code>
<code>S</code>	Split selection into sub selections on regex matches	<code>split_selection</code>

Key	Description	Command
<code>Alt-s</code>	Split selection on newlines	<code>split_selection_on_newline</code>
<code>Alt-minus</code>	Merge selections	<code>merge_selections</code>
<code>Alt-_</code>	Merge consecutive selections	<code>merge_consecutive_selections</code>
<code>&</code>	Align selection in columns	<code>align_selections</code>
<code>-</code>	Trim whitespace from the selection	<code>trim_selections</code>
<code>;</code>	Collapse selection onto a single cursor	<code>collapse_selection</code>
<code>Alt-;</code>	Flip selection cursor and anchor	<code>flip_selections</code>
<code>Alt-:</code>	Ensures the selection is in forward direction	<code>ensure_selections_forward</code>
<code>,</code>	Keep only the primary selection	<code>keep_primary_selection</code>
<code>Alt-,</code>	Remove the primary selection	<code>remove_primary_selection</code>
<code>C</code>	Copy selection onto the next line (Add cursor below)	<code>copy_selection_on_next_line</code>
<code>Alt-C</code>	Copy selection onto the previous line (Add cursor above)	<code>copy_selection_on_prev_line</code>
<code>(</code>	Rotate main selection backward	<code>rotate_selections_backward</code>
<code>)</code>	Rotate main selection forward	<code>rotate_selections_forward</code>
<code>Alt-(</code>	Rotate selection contents backward	<code>rotate_selection_contents_backward</code>
<code>Alt-)</code>	Rotate selection contents forward	<code>rotate_selection_contents_forward</code>
<code>%</code>	Select entire file	<code>select_all</code>
<code>x</code>	Select current line, if already selected, extend to next line	<code>extend_line_below</code>
<code>X</code>	Extend selection to line bounds (line-wise selection)	<code>extend_to_line_bounds</code>
<code>Alt-x</code>	Shrink selection to line bounds (line-wise selection)	<code>shrink_to_line_bounds</code>
<code>J</code>	Join lines inside selection	<code>join_selections</code>
<code>Alt-J</code>	Join lines inside selection and select the inserted space	<code>join_selections_space</code>
<code>K</code>	Keep selections matching the regex	<code>keep_selections</code>
<code>Alt-K</code>	Remove selections matching the regex	<code>remove_selections</code>
<code>Ctrl-c</code>	Comment/uncomment the selections	<code>toggle_comments</code>
<code>Alt-o, Alt-up</code>	Expand selection to parent syntax node (TS)	<code>expand_selection</code>
<code>Alt-i, Alt-down</code>	Shrink syntax tree object selection (TS)	<code>shrink_selection</code>
<code>Alt-p, Alt-left</code>	Select previous sibling node in syntax tree (TS)	<code>select_prev_sibling</code>
<code>Alt-n, Alt-right</code>	Select next sibling node in syntax tree (TS)	<code>select_next_sibling</code>
<code>Alt-a</code>	Select all sibling nodes in syntax tree (TS)	<code>select_all_siblings</code>
<code>Alt-I, Alt-Shift-down</code>	Select all children nodes in syntax tree (TS)	<code>select_all_children</code>
<code>Alt-e</code>	Move to end of parent node in syntax tree (TS)	<code>move_parent_node_end</code>

Key	Description	Command
Alt-b	Move to start of parent node in syntax tree (TS)	<code>move_parent_node_start</code>

Search

Search commands all operate on the `/` register by default. To use a different register, use "`<char>`".

Key	Description	Command
<code>/</code>	Search for regex pattern	<code>search</code>
<code>?</code>	Search for previous pattern	<code>rsearch</code>
<code>n</code>	Select next search match	<code>search_next</code>
<code>N</code>	Select previous search match	<code>search_prev</code>
<code>*</code>	Use current selection as the search pattern, automatically wrapping with <code>\b</code> on word boundaries	<code>search_selection_detect_word_boundaries</code>
Alt-*	Use current selection as the search pattern	<code>search_selection</code>

Minor modes

These sub-modes are accessible from normal mode and typically switch back to normal mode after a command.

Key	Description	Command
<code>v</code>	Enter select (extend) mode	<code>select_mode</code>
<code>g</code>	Enter <code>goto mode</code>	N/A
<code>m</code>	Enter <code>match mode</code>	N/A
<code>:</code>	Enter command mode	<code>command_mode</code>
<code>z</code>	Enter <code>view mode</code>	N/A
<code>Z</code>	Enter sticky <code>view mode</code>	N/A
<code>[ctrl-w</code>	Enter <code>window mode</code>	N/A
<code>Space</code>	Enter <code>space mode</code>	N/A

These modes (except command mode) can be configured by [remapping keys](#).

View mode

Accessed by typing `z` in `normal mode`.

View mode is intended for scrolling and manipulating the view without changing the selection. The "sticky" variant of this mode (accessed by typing `Z` in normal mode) is persistent and can be exited using the escape key. This is useful when you're simply looking over text and not actively editing it.

Key	Description	Command
<code>z, c</code>	Vertically center the line	<code>align_view_center</code>
<code>t</code>	Align the line to the top of the screen	<code>align_view_top</code>
<code>b</code>	Align the line to the bottom of the screen	<code>align_view_bottom</code>

Key	Description	Command
<code>m</code>	Align the line to the middle of the screen (horizontally)	<code>align_view_middle</code>
<code>j, down</code>	Scroll the view downwards	<code>scroll_down</code>
<code>k, up</code>	Scroll the view upwards	<code>scroll_up</code>
<code>[ctrl-f, PageDown]</code>	Move page down	<code>page_down</code>
<code>[ctrl-b, PageUp]</code>	Move page up	<code>page_up</code>
<code>[ctrl-u]</code>	Move cursor and page half page up	<code>page_cursor_half_up</code>
<code>[ctrl-d]</code>	Move cursor and page half page down	<code>page_cursor_half_down</code>

Goto mode

Accessed by typing `g` in **normal mode**.

Jumps to various locations.

Key	Description	Command
<code><n>g</code>	Go to line number <code><n></code>	<code>goto_file_start</code>
<code>g</code>	Go to the start of the file	<code>goto_file_start</code>
<code><n> </code>	Go to column number <code><n></code>	<code>goto_column</code>
<code> </code>	Go to the start of line	<code>goto_column</code>
<code>e</code>	Go to the end of the file	<code>goto_last_line</code>
<code>f</code>	Go to files/URLs in selections	<code>goto_file</code>
<code>h</code>	Go to the start of the line	<code>goto_line_start</code>
<code>l</code>	Go to the end of the line	<code>goto_line_end</code>
<code>s</code>	Go to first non-whitespace character of the line	<code>goto_first_nonwhitespace</code>
<code>t</code>	Go to the top of the screen	<code>goto_window_top</code>
<code>c</code>	Go to the middle of the screen	<code>goto_window_center</code>
<code>b</code>	Go to the bottom of the screen	<code>goto_window_bottom</code>
<code>d</code>	Go to definition (LSP)	<code>goto_definition</code>
<code>D</code>	Go to declaration (LSP)	<code>goto_declaration</code>
<code>y</code>	Go to type definition (LSP)	<code>goto_type_definition</code>
<code>r</code>	Go to references (LSP)	<code>goto_reference</code>
<code>i</code>	Go to implementation (LSP)	<code>goto_implementation</code>
<code>a</code>	Go to the last accessed/alternate file	<code>goto_last_accessed_file</code>
<code>m</code>	Go to the last modified/alternate file	<code>goto_last_modified_file</code>
<code>n</code>	Go to next buffer	<code>goto_next_buffer</code>
<code>p</code>	Go to previous buffer	<code>goto_previous_buffer</code>
<code>.</code>	Go to last modification in current file	<code>goto_last_modification</code>
<code>j</code>	Move down textual (instead of visual) line	<code>move_line_down</code>

Key	Description	Command
k	Move up textual (instead of visual) line	<code>move_line_up</code>
w	Show labels at each word and select the word that belongs to the entered labels	<code>goto_word</code>

Match mode

Accessed by typing **m** in **normal mode**.

Please refer to the relevant sections for detailed explanations about **surround** and **textobjects**.

Key	Description	Command
m	Goto matching bracket (TS)	<code>match_brackets</code>
s <char>	Surround current selection with <char>	<code>surround_add</code>
r <from> <to>	Replace surround character <from> with <to>	<code>surround_replace</code>
d <char>	Delete surround character <char>	<code>surround_delete</code>
a <object>	Select around textobject	<code>select_textobject_around</code>
i <object>	Select inside textobject	<code>select_textobject_inner</code>

TODO: Mappings for selecting syntax nodes (a superset of **[]**).

Window mode

Accessed by typing **Ctrl-w** in **normal mode**.

This layer is similar to Vim keybindings as Kakoune does not support windows.

Key	Description	Command
w , Ctrl-w	Switch to next window	<code>rotate_view</code>
v , Ctrl-v	Vertical right split	<code>vsplit</code>
s , Ctrl-s	Horizontal bottom split	<code>hsplit</code>
t , Ctrl-t	Transpose the two adjacent splits	<code>transpose_view</code>
f	Go to files/URLs in selections in horizontal splits	<code>goto_file_hsplit</code>
F	Go to files/URLs in selections in vertical splits	<code>goto_file_vsplit</code>
h , Ctrl-h , Left	Move to left split	<code>jump_view_left</code>
j , Ctrl-j , Down	Move to split below	<code>jump_view_down</code>
k , Ctrl-k , Up	Move to split above	<code>jump_view_up</code>
l , Ctrl-l , Right	Move to right split	<code>jump_view_right</code>
q , Ctrl-q	Close current window	<code>wclose</code>
o , Ctrl-o	Only keep the current window, closing all the others	<code>wonly</code>
H	Swap window to the left	<code>swap_view_left</code>
J	Swap window downwards	<code>swap_view_down</code>
K	Swap window upwards	<code>swap_view_up</code>

Key	Description	Command
L	Swap window to the right	<code>swap_view_right</code>
ns, nCtrl-s	New horizontal split with a scratch buffer	<code>hsplit_new</code>
nv, nCtrl-v	New vertical split with a scratch buffer	<code>vsplit_new</code>

Space mode

Accessed by typing **Space** in **normal mode**.

This layer is a kludge of mappings, mostly pickers.

Key	Description	Command
f	Open file picker at LSP workspace root	<code>file_picker</code>
F	Open file picker at current working directory	<code>file_picker_in_current_directory</code>
e	Open file explorer at workspace root	<code>file_explorer</code>
.	Open file explorer at current buffer's directory	<code>file_explorer_in_current_buffer_directory</code>
b	Open buffer picker	<code>buffer_picker</code>
j	Open jumplist picker	<code>jumplist_picker</code>
g	Open changed file picker	<code>changed_file_picker</code>
G	Debug (experimental)	N/A
k	Show documentation for item under cursor in a popup (LSP)	<code>hover</code>
s	Open document symbol picker (LSP or TS)	<code>lsp_or_syntax_symbol_picker</code>
S	Open workspace symbol picker (LSP or TS)	<code>lsp_or_syntax_workspace_symbol_picker</code>
d	Open document diagnostics picker (LSP)	<code>diagnostics_picker</code>
D	Open workspace diagnostics picker (LSP)	<code>workspace_diagnostics_picker</code>
r	Rename symbol (LSP)	<code>rename_symbol</code>
a	Apply code action (LSP)	<code>code_action</code>
h	Select symbol references (LSP)	<code>select_references_to_symbol_under_cursor</code>
,	Open last fuzzy picker	<code>last_picker</code>
w	Enter window mode	N/A
c	Comment/uncomment selections	<code>toggle_comments</code>
C	Block comment/uncomment selections	<code>toggle_block_comments</code>
Alt-c	Line comment/uncomment selections	<code>toggle_line_comments</code>
p	Paste system clipboard after selections	<code>paste_clipboard_after</code>
P	Paste system clipboard before selections	<code>paste_clipboard_before</code>

Key	Description	Command
y	Yank selections to clipboard	yank_to_clipboard
Y	Yank main selection to clipboard	yank_main_selection_to_clipboard
R	Replace selections by clipboard contents	replace_selections_with_clipboard
/	Global search in workspace folder	global_search
?	Open command palette	command_palette

□ Global search displays results in a fuzzy picker, use **Space + '** to bring it back up after opening a file.

Popup Displays documentation for item under cursor. Remapping currently not supported.

Key	Description
Ctrl-u	Scroll up
Ctrl-d	Scroll down

Completion Menu Displays documentation for the selected completion item. Remapping currently not supported.

Key	Description
Shift-Tab, Ctrl-p, Up	Previous entry
Tab, Ctrl-n, Down	Next entry
Enter	Close menu and accept completion
Ctrl-c	Close menu and reject completion

Any other keypresses result in the completion being accepted.

Signature-help Popup Displays the signature of the selected completion item. Remapping currently not supported.

Key	Description
Alt-p	Previous signature
Alt-n	Next signature

Unimpaired

These mappings are in the style of **vim-unimpaired**.

Key	Description	Command
]d	Go to next diagnostic (LSP)	goto_next_diag
[d	Go to previous diagnostic (LSP)	goto_prev_diag
]D	Go to last diagnostic in document (LSP)	goto_last_diag
[D	Go to first diagnostic in document (LSP)	goto_first_diag
]f	Go to next function (TS)	goto_next_function
[f	Go to previous function (TS)	goto_prev_function
]t	Go to next type definition (TS)	goto_next_class

Key	Description	Command
[t	Go to previous type definition (TS)	goto_prev_class
]a	Go to next argument/parameter (TS)	goto_next_parameter
[a	Go to previous argument/parameter (TS)	goto_prev_parameter
]c	Go to next comment (TS)	goto_next_comment
[c	Go to previous comment (TS)	goto_prev_comment
]e	Go to next entry (TS)	goto_next_entry
[e	Go to previous entry (TS)	goto_prev_entry
]T	Go to next test (TS)	goto_next_test
[T	Go to previous test (TS)	goto_prev_test
]p	Go to next paragraph	goto_next_paragraph
[p	Go to previous paragraph	goto_prev_paragraph
]g	Go to next change	goto_next_change
[g	Go to previous change	goto_prev_change
]G	Go to last change	goto_last_change
[G	Go to first change	goto_first_change
]n	Go to next merge-conflict marker	goto_next_conflict
[n	Go to previous merge-conflict marker	goto_prev_conflict
]x	Go to next (X)HTML element	goto_next_xml_element
[x	Go to previous (X)HTML element	goto_prev_xml_element
]Space	Add newline below	add_newline_below
[Space	Add newline above	add_newline_above

Wildfire

In normal mode, expand or shrink the selection by syntax-aware text objects.

Key	Description	Command
<ret>	Select / grow to the closest enclosing text object	wildfire
<backspace>	Shrink to the previously selected text object	wildfire_shrink

Git conflicts and merge

These live under the space-g git menu (see [Space mode](#)) and the unpaired]n/[n motions above.

Key	Description	Command
SPC g m, SPC g c r	Open the 3-pane merge-conflict resolver (:merge)	resolve_conflicts
SPC g c O	Resolve all conflicts by keeping our side	conflict_take_all_ours
SPC g c T	Resolve all conflicts by keeping their side	conflict_take_all_theirs

Key	Description	Command
<code>SPC g =</code>	Side-by-side diff of the buffer vs. git HEAD (:diff)	<code>git_diff</code>

Insert mode

Accessed by typing `i` in `normal mode`.

Insert mode bindings are minimal by default. Emacs is designed to be a modal editor, and this is reflected in the user experience and internal mechanics. Changes to the text are only saved for undos when escaping from insert mode to normal mode.

- New users are strongly encouraged to learn the modal editing paradigm to get the smoothest experience.

Key	Description	Command
<code>Escape</code>	Switch to normal mode	<code>normal_mode</code>
<code>Ctrl-s</code>	Commit undo checkpoint	<code>commit_undo_checkpoint</code>
<code>Ctrl-x</code>	Autocomplete	<code>completion</code>
<code>Ctrl-r</code>	Insert a register content	<code>insert_register</code>
<code>Ctrl-w</code> , <code>Alt-Backspace</code>	Delete previous word	<code>delete_word_backward</code>
<code>Alt-d</code> , <code>Alt-Delete</code>	Delete next word	<code>delete_word_forward</code>
<code>Ctrl-u</code>	Delete to start of line	<code>kill_to_line_start</code>
<code>Ctrl-k</code>	Delete to end of line	<code>kill_to_line_end</code>
<code>Ctrl-h</code> , <code>Backspace</code> , <code>Shift-Backspace</code>	Delete previous char	<code>delete_char_backward</code>
<code>Ctrl-d</code> , <code>Delete</code>	Delete next char	<code>delete_char_forward</code>
<code>Ctrl-j</code> , <code>Enter</code>	Insert new line	<code>insert_newline</code>
<code>Tab</code>	Advance a snippet tab stop, else expand a snippet trigger or emmet abbreviation, else <code>smart tab</code>	<code>emmet_expand</code>
<code>Shift-Tab</code>	Insert tab	<code>insert_tab</code>

- `Tab` is overloaded in insert mode: if a snippet is active it jumps to the next tab stop; otherwise it tries to expand the word before the cursor as a `snippet` trigger or an emmet abbreviation; failing both, it behaves as a `smart tab`.

These keys are not recommended, but are included for new users less familiar with modal editors.

Key	Description	Command
<code>Up</code>	Move to previous line	<code>move_line_up</code>
<code>Down</code>	Move to next line	<code>move_line_down</code>
<code>Left</code>	Backward a char	<code>move_char_left</code>
<code>Right</code>	Forward a char	<code>move_char_right</code>
<code>PageUp</code>	Move one page up	<code>page_up</code>
<code>PageDown</code>	Move one page down	<code>page_down</code>
<code>Home</code>	Move to line start	<code>goto_line_start</code>

Key	Description	Command
End	Move to line end	<code>goto_line_end_newline</code>

As you become more comfortable with modal editing, you may want to disable some insert mode bindings. You can do this by editing your `config.toml` file.

```
[keys.insert]
up = "no_op"
down = "no_op"
left = "no_op"
right = "no_op"
pageup = "no_op"
pagedown = "no_op"
home = "no_op"
end = "no_op"
```

Select / extend mode

Accessed by typing `v` in [normal mode](#).

Select mode echoes Normal mode, but changes any movements to extend selections rather than replace them. Goto motions are also changed to extend, so that `vg|`, for example, extends the selection to the end of the line.

Search is also affected. By default, `n` and `N` will remove the current selection and select the next instance of the search term. Toggling this mode before pressing `n` or `N` makes it possible to keep the current selection. Toggling it on and off during your iterative searching allows you to selectively add search terms to your selections.

Picker

Keys to use within picker. Remapping currently not supported. See the documentation page on [pickers](#) for more info. [Prompt](#) keybinds also work in pickers, except where they conflict with picker keybinds.

Key	Description
<code>Shift-Tab</code> , <code>Up</code> , <code>Ctrl-p</code>	Previous entry
<code>Tab</code> , <code>Down</code> , <code>Ctrl-n</code>	Next entry
<code>PageUp</code> , <code>Ctrl-u</code>	Page up
<code>PageDown</code> , <code>Ctrl-d</code>	Page down
<code>Home</code>	Go to first entry
<code>End</code>	Go to last entry
<code>Enter</code>	Open selected
<code>Alt-Enter</code>	Open selected in the background without closing the picker
<code>Ctrl-s</code>	Open horizontally
<code>Ctrl-v</code>	Open vertically
<code>Ctrl-t</code>	Toggle preview
<code>Escape</code> , <code>Ctrl-c</code>	Close picker

Prompt

Keys to use within prompt, Remapping currently not supported.

Key	Description
Escape, Ctrl-c	Close prompt
Alt-b, Ctrl-Left	Backward a word
Ctrl-b, Left	Backward a char
Alt-f, Ctrl-Right	Forward a word
Ctrl-f, Right	Forward a char
Ctrl-e, End	Move prompt end
Ctrl-a, Home	Move prompt start
Ctrl-w, Alt-Backspace, Ctrl-Backspace	Delete previous word
Alt-d, Alt-Delete, Ctrl-Delete	Delete next word
Ctrl-u	Delete to start of line
Ctrl-k	Delete to end of line
Backspace, Ctrl-h, Shift-Backspace	Delete previous char
Delete, Ctrl-d	Delete next char
Ctrl-s	Insert a word under doc cursor, may be changed to Ctrl-r Ctrl-w later
Ctrl-p, Up	Select previous history
Ctrl-n, Down	Select next history
Ctrl-r	Insert the content of the register selected by following input char
Tab	Select next completion item
BackTab	Select previous completion item
Enter	Open selected

Configuration and Settings

Editor

- `[editor]` Section
- `[editor.clipboard-provider]` Section
- `[editor.statusline]` Section
- `[editor.lsp]` Section
- `[editor.cursor-shape]` Section
- `[editor.file-picker]` Section
- `[editor.file-explorer]` Section
- `[editor.buffer-picker]` Section
- `[editor.auto-pairs]` Section
- `[editor.auto-save]` Section
- `[editor.search]` Section
- `[editor.whitespace]` Section
- `[editor.indent-guides]` Section
- `[editor.gutters]` Section
 - `[editor.gutters.line-numbers]` Section
 - `[editor.gutters.diagnostics]` Section
 - `[editor.gutters.diff]` Section
 - `[editor.gutters.spacer]` Section
 - `[editor.gutters.code-action-hint]` Section
- `[editor.soft-wrap]` Section
- `[editor.smart-tab]` Section
- `[editor.inline-diagnostics]` Section
- `[editor.word-completion]` Section
- `[editor.workspace-trust]` Section

`[editor]` Section

Key	Description	Default
<code>scrolloff</code>	Number of lines of padding around the edge of the screen when scrolling. <code>0</code> (default) matches vim/spacemacs — <code>H/L</code> reach the literal top/bottom row; set to e.g. <code>5</code> for Helix-style scroll margins	<code>0</code>
<code>mouse</code>	Enable mouse mode	<code>true</code>
<code>mouse-yank-register</code>	Which register to use for mouse yanks.	<code>*</code>
<code>middle-click-paste</code>	Middle click paste support	<code>true</code>

Key	Description	Default
<code>default-yank-register</code>	Default register used for yank/paste	<code>'</code>
<code>scroll-lines</code>	Number of lines to scroll per scroll wheel step	<code>3</code>
<code>shell</code>	Shell to use when running external commands	Unix: <code>["sh", "-c"]</code> Windows: <code>["cmd", "/C"]</code>
<code>line-number</code>	Line number display: <code>absolute</code> simply shows each line's number, while <code>relative</code> shows the distance from the current line. When unfocused or in insert mode, <code>relative</code> will still show absolute line numbers	<code>"absolute"</code>
<code>cursorline</code>	Highlight all lines with a cursor	<code>false</code>
<code>cursorcolumn</code>	Highlight all columns with a cursor	<code>false</code>
<code>highlight-word-under-cursor</code>	Highlight every whole-word occurrence of the word under the primary cursor within the visible viewport (styled with the <code>ui.highlight.word</code> theme scope)	<code>true</code>
<code>continue-comments</code>	if zemacs should automatically add a line comment token if you create a new line inside a comment.	<code>true</code>
<code>gutters</code>	Gutters to display: Available are <code>diagnostics</code> and <code>diff</code> and <code>line-numbers</code> and <code>spacer</code> and <code>code-action-hint</code> , note that <code>diagnostics</code> also includes other features like breakpoints, 1-width padding will be inserted if gutters is non-empty	<code>["diagnostics", "spacer", "line-numbers", "spacer", "diff"]</code>

Key	Description	Default
<code>auto-completion</code>	Enable automatic pop up of auto-completion	<code>true</code>
<code>path-completion</code>	Enable filepath completion. Show files and directories if an existing path at the cursor was recognized, either absolute or relative to the current opened document or current working directory (if the buffer is not yet saved). Defaults to true.	<code>true</code>
<code>auto-format</code>	Enable automatic formatting on save ¹	<code>true</code>
<code>idle-timeout</code>	Time in milliseconds since last keypress before idle timers trigger.	<code>250</code>
<code>completion-timeout</code>	Time in milliseconds after typing a word character before completions are shown, set to 5 for instant.	<code>250</code>
<code>preview-completion-insert</code>	Whether to apply completion item instantly when selected	<code>true</code>
<code>completion-trigger-len</code>	The min-length of word under cursor to trigger autocompletion	<code>2</code>
<code>completion-replace</code>	Whether to make completions always replace the entire word and not just the part before the cursor	<code>false</code>
<code>auto-info</code>	Whether to display info boxes	<code>true</code>

¹In most cases, you also need to enable the `auto-format` setting under `languages.toml`. You can find the reasoning [here](#).

Key	Description	Default
<code>true-color</code>	Whether to override automatic detection of terminal truecolor support in the event of a false negative	<code>false</code>
<code>transparent-background</code>	Don't paint the <code>ui.background</code> fill (editor surface, gutter/sign column, and the IDE file-tree sidebar), letting the terminal's own (e.g. translucent/blurred) background show through. Themed chrome — statusline, breadcrumbs, panel status bars — keeps its own colors	<code>false</code>
<code>undercurl</code>	Whether to override automatic detection of terminal undercurl support in the event of a false negative	<code>false</code>
<code>rulers</code>	List of column positions at which to display the rulers. Can be overridden by language specific <code>rulers</code> in <code>languages.toml</code> file	<code>[]</code>
<code>bufferline</code>	Renders a line at the top of the editor displaying open buffers. Can be <code>always</code> , <code>never</code> or <code>multiple</code> (only shown if more than one buffer is in use)	<code>"never"</code>
<code>color-modes</code>	Whether to color the mode indicator with different colors depending on the mode itself	<code>false</code>

Key	Description	Default
<code>text-width</code>	Maximum line length. Used for the <code>:reflow</code> command and soft-wrapping if <code>soft-wrap.wrap-at-text-width</code> is set	<code>80</code>
<code>workspace-lsp-roots</code>	Directories relative to the workspace root that are treated as LSP roots. Should only be set in <code>.zemacs/config.toml</code>	<code>[]</code>
<code>default-line-ending</code>	The line ending to use for new documents. Can be <code>native</code> , <code>lf</code> , <code>crlf</code> , <code>ff</code> , <code>cr</code> or <code>nel</code> . <code>native</code> uses the platform's native line ending (<code>crlf</code> on Windows, otherwise <code>lf</code>).	<code>"native"</code>
<code>insert-final-newline</code>	Whether to automatically insert a trailing line-ending on write if missing	<code>true</code>
<code>atomic-save</code>	Whether to use atomic operations to write documents to disk. This prevents data loss if the editor is interrupted while writing the file, but may confuse some file watching/hot reloading programs.	<code>true</code>
<code>trim-final-newlines</code>	Whether to automatically remove line-endings after the final one on write	<code>false</code>
<code>trim-trailing-whitespace</code>	Whether to automatically remove whitespace preceding line endings on write	<code>false</code>

Key	Description	Default
<code>popup-border</code>	Draw border around <code>popup</code> , <code>menu</code> , <code>all</code> , or <code>none</code>	"none"
<code>indent-heuristic</code>	How the indentation for a newly inserted line is computed: <code>simple</code> just copies the indentation level from the previous line, <code>tree-sitter</code> computes the indentation based on the syntax tree and <code>hybrid</code> combines both approaches. If the chosen heuristic is not available, a different one will be used as a fallback (the fallback order being <code>hybrid</code> -> <code>tree-sitter</code> -> <code>simple</code>).	"hybrid"
<code>jump-label-alphabet</code>	The characters that are used to generate two character jump labels. Characters at the start of the alphabet are used first.	"abcdefghijklmnopqrstuvwxyz"
<code>end-of-line-diagnostics</code>	Minimum severity of diagnostics to render at the end of the line. Set to <code>disable</code> to disable entirely. Refer to the setting about <code>inline-diagnostics</code> for more details	"hint"

Key	Description	Default
<code>clipboard-provider</code>	Which API to use for clipboard interaction. One of <code>pasteboard</code> (MacOS), <code>wayland</code> , <code>x-clip</code> , <code>x-sel</code> , <code>win32-yank</code> , <code>termux</code> , <code>tmux</code> , <code>windows</code> , <code>termcode</code> , <code>none</code> , or a custom command set.	Platform and environment specific.
<code>editor-config</code>	Whether to read settings from <code>EditorConfig</code> files	<code>true</code>
<code>rainbow-brackets</code>	Whether to render rainbow colors for matching brackets. Requires tree-sitter <code>rainbows.scm</code> queries for the language.	<code>false</code>
<code>kitty-keyboard-protocol</code>	Whether to enable Kitty Keyboard Protocol. Can be <code>enabled</code> , <code>disabled</code> or <code>auto</code>	<code>"auto"</code>
<code>startup</code>	What to open when launched with no file arguments. <code>startify</code> shows the start screen, <code>recent</code> opens the most-recently-used file, <code>session</code> restores the previous session's tabs and cursor, <code>file</code> opens <code>startup-file</code> . Each falls back to <code>startify</code> when its target is unavailable.	<code>"startify"</code>
<code>startup-file</code>	File to open on launch when <code>startup</code> is <code>"file"</code> . Ignored otherwise.	<code>" "</code>

Key	Description	Default
<code>sync-zwire-theme</code>	Bidirectionally sync the colorscheme with the <code>zwire</code> terminal host. When on, <code>zemacs</code> reads <code>~/.zwire/global.toml</code> (<code>[theme]</code> <code>scheme + [theme.ui] light</code>) and applies the matching <code>zgui-<scheme> / zgui-<scheme>-light</code> theme at startup, on <code>:config-reload</code>, and live via a filesystem watcher on <code>global.toml</code> (applied immediately on change, no keypress needed) — overriding the configured/persisted theme. Conversely, committing a <code>zgui-*</code> theme in <code>zemacs</code> (<code>:theme</code>, the picker, <code>:theme-toggle</code>) writes the scheme back into <code>global.toml</code>, which <code>zwire</code> fans out to the browser/HUD. Non-<code>zgui-*</code> themes are never pushed to <code>zwire</code>.	<code>false</code>

`[editor.clipboard-provider]` Section

`Zemacs` can be configured either to use a builtin clipboard configuration or to use a provided command.

For instance, setting it to use OSC 52 termcodes, the configuration would be:

```
[editor]
clipboard-provider = "termcode"
```

Alternatively, `Zemacs` can be configured to use arbitrary commands for clipboard integration:

```
[editor.clipboard-provider.custom]
yank = { command = "cat", args = ["test.txt"] }
paste = { command = "tee", args = ["test.txt"] }
primary-yank = { command = "cat", args = ["test-primary.txt"] } # optional
primary-paste = { command = "tee", args = ["test-primary.txt"] } # optional
```

For custom commands the contents of the yank/paste is communicated over stdin/stdout.

[editor.statusline] Section

Allows configuring the statusline at the bottom of the editor.

The configuration distinguishes between three areas of the status line:

```
[ ... .. LEFT ... .. | ... .. .. CENTER ... .. .. | ... .. RIGHT ... .. ]
```

Statusline elements can be defined as follows:

```
[editor.statusline]
left = ["mode", "spinner"]
center = ["file-name"]
right = ["diagnostics", "selections", "position", "file-encoding", "file-line-ending", "file-type"]
separator = "|"
mode.normal = "NORMAL"
mode.insert = "INSERT"
mode.select = "SELECT"
diagnostics = ["warning", "error"]
workspace-diagnostics = ["warning", "error"]
```

The [editor.statusline] key takes the following sub-keys:

Key	Description	Default
left	A list of elements aligned to the left of the statusline	["mode", "spinner", "file-name", "read-only-indicator", "file-modification-indicator"]
center	A list of elements aligned to the middle of the statusline	[]
right	A list of elements aligned to the right of the statusline	["diagnostics", "selections", "register", "position", "file-encoding"]
separator	The character used to separate elements in the statusline	" "
mode.normal	The text shown in the <code>mode</code> element for normal mode	"NOR"
mode.insert	The text shown in the <code>mode</code> element for insert mode	"INS"
mode.select	The text shown in the <code>mode</code> element for select mode	"SEL"
diagnostics	A list of severities which are displayed for the current buffer	["warning", "error"]
workspace-diagnostics	A list of severities which are displayed for the workspace	["warning", "error"]

The following statusline elements can be configured:

Key	Description
mode	The current editor mode (<code>mode.normal</code> / <code>mode.insert</code> / <code>mode.select</code>)
spinner	A progress spinner indicating LSP activity
file-name	The path/name of the opened file

Key	Description
<code>file-absolute-path</code>	The absolute path/name of the opened file
<code>file-base-name</code>	The basename of the opened file
<code>current-working-directory</code>	The current working directory
<code>file-modification-indicator</code>	The indicator to show whether the file is modified (a <code>[+]</code> appears when there are unsaved changes)
<code>file-encoding</code>	The encoding of the opened file if it differs from UTF-8
<code>file-line-ending</code>	The file line endings (CRLF or LF)
<code>file-indent-style</code>	The file indentation style
<code>read-only-indicator</code>	An indicator that shows <code>[readonly]</code> when a file cannot be written
<code>total-line-numbers</code>	The total line numbers of the opened file
<code>file-type</code>	The type of the opened file
<code>diagnostics</code>	The number of warnings and/or errors
<code>workspace-diagnostics</code>	The number of warnings and/or errors on workspace
<code>selections</code>	The primary selection index out of the number of active selections
<code>primary-selection-length</code>	The number of characters currently in primary selection
<code>position</code>	The cursor position
<code>position-percentage</code>	The cursor position as a percentage of the total number of lines
<code>separator</code>	The string defined in <code>editor.statusline.separator</code> (defaults to " ")
<code>spacer</code>	Inserts a space between elements (multiple/contiguous spacers may be specified)
<code>version-control</code>	The current branch name or detached commit hash of the opened workspace
<code>register</code>	The current selected register
<code>code-action-hint</code>	Indicator for when code actions are available

[`editor.lsp`] Section

Key	Description	Default
<code>enable</code>	Enables LSP integration. Setting to false will completely disable language servers regardless of language settings.	<code>true</code>
<code>display-messages</code>	Display LSP <code>window/showMessage</code> messages below statusline ²	<code>true</code>
<code>display-progress-messages</code>	Display LSP progress messages below statusline ³	<code>false</code>
<code>auto-signature-help</code>	Enable automatic popup of signature help (parameter hints)	<code>true</code>

²By default, a progress spinner is shown in the statusline beside the file path.

³By default, a progress spinner is shown in the statusline beside the file path.

Key	Description	Default
<code>auto-document-highlight</code>	Automatically highlight symbol references at the cursor	<code>false</code>
<code>display-inlay-hints</code>	Display inlay hints ⁴	<code>false</code>
<code>inlay-hints-length-limit</code>	Maximum displayed length (non-zero number) of inlay hints	Unset by default
<code>display-color-swatches</code>	Show color swatches next to colors	<code>true</code>
<code>display-signature-help-docs</code>	Display docs under signature help popup	<code>true</code>
<code>snippets</code>	Enables snippet completions. Requires a server restart (<code>:lsp-restart</code>) to take effect after <code>:config-reload/:set</code> . Include declaration in the goto references popup.	<code>true</code>
<code>goto-reference-include-declaration</code>		<code>true</code>

[`editor.cursor-shape`] Section

Defines the shape of cursor in each mode. Valid values for these options are `block`, `bar`, `underline`, or `hidden`.

- Due to limitations of the terminal environment, only the primary cursor can change shape.

Key	Description	Default
<code>normal</code>	Cursor shape in <code>normal mode</code>	<code>"block"</code>
<code>insert</code>	Cursor shape in <code>insert mode</code>	<code>"block"</code>
<code>select</code>	Cursor shape in <code>select mode</code>	<code>"block"</code>

[`editor.file-picker`] Section

Set options for file picker and global search. Ignoring a file means it is not visible in the Zemacs file picker and global search.

All git related options are only enabled in a git repository.

Key	Description	Default
<code>hidden</code>	Enables ignoring hidden files	<code>true</code>
<code>follow-symlinks</code>	Follow symlinks instead of ignoring them	<code>true</code>
<code>deduplicate-links</code>	Ignore symlinks that point at files already shown in the picker	<code>true</code>
<code>parents</code>	Enables reading ignore files from parent directories	<code>true</code>

⁴You may also have to activate them in the language server config for them to appear, not just in Zemacs. Inlay hints in Zemacs are still being improved on and may be a little bit laggy/janky under some circumstances. Please report any bugs you see so we can fix them!

Key	Description	Default
<code>ignore</code>	Enables reading <code>.ignore</code> files	<code>true</code>
<code>git-ignore</code>	Enables reading <code>.gitignore</code> files	<code>true</code>
<code>git-global</code>	Enables reading global <code>.gitignore</code> , whose path is specified in git's config: <code>core.excludesfile</code> option	<code>true</code>
<code>git-exclude</code>	Enables reading <code>.git/info/exclude</code> files	<code>true</code>
<code>max-depth</code>	Set with an integer value for maximum depth to recurse	Unset by default

Ignore files can be placed locally as `.ignore` or put in your home directory as `~/.ignore`. They support the usual ignore and negative ignore (unignore) rules used in `.gitignore` files.

Additionally, you can use Emacs-specific ignore files by creating a local `.emacs/ignore` file in the current workspace or a global `ignore` file located in your Emacs config directory: - Linux and Mac: `~/.emacs/ignore` - Windows: `%AppData%\emacs\ignore`

Example:

```
# unignore in file picker and global search
!.github/
!.gitignore
!.gitattributes
```

[`editor.file-explorer`] Section

In addition to the options for the file picker and global search, a similar set of options is presented to configure the file explorer separately. However, unlike the file picker, the defaults are set to avoid ignoring most files.

Note that the ignore files consulted by the file explorer when `ignore` is set to true are the same ones used by the file picker, including the aforementioned Emacs-specific ignore files.

Key	Description	Default
<code>hidden</code>	Enables ignoring hidden files	<code>false</code>
<code>follow-symlinks</code>	Follow symlinks instead of ignoring them	<code>false</code>
<code>parents</code>	Enables reading ignore files from parent directories	<code>false</code>
<code>ignore</code>	Enables reading <code>.ignore</code> files	<code>false</code>
<code>git-ignore</code>	Enables reading <code>.gitignore</code> files	<code>false</code>

Key	Description	Default
<code>git-global</code>	Enables reading global <code>.gitignore</code> , whose path is specified in git's config: <code>core.excludesfile</code> option	<code>false</code>
<code>git-exclude</code>	Enables reading <code>.git/info/exclude</code> files	<code>false</code>
<code>flatten-dirs</code>	Enables flattening single child directories	<code>true</code>

[editor.buffer-picker] Section

Set options for buffer picker.

Key	Description	Default
<code>start-position</code>	Controls behavior for which buffer is initially selected	<code>current</code>

Example

```
[editor.buffer-picker]
start-position = "previous"
```

[editor.auto-pairs] Section

Enables automatic insertion of pairs to parentheses, brackets, etc. Can be a simple boolean value, or a specific mapping of pairs of single characters.

To disable auto-pairs altogether, set `auto-pairs` to `false`:

```
[editor]
auto-pairs = false # defaults to `true`
```

The default pairs are `(){}""'</code>`, but these can be customized by setting `auto-pairs` to a TOML table:

```
[editor.auto-pairs]
'(' = ')'
'{' = '}'
'[' = ']'
'"' = '"'
'\'' = '\''
'<' = '>'
```

Additionally, this setting can be used in a language config. Unless the editor setting is `false`, this will override the editor config in documents with this language.

Example `languages.toml` that adds `<>` and removes `' '`

```
[[language]]
name = "rust"

[language.auto-pairs]
'(' = ')'
'{' = '}'
'[' = ']'
'"' = '"'
'`' = '`'
'<' = '>'
```

[editor.auto-save] Section

Control auto save behavior.

Key	Description	Default
<code>focus-lost</code>	Enable automatic saving on the focus moving away from zemacs. Requires <code>focus event support</code> from your terminal	<code>false</code>
<code>after-delay.enable</code>	Enable automatic saving after <code>auto-save.after-delay.timeout</code> milliseconds have passed since last edit.	<code>false</code>
<code>after-delay.timeout</code>	Time in milliseconds since last edit before auto save timer triggers.	<code>3000</code>

[editor.search] Section

Search specific options.

Key	Description	Default
<code>smart-case</code>	Enable smart case regex searching (case-insensitive unless pattern contains upper case characters)	<code>true</code>
<code>wrap-around</code>	Whether the search should wrap after depleting the matches	<code>true</code>

[editor.whitespace] Section

Options for rendering whitespace with visible characters. Use `:set whitespace.render all` to temporarily enable visible whitespace.

Key	Description	Default
<code>render</code>	Whether to render whitespace. May either be <code>all</code> or <code>none</code> , or a table with sub-keys <code>space</code> , <code>nbspace</code> , <code>nnbspace</code> , <code>tab</code> , and <code>newline</code>	<code>"none"</code>
<code>characters</code>	Literal characters to use when rendering whitespace. Sub-keys may be any of <code>tab</code> , <code>space</code> , <code>nbspace</code> , <code>nnbspace</code> , <code>newline</code> or <code>tabpad</code>	See example below

Example

```
[editor.whitespace]
render = "all"
# or control each character
[editor.whitespace.render]
space = "all"
tab = "all"
nbspace = "none"
nnbspace = "none"
newline = "none"

[editor.whitespace.characters]
space = " "
nbspace = "␣"
nnbspace = "␣"
tab = "→"
newline = "↵"
tabpad = " " # Tabs will look like "→..." (depending on tab width)
```

[editor.indent-guides] Section

Options for rendering vertical indent guides.

Key	Description	Default
<code>render</code>	Whether to render indent guides	<code>false</code>
<code>character</code>	Literal character to use for rendering the indent guide	<code>" "</code>
<code>skip-levels</code>	Number of indent levels to skip	<code>0</code>

Example:

```
[editor.indent-guides]
render = true
character = "|" # Some characters that work well: "|", "⋈", "⋊", "⋋"
skip-levels = 1
```

[editor.gutters] Section

For simplicity, `editor.gutters` accepts an array of gutter types, which will use default settings for all gutter components.

```
[editor]
gutters = ["diff", "diagnostics", "line-numbers", "spacer"]
```

To customize the behavior of gutters, the `[editor.gutters]` section must be used. This section contains top level settings, as well as settings for specific gutter components as subsections.

Key	Description	Default
<code>layout</code>	A vector of gutters to display	<code>["blame", "diagnostics", "marks", "signs", "spacer", "line-numbers", "spacer", "diff"]</code>

Available gutter types: `diagnostics`, `line-numbers`, `spacer`, `diff`, `code-action-hint`, `marks`, `signs`, and `blame`. The `blame` gutter is a JetBrains-style “Annotate” column (author + relative time per line); it takes no space until toggled on with `SPC g B (toggle_blame_annotate)`. The `signs` gutter shows vim signs placed with `:sign place`; it takes no space until a sign is placed in the file (vim’s `signcolumn=auto`).

Example:

```
[editor.gutters]
layout = ["diff", "diagnostics", "line-numbers", "spacer"]
```

[editor.gutters.line-numbers] Section

Options for the line number gutter

Key	Description	Default
<code>min-width</code>	The minimum number of characters to use	<code>3</code>

Example:

```
[editor.gutters.line-numbers]
min-width = 1
```

[editor.gutters.diagnostics] Section

Currently unused

[editor.gutters.diff] Section

The `diff` gutter option displays colored bars indicating whether a `git` diff represents that a line was added, removed or changed. These colors are controlled by the theme attributes `diff.plus`, `diff.minus` and `diff.delta`.

Other diff providers will eventually be supported by a future plugin system.

There are currently no options for this section.

[editor.gutters.spacer] Section

Currently unused

[editor.gutters.code-action-hint] Section

The `code-action-hint` gutter option displays an indicator for whether a code action is available at current selection.

There are currently no options for this section.

[editor.soft-wrap] Section

Options for soft wrapping lines that exceed the view width:

Key	Description	Default
<code>enable</code>	Whether soft wrapping is enabled.	<code>false</code>
<code>max-wrap</code>	Maximum free space left at the end of the line.	<code>20</code>

Key	Description	Default
<code>max-indent-retain</code>	Maximum indentation to carry over when soft wrapping a line.	<code>40</code>
<code>wrap-indicator</code>	Text inserted before soft wrapped lines, highlighted with <code>ui.virtual.wrap</code>	<code>"↪ "</code>
<code>wrap-at-text-width</code>	Soft wrap at <code>text-width</code> instead of using the full viewport size.	<code>false</code>

Example:

```
[editor.soft-wrap]
enable = true
max-wrap = 25 # increase value to reduce forced mid-word wrapping
max-indent-retain = 0
wrap-indicator = "" # set wrap-indicator to "" to hide it
```

[editor.smart-tab] Section

Options for navigating and editing using tab key.

Key	Description	Default
<code>enable</code>	If set to true, then when the cursor is in a position with non-whitespace to its left, instead of inserting a tab, it will run <code>move_parent_node_end</code> . If there is only whitespace to the left, then it inserts a tab as normal. With the default bindings, to explicitly insert a tab character, press Shift-tab.	<code>true</code>
<code>supersede-menu</code>	Normally, when a menu is on screen, such as when auto complete is triggered, the tab key is bound to cycling through the items. This means when menus are on screen, one cannot use the tab key to trigger the <code>smart-tab</code> command. If this option is set to true, the <code>smart-tab</code> command always takes precedence, which means one cannot use the tab key to cycle through menu items. One of the other bindings must be used instead, such as arrow keys or <code>C-n/C-p</code> .	<code>false</code>

Due to lack of support for S-tab in some terminals, the default keybindings don't fully embrace smart-tab editing experience. If you enjoy smart-tab navigation and a terminal that supports the [Enhanced Keyboard protocol](#), consider setting extra keybindings:

```
[keys.normal]
tab = "move_parent_node_end"
S-tab = "move_parent_node_start"

[keys.insert]
S-tab = "move_parent_node_start"

[keys.select]
tab = "extend_parent_node_end"
S-tab = "extend_parent_node_start"
```

[editor.inline-diagnostics] Section

Options for rendering diagnostics inside the text like shown below

```
fn main() {
  let foo = bar;
    └─ no such value in this scope
}
```

Key	Description	Default
<code>cursor-line</code>	The minimum severity that a diagnostic must have to be shown inline on the line that contains the primary cursor. Set to <code>disable</code> to not show any diagnostics inline. This option does not have any effect when in insert-mode and will only take effect 350ms after moving the cursor to a different line.	"warning"
<code>other-lines</code>	The minimum severity that a diagnostic must have to be shown inline on a line that does not contain the cursor-line. Set to <code>disable</code> to not show any diagnostics inline.	"disable"
<code>prefix-len</code>	How many horizontal bars <code>─</code> are rendered before the diagnostic text.	1
<code>max-wrap</code>	Equivalent of the <code>editor.soft-wrap.max-wrap</code> option for diagnostics.	20
<code>max-diagnostics</code>	Maximum number of diagnostics to render inline for a given line	10

The allowed values for `cursor-line` and `other-lines` are: `error`, `warning`, `info`, `hint`.

The (first) diagnostic with the highest severity that is not shown inline is rendered at the end of the line (as long as its severity is higher than the `end-of-line-diagnostics` config option):

```
fn main() {
  let baz = 1;
  let foo = bar; a local variable with a similar name exists: baz
    └─ no such value in this scope
}
```

[`editor.word-completion`] Section

Options for controlling completion of words from open buffers.

Key	Description	Default
<code>enable</code>	Whether word completion is enabled	<code>true</code>
<code>trigger-length</code>	Number of word characters to type before triggering completion	7

Example:

```
[editor.word-completion]
enable = true
# Set the trigger length lower so that words are completed more often
trigger-length = 4
```

[`editor.workspace-trust`] Section

Controls implicit workspace trust. See the [workspace trust](#) chapter for the full feature.

Key	Description	Default
<code>level</code>	The default level of trust for every workspace.	<code>"servers"</code>
<code>prompt</code>	Whether to show a modal when opening a file in an untrusted workspace.	<code>true</code>
<code>trusted</code>	Glob patterns whose matching workspaces are trusted without a grant.	<code>[]</code>

Example:

```
[editor workspace-trust]
# Even if `false`, the statusline `[△]` indicator is still shown.
prompt = false

# "none":    prompt for every workspace.
# "servers": trust LSP and DAP launches but still gate local config and git;
#            .zemacs/config.toml, .zemacs/languages.toml, etc. need :workspace-trust.
# "insecure": trust everything (discouraged).
level = "servers"

# Discouraged: skips .zemacs/ change detection and trusts anything that lands
# under a matching path. `~` and environment variables are expanded.
trusted = ["~/src/github.com/me/*"]
```

Typable Commands

Commands run from the `:` command line, with their aliases.

Name	Description
<code>:terminal</code> , <code>:term</code>	Open an integrated terminal (PTY shell) running \$SHELL.
<code>:ide</code> , <code>:workbench</code>	Enter IDE mode (file-tree sidebar + panels, like <code>--ide</code> / F2).
<code>:diff</code> , <code>:gdiff</code>	Open a read-only side-by-side diff of the buffer vs. its git HEAD version.
<code>:diff-buffer-with-file</code>	Show a unified diff of the buffer's contents vs. its file on disk (emacs diff-buffer-with-file).
<code>:reveal</code> , <code>:browse</code> , <code>:open-repo</code>	Open this repository's homepage (GitHub/GitLab/Bitbucket/...) in the browser.
<code>:compare-ref</code> , <code>:compare-branch</code>	Diff the buffer against its version at a git ref (JetBrains Compare with Branch).
<code>:merge</code> , <code>:resolve</code>	Resolve the buffer's git merge conflicts in a 3-pane (ours/result/theirs) view.
<code>:magit</code> , <code>:git</code> , <code>:gst</code>	Open the Magit-style git status (stage/unstage/discard/commit changes by section).
<code>:hex</code> , <code>:hexview</code> , <code>:hexedit</code>	Open a read-only xxd-style hex viewer of a file's raw bytes (optional path; defaults to the buffer's file).
<code>:snippets</code> , <code>:snip</code>	Open the user snippet library editor (create/edit/delete reusable snippets).
<code>:org-cycle</code> , <code>:org-fold</code>	Toggle a fold over the current org heading's subtree (TAB-style outline cycling).
<code>:org-todo</code>	Cycle the current org heading's TODO keyword: none -> TODO -> DONE -> none.
<code>:org-promote</code>	Promote the current org heading one level (remove a leading star).
<code>:org-demote</code>	Demote the current org heading one level (add a leading star).
<code>:org-move-subtree-down</code> , <code>:org-metadown</code>	Move the org subtree at point down past its next sibling (emacs org-move-subtree-down).
<code>:org-move-subtree-up</code> , <code>:org-metaup</code>	Move the org subtree at point up past its previous sibling (emacs org-move-subtree-up).
<code>:org-schedule</code>	Add/update a SCHEDULED: timestamp on the heading at point: <code>:org-schedule 2026-07-15</code> (emacs org-schedule).
<code>:org-deadline</code>	Add/update a DEADLINE: timestamp on the heading at point: <code>:org-deadline 2026-07-15</code> (emacs org-deadline).
<code>:org-next-heading</code>	Move the cursor to the next org heading line.
<code>:org-prev-heading</code>	Move the cursor to the previous org heading line.
<code>:org-fold-all</code>	Fold every org heading subtree in the buffer.
<code>:org-unfold-all</code>	Unfold every fold in the buffer.
<code>:org-agenda</code> , <code>:agenda</code>	Open the org agenda: TODO/DONE items across open .org buffers and *.org files under the working directory, grouped by scheduled/deadline date.
<code>:org-priority</code>	Cycle the current org heading's priority cookie: none -> [#A] -> [#B] -> [#C] -> none.

Name	Description
<code>:org-capture, :capture</code>	Prompt for a line of text and append it as a '* TODO ' entry to an inbox org file (default /inbox.org, or an explicit path argument).
<code>:exit, :x, :xit</code>	Write changes to disk if the buffer is modified and then quit. Accepts an optional path (:exit some/path.txt).
<code>:exit!, :x!, :xit!</code>	Force write changes to disk, creating necessary subdirectories, if the buffer is modified and then quit. Accepts an optional path (:exit! some/path.txt).
<code>:quit, :q</code>	Close the current view.
<code>:help, :h</code>	Open the inline Help browser (searchable: commands, keybindings, topics).
<code>:wc, :words, :count</code>	Show document line/word/char counts (and selection stats).
<code>:blame</code>	Show git blame for the current line in the status bar.
<code>:reopen, :reopen-closed</code>	Reopen the most recently closed file.
<code>:zen</code>	Toggle the IDE workbench (Zen / focus mode).
<code>:emmet, :encode</code>	Expand the emmet/zen HTML abbreviation before the cursor.
<code>:quit!, :q!</code>	Force close the current view, ignoring unsaved changes.
<code>:open, :o, :edit, :e, :ex, :visual</code>	Open a file from disk into the current view (vim :edit; :ex/:visual have no separate Ex mode here).
<code>:args, :ar, :argglobal, :argg, :arglocal, :argl</code>	Show the argument list, or set it to the given files and edit the first (vim :args).
<code>:argadd, :arga</code>	Add files to the argument list after the current entry (vim :argadd).
<code>:argedit, :arge</code>	Add a file to the argument list and edit it (vim :argedit).
<code>:argdelete, :argd</code>	Delete argument-list entries matching the given glob patterns (vim :argdelete).
<code>:argdedupe</code>	Remove duplicate entries from the argument list (vim :argdedupe).
<code>:next, :argnext</code>	Edit the next file in the argument list (vim :next).
<code>:previous, :Next, :prev, :argprev</code>	Edit the previous file in the argument list (vim :previous / :Next).
<code>:wnext, :wn</code>	Write the current buffer, then edit the next file in the argument list (vim :wnext).
<code>:wprevious, :wp, :wNext, :wN</code>	Write the current buffer, then edit the previous file in the argument list (vim :wprevious / :wNext).
<code>:find, :fin</code>	Find a file in the 'path' (buffer dir, cwd, then a recursive cwd walk) and edit it (vim :find).
<code>:sfind, :sf</code>	Split the window and edit a file found in the 'path' (vim :sfind).
<code>:first, :rewind, :rew</code>	Edit the first file in the argument list (vim :first / :rewind).
<code>:view, :vie, :find-file-read-only</code>	Edit a file read-only (vim :view / emacs find-file-read-only).
<code>:sview, :svie, :find-file-read-only-other-window</code>	Split and edit a file read-only (vim :sview / emacs find-file-read-only-other-window).
<code>:autocmd, :au</code>	Register an autocommand: :autocmd {events} {pattern} {command} (:autocmd ! clears).
<code>:last, :la</code>	Edit the last file in the argument list (vim :last).
<code>:argument, :argu</code>	Edit the Nth file in the argument list (vim :argument).
<code>:snext, :sn</code>	Split the window and edit the next argument (vim :snext).
<code>:sprevious, :sprev, :sNext, :sN</code>	Split the window and edit the previous argument (vim :sprevious / :sNext).

Name	Description
<code>:srewind, :sre, :sfirst, :sfir</code>	Split the window and edit the first argument (vim :srewind / :sfirst).
<code>:slast, :sla</code>	Split the window and edit the last argument (vim :slast).
<code>:sargument, :sa</code>	Split the window and edit the Nth argument (vim :sargument).
<code>:argdo</code>	Run an Ex command on each file in the argument list (vim :argdo).
<code>:all, :sall</code>	Open a window for each file in the argument list (vim :all / :sall).
<code>:compile</code>	Run a shell command and collect its errors into the compilation list (emacs compile / M-x compile).
<code>:recompile</code>	Re-run the last compile command (emacs recompile).
<code>:make</code>	Run <code>make [args]</code> , collect errors into the quickfix list, jump to the first (vim :make).
<code>:lmake, :lmak</code>	Run <code>make [args]</code> and collect errors, like :make (vim :lmake targets the location list; zemacs uses one unified results console).
<code>:tag, :ta</code>	Jump to the ctags definition of {name} from the tags file, pushing the tag stack (vim :tag).
<code>:tselect, :ts</code>	List every matching tag in a picker; select one to jump (vim :tselect).
<code>:tjump, :tj</code>	Jump to the tag if unique, else show the tag picker (vim :tjump).
<code>:stag, :pt, :ptag</code>	Open the tag's definition in a new horizontal split (vim :stag).
<code>:tnext, :tn</code>	Jump to the next matching tag (vim :tnext).
<code>:tprevious, :tp, :tNext, :tN</code>	Jump to the previous matching tag (vim :tprevious).
<code>:tfirst, :trewind, :tr</code>	Jump to the first matching tag (vim :tfirst).
<code>:tlast</code>	Jump to the last matching tag (vim :tlast).
<code>:pop, :po</code>	Pop the tag stack, returning to where the last :tag jumped from (vim :pop).
<code>:tags</code>	Show the tag stack depth and the current matching tag (vim :tags).
<code>:messages, :mes</code>	Show the message log — every status/error shown this session (vim :messages).
<code>:Man, :man</code>	Open a man page in the run console (neovim :Man).
<code>:redir</code>	Capture message output to a register/file: :redir @a
<code>:arduino-compile, :averify, :arduino-verify</code>	Compile the sketch with arduino-cli for the selected board; errors go to the compilation list (Arduino IDE Verify).
<code>:arduino-upload, :aupload</code>	Compile and flash the sketch to the connected board (arduino-cli compile -upload), live in a terminal panel.
<code>:arduino-monitor, :amonitor, :serial-monitor</code>	Open the serial monitor for the selected port/ baud (arduino-cli monitor).
<code>:arduino-compile-verbose, :arduino-verify-verbose</code>	Verbose compile (<code>arduino-cli compile -v</code>); diagnostics into <i>compilation</i> .
<code>:arduino-compile-clean, :arduino-rebuild</code>	Compile without cached build artifacts (<code>arduino-cli compile --clean</code>).
<code>:arduino-compile-jobs, :arduino-compile-j</code>	Compile with N parallel jobs (<code>arduino-cli compile -j <n></code>).
<code>:arduino-compiledb, :arduino-compilation-database</code>	Generate compile_commands.json for the C/C++ LSP (<code>arduino-cli compile --only-compilation-database</code>).
<code>:arduino-compile-warnings, :arduino-warnings</code>	Compile at a warning level (<code>arduino-cli compile --warnings <none default more all></code>).

Name	Description
<code>:arduino-compile-profile,</code> <code>:arduino-build-profile</code>	Compile using a sketch build profile (<code>arduino-cli compile --profile <name></code>).
<code>:arduino-compile-debug-opt,</code> <code>:arduino-compile-for-debug</code>	Compile with debug-friendly optimization (<code>arduino-cli compile --optimize-for-debug</code>).
<code>:arduino-upload-verify,</code> <code>:arduino-upload-verified</code>	Build + flash, then verify the flashed program (<code>arduino-cli compile --upload --verify</code>).
<code>:arduino-upload-programmer,</code> <code>:arduino-upload-via-programmer</code>	Build + flash through a programmer (<code>arduino-cli compile --upload --programmer <id></code>).
<code>:arduino-upload-dir, :arduino-upload-input-dir</code>	Flash a pre-built binary folder without recompiling (<code>arduino-cli upload --input-dir <dir></code>).
<code>:arduino-upload-file,</code> <code>:arduino-upload-input-file</code>	Flash a specific pre-built binary without recompiling (<code>arduino-cli upload --input-file <file></code>).
<code>:arduino-monitor-raw, :arduino-monitor-noraw</code>	Serial monitor without output transformations (<code>arduino-cli monitor --raw</code>).
<code>:arduino-monitor-timestamp, :arduino-monitor-ts</code>	Serial monitor prefixing each line with a timestamp (<code>arduino-cli monitor --timestamp</code>).
<code>:arduino-board-programmers,</code> <code>:arduino-list-programmers</code>	List programmers the selected board supports (<code>arduino-cli board details --list-programmers</code>).
<code>:arduino-board-list-watch,</code> <code>:arduino-boards-watch</code>	Watch for boards connecting/disconnecting (<code>arduino-cli board list --watch</code>), live in a panel.
<code>:arduino-lib-list-updatable,</code> <code>:arduino-libs-updatable</code>	Installed libraries with a newer version available (<code>arduino-cli lib list --updatable</code>).
<code>:arduino-lib-install-git,</code> <code>:arduino-lib-install-url</code>	Install a library from a git repository (<code>arduino-cli lib install --git-url <url></code>).
<code>:arduino-lib-install-zip,</code> <code>:arduino-lib-install-archive</code>	Install a library from a local .zip (<code>arduino-cli lib install --zip-path <path></code>).
<code>:arduino-compile-quiet, :arduino-compile-silent</code>	Quiet compile, errors only (<code>arduino-cli compile -q</code>).
<code>:arduino-compile-properties,</code> <code>:arduino-build-properties</code>	Dump resolved build properties without building (<code>arduino-cli compile --show-properties</code>).
<code>:arduino-compile-preprocess,</code> <code>:arduino-preprocess</code>	Output the preprocessed sketch (<code>arduino-cli compile --preprocess</code>).
<code>:arduino-compile-dump-profile,</code> <code>:arduino-dump-profile</code>	Print a reproducible build profile for the sketch (<code>arduino-cli compile --dump-profile</code>).
<code>:arduino-compile-board-options,</code> <code>:arduino-board-options</code>	Compile with custom board menu options (<code>arduino-cli compile --board-options <opts></code>).
<code>:arduino-compile-build-property,</code> <code>:arduino-build-property</code>	Compile overriding a build property (<code>arduino-cli compile --build-property <key=value></code>).
<code>:arduino-compile-output-dir,</code> <code>:arduino-compile-out-dir</code>	Compile and save build artifacts to a directory (<code>arduino-cli compile --output-dir <dir></code>).
<code>:arduino-upload-verbose, :arduino-flash-verbose</code>	Verbose build + flash (<code>arduino-cli compile --upload -v</code>).
<code>:arduino-debug-info, :arduino-debug-config</code>	Print the debugger configuration without starting a session (<code>arduino-cli debug --info</code>).
<code>:arduino-debug-programmer,</code> <code>:arduino-debug-via-programmer</code>	Launch the debugger through a programmer (<code>arduino-cli debug --programmer <id></code>).
<code>:arduino-monitor-quiet, :arduino-monitor-silent</code>	Serial monitor suppressing non-error diagnostics (<code>arduino-cli monitor --quiet</code>).

Name	Description
<code>:arduino-monitor-describe,</code> <code>:arduino-monitor-port-info</code>	Describe the port's supported monitor settings (<code>arduino-cli monitor --describe</code>).
<code>:arduino-core-list-updatable,</code> <code>:arduino-cores-updatable</code>	Installed platforms with a newer version available (<code>arduino-cli core list --updatable</code>).
<code>:arduino-core-list-all,</code> <code>:arduino-cores-all</code>	Every installed platform incl. release channels (<code>arduino-cli core list --all</code>).
<code>:arduino-lib-list-all,</code> <code>:arduino-libs-all</code>	Installed libraries across all locations incl. built-in (<code>arduino-cli lib list --all</code>).
<code>:arduino-update-outdated,</code> <code>:arduino-update-show-outdated</code>	Refresh indexes then list upgradable cores/libraries (<code>arduino-cli update --show-outdated</code>).
<code>:arduino-boards-hidden,</code> <code>:arduino-board-listall-hidden</code>	Every known board including platform-hidden variants (<code>arduino-cli board listall --show-hidden</code>).
<code>:arduino-lib-search-names,</code> <code>:arduino-lib-search-name</code>	A names-only library search (<code>arduino-cli lib search <query> --names</code>).
<code>:arduino-sketch-archive-full,</code> <code>:arduino-archive-full</code>	Archive the sketch with its build output (<code>arduino-cli sketch archive --include-build-dir</code>).
<code>:arduino-lib-install-no-deps,</code> <code>:arduino-lib-install-nodeps</code>	Install a library without its declared dependencies (<code>arduino-cli lib install <name> --no-deps</code>).
<code>:arduino-board-details-full,</code> <code>:arduino-board-info-full</code>	The complete board detail dump for the selected FQBN (<code>arduino-cli board details --full</code>).
<code>:arduino-boards,</code> <code>:arduino-board</code>	Pick the target board (FQBN) from installed platforms (Arduino IDE board selector).
<code>:arduino-ports,</code> <code>:arduino-port</code>	Pick the serial port from connected devices (<code>arduino-cli board list</code>).
<code>:arduino-lib-search,</code> <code>:arduino-lib</code>	Search the Arduino library index and install the pick (Arduino IDE Library Manager).
<code>:arduino-core-install,</code> <code>:arduino-core</code>	Install a board-support core, e.g. <code>arduino:avr</code> (Arduino IDE Boards Manager).
<code>:pio-build,</code> <code>:pio-run,</code> <code>:platformio-build</code>	Build the PlatformIO project (<code>pio run</code>); errors go to the compilation list.
<code>:pio-upload,</code> <code>:platformio-upload</code>	Build and upload the PlatformIO project (<code>pio run -t upload</code>), live in a terminal panel.
<code>:pio-exec,</code> <code>:platformio-exec,</code> <code>:pio-run-exec</code>	Build and run the native program (<code>pio run -t exec</code>); args pass through as <code>--program-arg</code> .
<code>:pio-upload-monitor,</code> <code>:platformio-upload-monitor,</code> <code>:pio-flash-monitor</code>	Build, flash, then open the serial monitor in one shot (<code>pio run -t upload -t monitor</code>); optional port overrides <code>--monitor-port</code> .
<code>:pio-monitor,</code> <code>:platformio-monitor</code>	Open the PlatformIO serial monitor (<code>pio device monitor</code>).
<code>:pio-devices,</code> <code>:pio-device-list</code>	Pick a serial port from <code>pio device list</code> and set it for this project.
<code>:pio-device-logical,</code> <code>:platformio-device-logical</code>	List logical (disk) devices (<code>pio device list --logical</code>).
<code>:pio-device-mdns,</code> <code>:platformio-device-mdns</code>	List multicast-DNS / network (OTA) devices (<code>pio device list --mdns</code>).
<code>:pio-device-serial,</code> <code>:platformio-device-serial</code>	List serial ports only (<code>pio device list --serial</code>).
<code>:pio-pkg-list-libraries,</code> <code>:platformio-pkg-list-libraries,</code> <code>:pio-libs-only</code>	List installed libraries only (<code>pio pkg list --only-libraries</code>).
<code>:pio-pkg-list-platforms,</code> <code>:platformio-pkg-list-platforms</code>	List installed platforms only (<code>pio pkg list --only-platforms</code>).
<code>:pio-pkg-list-tools,</code> <code>:platformio-pkg-list-tools</code>	List installed tool packages only (<code>pio pkg list --only-tools</code>).

Name	Description
<code>:pio-test-json, :platformio-test-json</code>	Unit-test results as JSON (<code>pio test --json-output</code>), shown in a scratch buffer.
<code>:pio-project-config-json, :platformio-project-config-json</code>	Computed project configuration as JSON (<code>pio project config --json-output</code>).
<code>:pio-project-metadata-json, :platformio-project-metadata-json</code>	IDE/LSP metadata as JSON (<code>pio project metadata --json-output</code>).
<code>:pio-system-info-json, :platformio-system-info-json</code>	System-wide PlatformIO information as JSON (<code>pio system info --json-output</code>).
<code>:pio-pkg-search-page, :platformio-pkg-search-page</code>	A specific page of registry search results (<code>pio pkg search <query> --page <n></code>).
<code>:pio-pkg-unpublish-undo, :platformio-pkg-unpublish-undo</code>	Restore a previously unpublished package (<code>pio pkg unpublish <pkg> --undo</code>).
<code>:pio-init-no-deps, :platformio-init-no-deps</code>	Scaffold a project without installing dependencies (<code>pio project init --board <board> --no-install-dependencies</code>).
<code>:pio-home, :platformio-home</code>	Launch the PlatformIO Home GUI (<code>pio home</code>), live in a terminal panel; extra args tune the server.
<code>:pio-init, :platformio-init</code>	Scaffold a PlatformIO project for a board (<code>pio project init --board <id></code>).
<code>:embedded-baud, :serial-baud</code>	Set the serial monitor baud rate for this project (e.g. 9600, 115200).
<code>:arduino-plotter, :serial-plotter</code>	Live-graph the numbers streaming from the serial port (Arduino IDE Serial Plotter).
<code>:pio-plotter, :platformio-plotter</code>	Live-graph the PlatformIO serial monitor output (serial plotter).
<code>:arduino-new-sketch, :arduino-sketch-new</code>	Scaffold a new sketch (<code>arduino-cli sketch new</code>) and open its .ino (Arduino IDE New Sketch).
<code>:arduino-compile-export, :aexport, :arduino-export</code>	Compile and export the built binaries to the sketch folder (Arduino IDE Export Compiled Binary).
<code>:arduino-burn-bootloader, :arduino-bootloader</code>	Flash the bootloader to the selected board (Arduino IDE Burn Bootloader), live in a terminal panel.
<code>:arduino-board-info, :arduino-board-details</code>	Print the selected board's specs and menu options (Arduino IDE Get Board Info).
<code>:arduino-core-search, :arduino-boards-manager</code>	Search the Boards Manager index for a core (Arduino IDE Boards Manager).
<code>:arduino-core-list, :arduino-cores</code>	List installed board-support cores (Boards Manager, installed tab).
<code>:arduino-core-uninstall</code>	Uninstall a board-support core, e.g. <code>arduino:avr</code> .
<code>:arduino-core-update-index, :arduino-update-index</code>	Refresh the Boards Manager package index (<code>arduino-cli core update-index</code>).
<code>:arduino-core-upgrade</code>	Upgrade all installed board-support cores (<code>arduino-cli core upgrade</code>), live in a terminal panel.
<code>:arduino-lib-list, :arduino-libs</code>	List installed Arduino libraries (Library Manager, installed tab).
<code>:arduino-lib-uninstall</code>	Uninstall an Arduino library by name (<code>arduino-cli lib uninstall</code>).
<code>:arduino-lib-upgrade</code>	Upgrade all installed Arduino libraries (<code>arduino-cli lib upgrade</code>), live in a terminal panel.
<code>:arduino-lib-examples</code>	List a library's example sketches (<code>arduino-cli lib examples</code>).
<code>:arduino-sketch-archive, :arduino-archive</code>	Zip the whole sketch (Arduino IDE Sketch → Archive Sketch).
<code>:pio-clean, :platformio-clean</code>	Remove PlatformIO build artifacts (<code>pio run -t clean</code>); output goes to the compilation list.

Name	Description
<code>:pio-test, :platformio-test</code>	Run the PlatformIO project's unit tests (<code>pio test</code>); failures go to the compilation list.
<code>:pio-check, :platformio-check</code>	PlatformIO static code analysis (<code>pio check</code>); findings go to the compilation list.
<code>:pio-boards, :platformio-boards</code>	The PlatformIO Board Explorer (<code>pio boards [query]</code>); output in the Run console.
<code>:pio-boards-json, :platformio-boards-json</code>	The Board Explorer as JSON (<code>pio boards --json-output [query]</code>), shown in a scratch buffer.
<code>:pio-lib-install, :pio-pkg-install, :platformio-lib-install</code>	Add a library to the PlatformIO project (<code>pio pkg install -l <name></code>), live in a terminal panel.
<code>:pio-lib-list, :pio-pkg-list, :pio-libs</code>	List installed PlatformIO project packages (<code>pio pkg list</code>).
<code>:pio-lib-uninstall, :pio-pkg-uninstall</code>	Remove a PlatformIO project library (<code>pio pkg uninstall -l <name></code>), live in a terminal panel.
<code>:pio-lib-update, :pio-pkg-update</code>	Update installed PlatformIO project packages (<code>pio pkg update</code>), live in a terminal panel.
<code>:arduino-update</code>	Refresh the core and library indexes together (arduino-cli update).
<code>:arduino-upgrade</code>	Upgrade all installed cores and libraries (arduino-cli upgrade), live in a terminal panel.
<code>:arduino-outdated</code>	List cores and libraries with newer versions available (arduino-cli outdated).
<code>:arduino-lib-deps</code>	Show dependency status for a library (arduino-cli lib deps).
<code>:arduino-config, :arduino-config-dump</code>	Print the active arduino-cli configuration (arduino-cli config dump).
<code>:arduino-debug</code>	Launch the arduino-cli debugger for the selected board, live in a terminal panel.
<code>:pio-lib-search, :pio-pkg-search, :platformio-lib-search</code>	Search the PlatformIO registry for a library (<code>pio pkg search <query></code>).
<code>:pio-lib-outdated, :pio-pkg-outdated</code>	List installed project packages with newer versions available (<code>pio pkg outdated</code>).
<code>:pio-lib-show, :pio-pkg-show</code>	Show PlatformIO registry details for a package (<code>pio pkg show <pkg></code>).
<code>:pio-debug, :platformio-debug</code>	Launch the PlatformIO Unified Debugger for the project, live in a terminal panel.
<code>:pio-upgrade, :platformio-upgrade</code>	Upgrade PlatformIO Core itself (<code>pio upgrade</code>), live in a terminal panel.
<code>:pio-size, :platformio-size</code>	Show the program size report (<code>pio run -t size</code>).
<code>:pio-compiledb, :platformio-compiledb</code>	Generate compile_commands.json for the C/C++ LSP (<code>pio run -t compiledb</code>).
<code>:pio-buildfs, :platformio-buildfs</code>	Build the SPIFFS/LittleFS filesystem image (<code>pio run -t buildfs</code>).
<code>:pio-uploadfs, :platformio-uploadfs</code>	Flash the filesystem image to the board (<code>pio run -t uploadfs</code>), live in a terminal panel.
<code>:pio-uploadeep, :platformio-uploadeep</code>	Flash the EEPROM (<code>pio run -t uploadeep</code> , AVR boards), live in a terminal panel.
<code>:pio-bootloader, :platformio-bootloader</code>	Burn the bootloader (<code>pio run -t bootloader</code> , AVR boards), live in a terminal panel.
<code>:pio-fuses, :platformio-fuses</code>	Set the microcontroller fuses (<code>pio run -t fuses</code> , AVR boards), live in a terminal panel.
<code>:pio-nobuild, :platformio-nobuild</code>	Flash the existing firmware without rebuilding (<code>pio run -t nobuild</code>), live in a terminal panel.
<code>:pio-envdump, :platformio-envdump</code>	Dump the resolved build environment (<code>pio run -t envdump</code>).
<code>:pio-cleanall, :platformio-cleanall</code>	Remove all build artifacts including dependencies (<code>pio run -t cleanall</code>).

Name	Description
<code>:pio-project-config, :platformio-project-config</code>	Show the computed PlatformIO project configuration (<code>pio project config</code>).
<code>:pio-project-metadata, :platformio-project-metadata</code>	Dump the IDE/LSP metadata for the project (<code>pio project metadata</code>).
<code>:pio-pkg-exec, :platformio-pkg-exec</code>	Run a tool from an installed package (<code>pio pkg exec -- <argv></code>), live in a terminal panel.
<code>:pio-pkg-exec-pkg, :platformio-pkg-exec-pkg</code>	Run a tool from a specific installed package (<code>pio pkg exec -p <pkg> -- <argv></code>), live in a terminal panel.
<code>:pio-platform-install, :pio-core-install, :platformio-platform-install</code>	Install a development platform globally (<code>pio pkg install -g -p <spec></code>), live in a terminal panel.
<code>:pio-pkg-pack, :platformio-pkg-pack</code>	Build a tarball of the current package (<code>pio pkg pack [-o <path>]</code>), live in a terminal panel.
<code>:pio-pkg-publish, :platformio-pkg-publish</code>	Publish the current package to the PlatformIO registry (<code>pio pkg publish</code>); extra args forward <code>--owner/--type/--private/--no-notify</code> .
<code>:pio-project-config-lint, :platformio-project-config-lint, :pio-lint</code>	Validate platformio.ini without building (<code>pio project config --lint</code>).
<code>:pio-pkg-list-global, :platformio-pkg-list-global, :pio-pkg-list-g</code>	Globally installed packages (<code>pio pkg list -g</code>).
<code>:pio-pkg-update-global, :platformio-pkg-update-global, :pio-pkg-update-g</code>	Update globally installed packages (<code>pio pkg update -g</code>), live in a terminal panel.
<code>:pio-pkg-install-skip-deps, :platformio-pkg-install-skip-deps</code>	Install a package without its dependencies (<code>pio pkg install --skip-dependencies <spec></code>).
<code>:pio-test-junit, :platformio-test-junit</code>	Run unit tests and write a JUnit XML report (<code>pio test --junit-output-path <path></code>).
<code>:pio-test-json-path, :platformio-test-json-path</code>	Run unit tests and write a JSON report (<code>pio test --json-output-path <path></code>).
<code>:pio-test-port, :platformio-test-port</code>	Run unit tests over a specific serial port (<code>pio test --test-port <port></code>).
<code>:pio-test-upload-port, :platformio-test-upload-port</code>	Flash the test firmware to a specific serial port (<code>pio test --upload-port <port></code>).
<code>:pio-test-monitor-dtr, :platformio-test-monitor-dtr</code>	Set the DTR line state for the post-test monitor (<code>pio test --monitor-dtr <0 1></code>).
<code>:pio-test-monitor-rts, :platformio-test-monitor-rts</code>	Set the RTS line state for the post-test monitor (<code>pio test --monitor-rts <0 1></code>).
<code>:pio-project-metadata-path, :platformio-project-metadata-path</code>	Write the IDE/LSP metadata JSON to a file (<code>pio project metadata --json-output-path <path></code>).
<code>:pio-check-silent, :platformio-check-silent</code>	Quiet static analysis, warnings/errors only (<code>pio check -s</code>).
<code>:pio-upgrade-deps-only, :platformio-upgrade-deps-only</code>	Upgrade only PlatformIO Core's dependencies (<code>pio upgrade --only-dependencies</code>).
<code>:pio-init-env-prefix, :platformio-init-env-prefix</code>	Scaffold a project prefixing generated environment names (<code>pio project init --env-prefix <prefix></code>).
<code>:pio-pkg-unpublish, :platformio-pkg-unpublish</code>	Remove a previously published package (<code>pio pkg unpublish <pkg></code>), live in a terminal panel.
<code>:pio-system-info, :platformio-system-info</code>	Show system-wide PlatformIO information (<code>pio system info</code>).
<code>:pio-system-prune, :platformio-system-prune</code>	Remove unused PlatformIO caches/packages (<code>pio system prune -f</code>), live in a terminal panel.

Name	Description
<code>:pio-prune-cache, :platformio-prune-cache</code>	Prune only cached PlatformIO data (<code>pio system prune -f --cache</code>).
<code>:pio-prune-core, :platformio-prune-core</code>	Prune only unnecessary core packages (<code>pio system prune -f --core-packages</code>).
<code>:pio-prune-platform, :platformio-prune-platform</code>	Prune only unnecessary development-platform packages (<code>pio system prune -f --platform-packages</code>).
<code>:pio-prune-dry-run, :platformio-prune-dry-run</code>	Show what <code>pio system prune</code> would remove without deleting (<code>pio system prune --dry-run</code>).
<code>:pio-settings-get, :platformio-settings-get</code>	Print PlatformIO Core settings, all or one key (<code>pio settings get [name]</code>).
<code>:pio-settings-set, :platformio-settings-set</code>	Change a PlatformIO Core setting (<code>pio settings set <name> <value></code>).
<code>:pio-remote-agent-list, :pio-remote-agents, :platformio-remote-agent-list</code>	List active PlatformIO Remote agents (<code>pio remote agent list</code>).
<code>:pio-remote-agent-start, :platformio-remote-agent-start</code>	Start a PlatformIO Remote agent (<code>pio remote agent start [--name <n>] [--share <email>] [--working-dir <dir>]</code>).
<code>:pio-remote-devices, :pio-remote-device-list, :platformio-remote-devices</code>	List serial devices attached to remote agents (<code>pio remote device list</code>).
<code>:pio-remote-run, :platformio-remote-run</code>	Build/upload the project via a remote agent (<code>pio remote run</code>), live in a terminal panel.
<code>:pio-remote-test, :platformio-remote-test</code>	Run unit tests via a remote agent (<code>pio remote test</code>), live in a terminal panel.
<code>:pio-remote-update, :platformio-remote-update</code>	Update platforms/packages/libraries on remote agents (<code>pio remote update [--dry-run]</code>).
<code>:pio-account-login, :platformio-account-login</code>	Sign in to a PlatformIO account (<code>pio account login</code>), live in a terminal panel.
<code>:pio-account-logout, :platformio-account-logout</code>	Sign out of the PlatformIO account (<code>pio account logout</code>).
<code>:pio-account-show, :platformio-account-show</code>	Show the current PlatformIO account information (<code>pio account show</code>); extra args forward <code>--offline/--json-output</code> .
<code>:pio-account-token, :platformio-account-token</code>	Print the account auth token (<code>pio account token [--regenerate] [--json-output] [-p <password>]</code>).
<code>:pio-env, :pio-environment, :platformio-env</code>	Select the PlatformIO build environment (<code>[env:...]</code>); no arg picks from <code>platformio.ini</code> , <code>-</code> clears.
<code>:pio-list-targets, :platformio-list-targets</code>	Enumerate the project's build targets (<code>pio run --list-targets</code>).
<code>:pio-list-tests, :platformio-list-tests</code>	List the project's test suites (<code>pio test --list-tests</code>).
<code>:pio-test-filter, :platformio-test-filter</code>	Run only the tests matching a pattern (<code>pio test -f <pattern></code>).
<code>:pio-check-severity, :platformio-check-severity</code>	Static analysis filtered by minimum defect severity (<code>pio check --severity <low medium high></code>).
<code>:pio-tool-install, :platformio-tool-install</code>	Install a tool package globally (<code>pio pkg install -g -t <spec></code>), live in a terminal panel.
<code>:pio-boards-installed, :platformio-boards-installed</code>	List boards from installed platforms only (<code>pio boards --installed</code>).
<code>:pio-monitor-filter, :platformio-monitor-filter</code>	Add a serial monitor filter for this project (e.g. time, log2file, hexlify, send_on_enter).
<code>:pio-monitor-filters-clear, :platformio-monitor-filters-clear</code>	Remove all configured serial monitor filters.
<code>:pio-monitor-eol, :platformio-monitor-eol</code>	Set the serial monitor end-of-line mode (CR, LF, or CRLF).
<code>:pio-monitor-parity, :platformio-monitor-parity</code>	Set the serial monitor parity (N, E, O, S, or M).
<code>:pio-monitor-rts, :platformio-monitor-rts</code>	Set the initial RTS line state for the serial monitor (0 or 1).

Name	Description
<code>:pio-monitor-dtr, :platformio-monitor-dtr</code>	Set the initial DTR line state for the serial monitor (0 or 1).
<code>:pio-monitor-echo, :platformio-monitor-echo</code>	Toggle local echo in the serial monitor (--echo).
<code>:pio-monitor-raw, :platformio-monitor-raw</code>	Toggle raw serial monitor mode, disabling output transforms (--raw).
<code>:pio-monitor-encoding, :platformio-monitor-encoding</code>	Set the serial monitor encoding (e.g. UTF-8 , Latin-1 , hexlify); empty resets.
<code>:pio-monitor-flow, :platformio-monitor-flow</code>	Select serial monitor flow control (none , rtscts , or xonxoff).
<code>:pio-monitor-reconnect, :platformio-monitor-reconnect</code>	Toggle serial monitor auto-reconnect (on or off ; off = --no-reconnect).
<code>:pio-monitor-quiet, :platformio-monitor-quiet</code>	Toggle suppression of non-error serial monitor diagnostics (--quiet).
<code>:pio-monitor-exit-char, :platformio-monitor-exit-char</code>	Set the serial monitor exit-char ASCII code (--exit-char ; default 3 = Ctrl+C); empty resets.
<code>:pio-monitor-menu-char, :platformio-monitor-menu-char</code>	Set the serial monitor menu-char ASCII code (--menu-char ; default 20 = Ctrl+T); empty resets.
<code>:pio-pkg-show-type, :platformio-pkg-show-type</code>	Registry details scoped to a package type (pio pkg show --type <library platform tool> <pkg>).
<code>:pio-pkg-exec-call, :platformio-pkg-exec-call</code>	Run a package tool via the call form (pio pkg exec -c <argv...>), live in a terminal panel.
<code>:pio-build-verbose, :platformio-build-verbose</code>	Verbose PlatformIO build (pio run -v), routed through *compilation* .
<code>:pio-build-silent, :platformio-build-silent</code>	Quiet PlatformIO build showing warnings/errors only (pio run -s).
<code>:pio-run-jobs, :platformio-run-jobs</code>	Build with N parallel jobs (pio run -j <n>).
<code>:pio-build-no-auto-clean, :platformio-build-no-auto-clean</code>	Build without the pre-build clean (pio run --disable-auto-clean).
<code>:pio-target, :platformio-target</code>	Run an arbitrary PlatformIO build target (pio run -t <name>), live in a terminal panel.
<code>:pio-upload-to, :platformio-upload-to</code>	Build + flash to a specific port (pio run -t upload --upload-port <port>).
<code>:pio-test-verbose, :platformio-test-verbose</code>	Verbose PlatformIO unit tests (pio test -v).
<code>:pio-test-ignore, :platformio-test-ignore</code>	Run tests except those matching a pattern (pio test -i <pattern>).
<code>:pio-test-without-building, :platformio-test-without-building</code>	Test the last build without rebuilding (pio test --without-building).
<code>:pio-test-without-uploading, :platformio-test-without-uploading</code>	Run tests without flashing first (pio test --without-uploading).
<code>:pio-test-without-testing, :platformio-test-without-testing</code>	Build + upload but skip running the tests (pio test --without-testing).
<code>:pio-test-no-reset, :platformio-test-no-reset</code>	Do not reset the board between tests (pio test --no-reset).
<code>:pio-check-verbose, :platformio-check-verbose</code>	Verbose static code analysis (pio check -v).
<code>:pio-check-json, :platformio-check-json</code>	Static analysis as JSON (pio check --json-output), shown in a scratch buffer.
<code>:pio-check-flags, :platformio-check-flags</code>	Static analysis with extra tool flags (pio check --flags <flags>).
<code>:pio-check-fail-on, :platformio-check-fail-on</code>	Fail on defects at or above a severity (pio check --fail-on-defect <low medium high>).

Name	Description
<code>:pio-check-skip-packages,</code> <code>:platformio-check-skip-packages</code>	Analyse only project sources, skipping libraries (<code>pio check --skip-packages</code>).
<code>:pio-check-src-filters,</code> <code>:platformio-check-src-filters</code>	Restrict analysis to matching sources (<code>pio check --src-filters <pattern></code>).
<code>:pio-debug-verbose,</code> <code>:platformio-debug-verbose</code>	Verbose PlatformIO debugger session (<code>pio debug -v</code>).
<code>:pio-debug-interface,</code> <code>:platformio-debug-interface</code>	Debug with a specific interface (<code>pio debug --interface <name></code>).
<code>:pio-debug-load-mode,</code> <code>:platformio-debug-load-mode</code>	Control firmware reloading on debug start (<code>pio debug --load-mode <always modified manual></code>).
<code>:pio-build-conf,</code> <code>:platformio-build-conf,</code> <code>:pio-build-project-conf</code>	Build using an alternate platformio.ini (<code>pio run -c <path></code>).
<code>:pio-test-conf,</code> <code>:platformio-test-conf</code>	Run unit tests using an alternate platformio.ini (<code>pio test -c <path></code>).
<code>:pio-check-conf,</code> <code>:platformio-check-conf</code>	Static analysis using an alternate platformio.ini (<code>pio check -c <path></code>).
<code>:pio-debug-conf,</code> <code>:platformio-debug-conf</code>	Debug using an alternate platformio.ini (<code>pio debug -c <path></code>), live in a terminal panel.
<code>:pio-init-ide,</code> <code>:platformio-init-ide</code>	Generate IDE integration files (<code>pio project init --ide <ide></code>).
<code>:pio-init-sample,</code> <code>:platformio-init-sample</code>	Scaffold a project with example code (<code>pio project init --board <board> --sample-code</code>).
<code>:pio-init-option,</code> <code>:platformio-init-option</code>	Set a <code>platformio.ini</code> option while initialising (<code>pio project init -O <name=value></code>).
<code>:pio-pkg-install-force,</code> <code>:platformio-pkg-install-force</code>	Reinstall a package even if present (<code>pio pkg install -f <spec></code>).
<code>:pio-pkg-install-global,</code> <code>:platformio-pkg-install-global</code>	Install a package globally (<code>pio pkg install -g <spec></code>).
<code>:pio-lib-install-nosave,</code> <code>:platformio-lib-install-nosave</code>	Install a library without writing it to <code>platformio.ini</code> (<code>pio pkg install -l <name> --no-save</code>).
<code>:pio-pkg-search-sort,</code> <code>:platformio-pkg-search-sort</code>	Registry search with a sort order (<code>pio pkg search <query> --sort <relevance popularity trending added updated></code>).
<code>:pio-upgrade-dev,</code> <code>:platformio-upgrade-dev</code>	Upgrade PlatformIO Core to the development branch (<code>pio upgrade --dev</code>).
<code>:pio-remote-run-force,</code> <code>:platformio-remote-run-force</code>	Force the build to run on the remote agent (<code>pio remote run -r</code>).
<code>:pio-remote-agent-start-named,</code> <code>:platformio-remote-agent-start-named</code>	Start a named Remote agent (<code>pio remote agent start --name <name></code>).
<code>:pio-remote-monitor,</code> <code>:platformio-remote-monitor</code>	Serial monitor over a Remote agent (<code>pio remote device monitor</code>); extra args forward <code>-p/-b/-f/-eol/-sock</code> etc.
<code>:pio-settings-reset,</code> <code>:platformio-settings-reset</code>	Restore PlatformIO Core settings to their defaults (<code>pio settings reset</code>).
<code>:pio-system-completion,</code> <code>:platformio-system-completion</code>	Emit a shell completion script (<code>pio system completion <bash zsh fish powershell></code>).
<code>:pio-ci,</code> <code>:platformio-ci</code>	Build a standalone source tree in an isolated project (<code>pio ci <src> -b <board></code>), live in a terminal panel.
<code>:pio-account-register,</code> <code>:platformio-account-register</code>	Create a new PlatformIO account (<code>pio account register</code>), live in a terminal panel.
<code>:pio-account-password,</code> <code>:platformio-account-password</code>	Change the PlatformIO account password (<code>pio account password</code>), live in a terminal panel.

Name	Description
<code>:pio-account-update, :platformio-account-update</code>	Update PlatformIO profile information (<code>pio account update</code>), live in a terminal panel.
<code>:pio-account-forgot, :platformio-account-forgot</code>	Begin PlatformIO account password recovery (<code>pio account forgot</code>), live in a terminal panel.
<code>:pio-account-destroy, :platformio-account-destroy</code>	Permanently destroy the PlatformIO account (<code>pio account destroy</code>), live in a terminal panel.
<code>:pio-org-list, :platformio-org-list</code>	List PlatformIO organizations and their members (<code>pio org list</code>).
<code>:pio-org-create, :platformio-org-create</code>	Create a new PlatformIO organization (<code>pio org create <orgname></code>), live in a terminal panel.
<code>:pio-org-add, :platformio-org-add</code>	Add an owner to a PlatformIO organization (<code>pio org add <orgname> <username></code>), live in a terminal panel.
<code>:pio-org-remove, :platformio-org-remove</code>	Remove an owner from a PlatformIO organization (<code>pio org remove <orgname> <username></code>), live in a terminal panel.
<code>:pio-org-update, :platformio-org-update</code>	Update a PlatformIO organization (<code>pio org update <orgname></code>), live in a terminal panel.
<code>:pio-org-destroy, :platformio-org-destroy</code>	Destroy a PlatformIO organization (<code>pio org destroy <orgname></code>), live in a terminal panel.
<code>:pio-team-list, :platformio-team-list</code>	List teams in a PlatformIO organization (<code>pio team list <orgname></code>).
<code>:pio-team-create, :platformio-team-create</code>	Create a team in a PlatformIO organization (<code>pio team create <orgname:team></code>), live in a terminal panel.
<code>:pio-team-add, :platformio-team-add</code>	Add a member to a PlatformIO team (<code>pio team add <orgname:team> <username></code>), live in a terminal panel.
<code>:pio-team-remove, :platformio-team-remove</code>	Remove a member from a PlatformIO team (<code>pio team remove <orgname:team> <username></code>), live in a terminal panel.
<code>:pio-team-update, :platformio-team-update</code>	Update a PlatformIO team (<code>pio team update <orgname:team></code>), live in a terminal panel.
<code>:pio-team-destroy, :platformio-team-destroy</code>	Destroy a PlatformIO team (<code>pio team destroy <orgname:team></code>), live in a terminal panel.
<code>:pio-access-list, :platformio-access-list</code>	List published resources and their access levels (<code>pio access list</code>).
<code>:pio-access-grant, :platformio-access-grant</code>	Grant access to a resource (<code>pio access grant <level> <resource> <team></code>), live in a terminal panel.
<code>:pio-access-revoke, :platformio-access-revoke</code>	Revoke access to a resource (<code>pio access revoke <resource> <team></code>), live in a terminal panel.
<code>:pio-access-public, :platformio-access-public</code>	Make a published resource public (<code>pio access public <resource></code>), live in a terminal panel.
<code>:pio-access-private, :platformio-access-private</code>	Make a published resource private (<code>pio access private <resource></code>), live in a terminal panel.
<code>:pio, :platformio</code>	Run an arbitrary <code>pio</code> command in a terminal panel (any subcommand/flag, including <code>pio home</code>).
<code>:arduino-cli, :accli, :arduino</code>	Run an arbitrary <code>arduino-cli</code> command in a terminal panel (any subcommand/flag).
<code>:arduino-core-download, :arduino-download-core</code>	Fetch a core without installing it (<code>arduino-cli core download <package></code>), live in a terminal panel.
<code>:arduino-lib-download, :arduino-download-lib</code>	Fetch a library without installing it (<code>arduino-cli lib download <name></code>), live in a terminal panel.
<code>:arduino-lib-update-index, :arduino-lib-index</code>	Refresh the arduino-cli library index (<code>arduino-cli lib update-index</code>).
<code>:arduino-board-search, :arduino-boards-search</code>	Search the Boards Manager for a board (<code>arduino-cli board search [query]</code>).
<code>:arduino-cache-clean, :arduino-clean-cache</code>	Delete the Boards/Library Manager download cache (<code>arduino-cli cache clean</code>).
<code>:arduino-completion, :arduino-cli-completion</code>	Emit a shell completion script (<code>arduino-cli completion <bash zsh fish powershell></code>).
<code>:arduino-config-get, :arduino-cfg-get</code>	Read one arduino-cli configuration key (<code>arduino-cli config get <key></code>).

Name	Description
<code>:arduino-config-set, :arduino-cfg-set</code>	Set an arduino-cli configuration value (<code>arduino-cli config set <key> <value...></code>).
<code>:arduino-config-add, :arduino-cfg-add</code>	Append value(s) to an arduino-cli list setting (<code>arduino-cli config add <key> <value...></code>).
<code>:arduino-config-remove, :arduino-cfg-remove</code>	Remove value(s) from an arduino-cli list setting (<code>arduino-cli config remove <key> <value...></code>).
<code>:arduino-config-delete, :arduino-cfg-delete</code>	Delete an arduino-cli settings key and its sub-keys (<code>arduino-cli config delete <key></code>).
<code>:arduino-config-init, :arduino-cfg-init</code>	Write the current configuration to a config file (<code>arduino-cli config init</code>).
<code>:arduino-board-attach, :arduino-attach</code>	Attach a sketch to a board (<code>arduino-cli board attach -p <port> -b <fqbn> [sketch]</code>), live in a terminal panel.
<code>:arduino-profile-create, :arduino-profile-new</code>	Create/update a build profile in the sketch project file (<code>arduino-cli profile create</code>), live in a terminal panel.
<code>:arduino-profile-set-default, :arduino-profile-default</code>	Set the default build profile (<code>arduino-cli profile set-default <name></code>).
<code>:arduino-profile-lib-add, :arduino-profile-add-lib</code>	Add a library to the sketch build profile (<code>arduino-cli profile lib add <lib></code>).
<code>:arduino-profile-lib-remove, :arduino-profile-remove-lib</code>	Remove a library from the sketch build profile (<code>arduino-cli profile lib remove <lib></code>).
<code>:arduino-lib-install, :arduino-lib-add</code>	Install a library by name (<code>arduino-cli lib install <name></code>), live in a terminal panel.
<code>:arduino-daemon, :acli-daemon</code>	Run arduino-cli as a gRPC daemon (<code>arduino-cli daemon</code>), live in a terminal panel; extra args tune the server.
<code>:arduino-version, :acli-version</code>	Show the arduino-cli version (<code>arduino-cli version</code> ; add <code>--format json</code>).
<code>:next-error, :cnext-error</code>	Visit the next compilation error's location (emacs <code>next-error</code> / <code>M-g n</code>).
<code>:previous-error, :cprevious-error</code>	Visit the previous compilation error's location (emacs <code>previous-error</code> / <code>M-g p</code>).
<code>:first-error</code>	Visit the first compilation error's location (emacs <code>first-error</code>).
<code>:abbreviate, :ab</code>	List or define an abbreviation for Insert and Command-line mode (vim <code>:abbreviate</code>).
<code>:list-abbrevs</code>	Show all defined abbreviations in a buffer (emacs <code>list-abbrevs</code>).
<code>:define-global-abbrev</code>	Define a global abbreviation: <code>:define-global-abbrev NAME EXPANSION</code> (emacs <code>define-global-abbrev</code>).
<code>:define-mode-abbrev</code>	Define a major-mode-local abbreviation: <code>:define-mode-abbrev NAME EXPANSION</code> (emacs <code>define-mode-abbrev</code>).
<code>:abbrev-mode</code>	Toggle abbrev-mode (auto-expand abbrevs on a typed word separator); <code>:abbrev-mode on</code>
<code>:kill-all-abbrevs</code>	Remove all defined abbreviations (emacs <code>kill-all-abbrevs</code>).
<code>:iabbrev, :ia</code>	List or define an Insert-mode abbreviation (vim <code>:iabbrev</code>).
<code>:cabbrev, :ca</code>	List or define a Command-line-mode abbreviation (vim <code>:cabbrev</code>).
<code>:noreabbrev, :norea</code>	List or define a non-recursive Insert+Command-line abbreviation (vim <code>:noreabbrev</code>).
<code>:inoreabbrev, :inorea</code>	List or define a non-recursive Insert-mode abbreviation (vim <code>:inoreabbrev</code>).
<code>:cnoreabbrev, :cnorea</code>	List or define a non-recursive Command-line-mode abbreviation (vim <code>:cnoreabbrev</code>).
<code>:unabbreviate, :una</code>	Remove an abbreviation for both modes (vim <code>:unabbreviate</code>).

Name	Description
<code>:iunabbreviate, :iuna</code>	Remove an Insert-mode abbreviation (vim :iunabbreviate).
<code>:cunabbreviate, :cuna</code>	Remove a Command-line-mode abbreviation (vim :cunabbreviate).
<code>:abclear, :abc</code>	Remove all abbreviations for both modes (vim :abclear).
<code>:iabclear, :iabc</code>	Remove all Insert-mode abbreviations (vim :iabclear).
<code>:cabclear, :cabc</code>	Remove all Command-line-mode abbreviations (vim :cabclear).
<code>:bfirst, :brewind, :brew</code>	Go to the first buffer in the buffer list (vim :bfirst / :brewind).
<code>:blast, :bl</code>	Go to the last buffer in the buffer list (vim :blast).
<code>:bmodified, :bm</code>	Go to the next modified buffer (vim :bmodified).
<code>:ball, :sball, :unhide, :unh, :sunhide, :sun</code>	Open a window for each buffer in the buffer list (vim :ball; :unhide/:sunhide — every zemacs buffer is loaded).
<code>:badd</code>	Add a file to the buffer list without editing it (vim :badd).
<code>:balt</code>	Add a file to the buffer list and set it as the alternate file (vim :balt).
<code>:bufdo</code>	Run an Ex command in each listed buffer (vim :bufdo).
<code>:spellwrong, :spellw</code>	Mark words as misspelled (vim :spellwrong).
<code>:spellrare, :spellra</code>	Flag words as rare (vim :spellrare).
<code>:spellundo, :spellu</code>	Remove words from the good/bad spell lists (vim :spellundo).
<code>:spelldump, :spelled</code>	Open a buffer listing the user's known-good words (vim :spelldump).
<code>:spellinfo, :spelli</code>	Show the spell wordlist location and sizes (vim :spellinfo).
<code>:buffer-close, :bc, :bclose, :bd, :bdelete, :bun, :bunload, :bw, :bwipe, :bwipeout</code>	Close the current buffer.
<code>:buffer-close!, :bc!, :bclose!</code>	Close the current buffer forcefully, ignoring unsaved changes.
<code>:buffer-close-others, :bco, :bcloseother</code>	Close all buffers but the currently focused one.
<code>:buffer-close-others!, :bco!, :bcloseother!</code>	Force close all buffers but the currently focused one.
<code>:buffer-close-all, :bca, :bcloseall</code>	Close all buffers without quitting.
<code>:buffer-close-all!, :bca!, :bcloseall!</code>	Force close all buffers ignoring unsaved changes without quitting.
<code>:buffer-next, :bn, :bnext</code>	Goto next buffer.
<code>:buffer-previous, :bp, :bprev, :bN, :bNext</code>	Goto previous buffer.
<code>:sbnext, :sbn</code>	Split window and go to the next buffer (vim :sbnext).
<code>:sbprevious, :sbp, :sbNext, :sbN</code>	Split window and go to the previous buffer (vim :sbprevious / :sbNext).
<code>:sbfirst, :sbf, :sbrewind, :sbr</code>	Split window and go to the first buffer (vim :sbfirst / :sbrewind).
<code>:sblast, :sbl</code>	Split window and go to the last buffer (vim :sblast).
<code>:sbmodified, :sbm</code>	Split window and go to the next modified buffer (vim :sbmodified).
<code>:sbuffer, :sb</code>	Split window and go to the buffer whose path contains {name} (vim :sbuffer / :sb).
<code>:pedit, :ped</code>	Edit {file} in a preview (horizontal split) window (vim :pedit).

Name	Description
<code>:pbuffer, :pb</code>	Show the buffer whose path contains {name} in a preview (split) window (vim :pbuffer).
<code>:nohlsearch, :noh, :nohl</code>	Clear the persistent search highlight (vim :nohlsearch).
<code>:clearjumps</code>	Clear the current view's jump list (vim :clearjumps).
<code>:project-replace, :replace-in-files</code>	Regex-replace across all matching workspace files (JetBrains Replace in Files).
<code>:buffers, :ls, :files</code>	List open buffers in the buffer picker (vim :buffers/:ls/:files).
<code>:jumps</code>	List the jump list in a picker (vim :jumps).
<code>:oldfiles</code>	Pick from recently edited files (vim :oldfiles).
<code>:marks</code>	List the buffer's marks in a picker (vim :marks).
<code>:changes</code>	List the buffer's changelist in a picker (vim :changes).
<code>:history</code>	Pick from the command-line history (vim :history).
<code>:delmarks, :delm</code>	Delete the listed named marks (vim :delmarks abc).
<code>:delmarks!, :delm!</code>	Delete all letter marks (vim :delmarks!).
<code>:write, :w, :sav, :saveas</code>	Write changes to disk. Accepts an optional path (:write some/path.txt)
<code>:write!, :w!</code>	Force write changes to disk creating necessary subdirectories. Accepts an optional path (:write! some/path.txt)
<code>:write-buffer-close, :wbc</code>	Write changes to disk and closes the buffer. Accepts an optional path (:write-buffer-close some/path.txt)
<code>:write-buffer-close!, :wbc!</code>	Force write changes to disk creating necessary subdirectories and closes the buffer. Accepts an optional path (:write-buffer-close! some/path.txt)
<code>:new, :n, :enew</code>	Create a new scratch buffer.
<code>:Scratch, :scratch</code>	Open a new scratch buffer, optionally with a language (SPC b S).
<code>:RevealInFinder, :reveal-in-finder</code>	Reveal the current file in the OS file manager (JetBrains Reveal in Finder).
<code>:compose-mail, :mail, :compose</code>	Open a message-mode mail draft (emacs compose-mail, C-x m). :compose-mail [to] [subject...]
<code>:compose-mail-other-window</code>	Open a mail draft in a split (emacs compose-mail-other-window).
<code>:message-send</code>	Assemble and queue the current mail draft (emacs message-send, C-c C-s).
<code>:message-send-and-exit</code>	Queue the draft and kill its buffer (emacs message-send-and-exit, C-c C-c).
<code>:message-kill-buffer</code>	Kill the mail compose buffer (emacs message-kill-buffer, C-c C-k).
<code>:message-goto-to</code>	Move point to the To: header (emacs message-goto-to, C-c C-f C-t).
<code>:message-goto-subject</code>	Move point to the Subject: header (emacs message-goto-subject, C-c C-f C-s).
<code>:message-goto-cc</code>	Move point to the Cc: header (emacs message-goto-cc, C-c C-f C-c).
<code>:message-goto-bcc</code>	Move point to the Bcc: header (emacs message-goto-bcc, C-c C-f C-b).
<code>:message-goto-body, :mail-text</code>	Move point to the message body (emacs message-goto-body / mail-text, C-c C-b).
<code>:message-insert-signature</code>	Insert the signature block at point (emacs message-insert-signature, C-c C-w).

Name	Description
<code>:mml-attach-file</code> , <code>:mail-add-attachment</code>	Attach a file as a MIME part (emacs mml-attach-file, C-c C-a). <code>:mml-attach-file</code>
<code>:format</code> , <code>:fmt</code>	Format the file using an external formatter or language server.
<code>:indent-style</code>	Set the indentation style for editing. ('t' for tabs or 1-16 for number of spaces.)
<code>:line-ending</code>	Set the document's default line ending. Options: crlf, lf.
<code>:earlier</code> , <code>:ear</code>	Jump back to an earlier point in edit history. Accepts a number of steps or a time span.
<code>:later</code> , <code>:lat</code>	Jump to a later point in edit history. Accepts a number of steps or a time span.
<code>:undotree</code> , <code>:undo-tree</code> , <code>:UndotreeToggle</code>	Open the branching undo-history browser (vim undotree).
<code>:undolist</code> , <code>:undol</code>	List the undo states as a text popup (vim :undolist).
<code>:injections</code> , <code>:injection-rules</code>	List the active language-injection rules (defaults + injections.toml).
<code>:injection-info</code> , <code>:what-injection</code>	Report the effective (possibly injected) language at the cursor.
<code>:edit-fragment</code> , <code>:edit-injected-fragment</code>	Edit the injected-language fragment at point in its own buffer (JetBrains Edit Fragment).
<code>:apply-fragment</code> , <code>:apply-injected-fragment</code>	Write the fragment buffer back into its host string.
<code>:inject-language</code> , <code>:inject-lang</code>	Inject a language into the string at point via a /* language=... */ hint (JetBrains inject-here).
<code>:write-quit</code> , <code>:wq</code>	Write changes to disk and close the current view. Accepts an optional path (:wq some/path.txt)
<code>:write-quit!</code> , <code>:wq!</code>	Write changes to disk and close the current view forcefully. Accepts an optional path (:wq! some/path.txt)
<code>:write-all</code> , <code>:wa</code>	Write changes from all buffers to disk.
<code>:write-all!</code> , <code>:wa!</code>	Forcefully write changes from all buffers to disk creating necessary subdirectories.
<code>:write-quit-all</code> , <code>:wqa</code> , <code>:xa</code>	Write changes from all buffers to disk and close all views.
<code>:write-quit-all!</code> , <code>:wqa!</code> , <code>:xa!</code>	Forcefully write changes from all buffers to disk, creating necessary subdirectories, and close all views (ignoring unsaved changes).
<code>:quit-all</code> , <code>:qa</code> , <code>:qall</code> , <code>:quita</code> , <code>:quitall</code>	Close all views.
<code>:quit-all!</code> , <code>:qa!</code>	Force close all views ignoring unsaved changes.
<code>:cquit</code> , <code>:cq</code>	Quit with exit code (default 1). Accepts an optional integer exit code (:cq 2).
<code>:cquit!</code> , <code>:cq!</code>	Force quit with exit code (default 1) ignoring unsaved changes. Accepts an optional integer exit code (:cq! 2).
<code>:theme</code> , <code>:colorscheme</code> , <code>:colo</code>	Change the editor theme (show current theme if no name specified).
<code>:describe-theme</code>	Show a theme's faces and their fg/bg colors in a scratch buffer (emacs describe-theme).
<code>:disable-theme</code>	Turn off the active theme, reverting to the default (emacs disable-theme).
<code>:hunk-reset</code> , <code>:reset-hunk</code> , <code>:hunk-undo</code>	Undo the git hunk under the cursor, restoring it from HEAD (gitsigns reset_hunk).
<code>:hunk-next</code> , <code>:next-hunk</code>	Move the cursor to the next git hunk.
<code>:hunk-prev</code> , <code>:prev-hunk</code>	Move the cursor to the previous git hunk.
<code>:conflict-ours</code> , <code>:diffget-ours</code> , <code>:conflict-keep-ours</code>	Resolve the merge conflict at the cursor by keeping OUR side (HEAD).

Name	Description
<code>:conflict-theirs</code> , <code>:diffget-theirs</code> , <code>:conflict-keep-theirs</code>	Resolve the merge conflict at the cursor by keeping THEIR side (incoming).
<code>:conflict-both</code> , <code>:conflict-keep-both</code>	Resolve the merge conflict at the cursor by keeping BOTH sides.
<code>:conflict-next</code>	Jump to the next merge-conflict marker.
<code>:conflict-prev</code>	Jump to the previous merge-conflict marker.
<code>:theme-toggle</code> , <code>:toggle-theme</code> , <code>:light-dark</code>	Toggle between a dark and light theme (<code>:theme-toggle [dark] [light]</code>).
<code>:theme-next</code>	Switch to the next theme (alphabetical).
<code>:theme-prev</code>	Switch to the previous theme (alphabetical).
<code>:run</code> , <code>:r!</code>	Run a command in the IDE Run tool window (defaults to <code>cargo run</code>).
<code>:grep</code> , <code>:rg</code> , <code>:search-project</code> , <code>:lv</code> , <code>:lvim</code> , <code>:lvimgrep</code>	Search the project (ripgrep) and show jumpable results in the Run console.
<code>:grepadd</code> , <code>:grepa</code>	Search the project like <code>:grep</code> (vim <code>:grepadd</code> appends; zemacs uses one unified results console).
<code>:lgrep</code> , <code>:lgr</code>	Location-list variant of <code>:grep</code> (vim <code>:lgrep</code> ; zemacs uses one unified results console).
<code>:lgrepadd</code> , <code>:lgrepa</code>	Location-list append variant of <code>:grep</code> (vim <code>:lgrepadd</code> ; zemacs uses one unified results console).
<code>:vimgrepadd</code> , <code>:vimgrepa</code>	Search the project like <code>:vimgrep</code> , appending (vim <code>:vimgrepadd</code> ; zemacs uses one unified results console).
<code>:lvimgrepadd</code> , <code>:lvimgrepa</code>	Location-list append variant of <code>:vimgrep</code> (vim <code>:lvimgrepadd</code> ; zemacs uses one unified results console).
<code>:helpgrep</code> , <code>:helpg</code>	Search the help: open the inline Help browser filtered to {pattern} (vim <code>:helpgrep</code>).
<code>:lhelpgrep</code> , <code>:lh</code>	Location-list variant of <code>:helpgrep</code> — open the Help browser filtered to {pattern} (vim <code>:lhelpgrep</code>).
<code>:copen</code> , <code>:cwindow</code> , <code>:cw</code>	Open the quickfix list window.
<code>:cclose</code> , <code>:ccl</code>	Close the quickfix list window.
<code>:cnext</code> , <code>:cn</code>	Jump to the next quickfix entry.
<code>:cprevious</code> , <code>:cprev</code> , <code>:cp</code> , <code>:cN</code>	Jump to the previous quickfix entry.
<code>:cfirst</code> , <code>:crewind</code> , <code>:cr</code>	Jump to the first quickfix entry.
<code>:clast</code> , <code>:cla</code>	Jump to the last quickfix entry.
<code>:cc</code>	Jump to the [count]th quickfix entry (or the current one).
<code>:cnfile</code> , <code>:cnf</code>	Jump to the first quickfix entry in the next file.
<code>:cpfile</code> , <code>:cpf</code> , <code>:cNf</code> , <code>:cNfile</code>	Jump to the last quickfix entry in the previous file.
<code>:cabove</code> , <code>:cabo</code>	Jump to the [count]th quickfix entry above the cursor line.
<code>:cbelow</code> , <code>:cbe1</code>	Jump to the [count]th quickfix entry below the cursor line.
<code>:cbefore</code> , <code>:cbe</code>	Jump to the [count]th quickfix entry before the cursor position.
<code>:cafter</code> , <code>:caf</code>	Jump to the [count]th quickfix entry after the cursor position.
<code>:cbuffer</code> , <code>:cb</code>	Read the current buffer as error lines into the quickfix list.
<code>:cgetbuffer</code>	Read the current buffer into the quickfix list without jumping.
<code>:caddbuffer</code>	Append the current buffer's error lines to the quickfix list.

Name	Description
<code>:cexpr, :cex</code>	Parse the argument text into the quickfix list and jump to the first entry.
<code>:cgetexpr</code>	Parse the argument text into the quickfix list without jumping.
<code>:caddexpr</code>	Append the argument text's entries to the quickfix list.
<code>:cfile, :cf</code>	Read a file of error lines into the quickfix list and jump to the first entry.
<code>:cgetfile</code>	Read a file of error lines into the quickfix list without jumping.
<code>:lopen, :lwindow, :lw</code>	Open the location list window for the current window.
<code>:lclose, :lcl</code>	Close the location list window.
<code>:lnext, :lne, :ln</code>	Jump to the next location list entry.
<code>:lprevious, :lprev, :lp, :lN</code>	Jump to the previous location list entry.
<code>:lfirst, :lrewind, :lr</code>	Jump to the first location list entry.
<code>:llast, :lla</code>	Jump to the last location list entry.
<code>:ll</code>	Jump to the [count]th location list entry (or the current one).
<code>:labove, :lab</code>	Jump to the [count]th location list entry above the cursor line.
<code>:lbelow, :lbel</code>	Jump to the [count]th location list entry below the cursor line.
<code>:lbefore, :lbe</code>	Jump to the [count]th location list entry before the cursor position.
<code>:lafter, :laf</code>	Jump to the [count]th location list entry after the cursor position.
<code>:lbuffer, :lb</code>	Read the current buffer as error lines into the location list.
<code>:lgetbuffer</code>	Read the current buffer into the location list without jumping.
<code>:lexpr, :lex</code>	Parse the argument text into the location list and jump to the first entry.
<code>:lgetexpr</code>	Parse the argument text into the location list without jumping.
<code>:lfile, :lf</code>	Read a file of error lines into the location list and jump to the first entry.
<code>:lgetfile</code>	Read a file of error lines into the location list without jumping.
<code>:tabnew, :tabe, :tabedit</code>	Open a new tabpage (optionally editing a file).
<code>:tabnext, :tabn</code>	Go to the next tabpage (or tab [count]).
<code>:tabprevious, :tabp, :tabNext, :tabN</code>	Go to the previous tabpage.
<code>:tabclose, :tabc</code>	Close the current tabpage.
<code>:tab-rename</code>	Name the current tab, or clear it when given no name (emacs tab-rename).
<code>:tab-switch</code>	Switch to a tab by name or 1-based number (emacs tab-switch).
<code>:tab-undo</code>	Reopen the most recently closed tab (emacs tab-undo).
<code>:tab-bar-history-back</code>	Return to the previously visited tab (emacs tab-bar-history-back).
<code>:tab-bar-history-forward</code>	Re-visit a tab left via history-back (emacs tab-bar-history-forward).
<code>:tabonly, :tabo</code>	Close all tabpages except the current one.
<code>:tabfirst, :tabrewind, :tabr</code>	Go to the first tabpage.

Name	Description
<code>:tablast</code>	Go to the last tabpage.
<code>:tabmove, :tabm</code>	Move the current tabpage to position [N] (default: last).
<code>:tabdo</code>	Run an ex-command in every tabpage.
<code>:windo</code>	Run an ex-command in every window of the current tabpage.
<code>:wincmd</code>	Run a window (CTRL-W) command by key, e.g. <code>:wincmd h</code> focuses left.
<code>:source, :so</code>	Source a Vimscript file through the embedded vimlrs interpreter.
<code>:runtime, :ru</code>	Source a file from the runtimepath (zemacs config dir) via vimlrs (vim <code>:runtime</code>).
<code>:diffthis, :diff</code>	Show the current buffer's changes as a side-by-side diff vs git HEAD (vim <code>:diffthis</code>).
<code>:diffupdate, :diffu</code>	Recompute and redisplay the buffer's diff vs git HEAD (vim <code>:diffupdate</code>).
<code>:diffoff, :diffo</code>	Turn off diff mode: remove the side-by-side diff overlay (vim <code>:diffoff</code>).
<code>:match</code>	Highlight {pattern} in match group 1, or clear it with <code>:match none</code> (vim <code>:match</code>).
<code>:2match</code>	Highlight {pattern} in match group 2, or clear it with <code>:2match none</code> (vim <code>:2match</code>).
<code>:3match</code>	Highlight {pattern} in match group 3, or clear it with <code>:3match none</code> (vim <code>:3match</code>).
<code>:helptags, :helpt</code>	Regenerate help tags (vim <code>:helptags</code>); no-op — zemacs help is indexed directly.
<code>:sign, :sig</code>	Define/place/unplace/list/jump gutter signs (vim <code>:sign</code>); e.g. <code>:sign define warn text=» texthl=WarningMsg</code> then <code>:sign place 1 line=10 name=warn</code> .
<code>:undojoin, :undoj</code>	Join the next change with the previous undo block, so one undo reverts both (vim <code>:undojoin</code>).
<code>:image-mode, :image-display, :image-toggle-display</code>	Display the current image file in the terminal (emacs <code>image-mode</code> / <code>image-toggle-display</code>).
<code>:image-rotate</code>	Rotate the current image 90 degrees and redisplay (emacs <code>image-rotate</code>).
<code>:image-flip-horizontally</code>	Flip the current image left-to-right and redisplay (emacs <code>image-flip-horizontally</code>).
<code>:image-flip-vertically</code>	Flip the current image top-to-bottom and redisplay (emacs <code>image-flip-vertically</code>).
<code>:add-file-local-variable</code>	Set a file-local variable in the Local Variables block (emacs <code>add-file-local-variable</code>).
<code>:add-file-local-variable-prop-line</code>	Set a file-local variable in the first-line *- prop line (emacs <code>add-file-local-variable-prop-line</code>).
<code>:delete-file-local-variable</code>	Remove a file-local variable from the Local Variables block (emacs <code>delete-file-local-variable</code>).
<code>:delete-file-local-variable-prop-line</code>	Remove a file-local variable from the *- prop line (emacs <code>delete-file-local-variable-prop-line</code>).
<code>:image-increase-size</code>	Scale the current image up by 25% and redisplay (emacs <code>image-increase-size</code>).
<code>:image-decrease-size</code>	Scale the current image down by 20% and redisplay (emacs <code>image-decrease-size</code>).
<code>:image-transform-set-percent</code>	Set the current image's scale to N percent and redisplay (emacs <code>image-transform-set-percent</code>).
<code>:image-transform-set-scale</code>	Set the current image's scale from a multiplier, e.g. 1.5 (emacs <code>image-transform-set-scale</code>).
<code>:image-transform-fit-to-window</code>	Fit the current image to the window (emacs <code>image-transform-fit-to-window</code>).

Name	Description
<code>:image-transform-reset-to-original</code> , <code>:image-transform-reset-to-initial</code>	Drop all rotation/flip/scale transforms and redisplay (emacs <code>image-transform-reset-to-original</code>).
<code>:image-next-file</code>	Open the next image file in the directory (emacs <code>image-next-file</code>).
<code>:image-previous-file</code>	Open the previous image file in the directory (emacs <code>image-previous-file</code>).
<code>:image-mode-copy-file-name-as-kill</code>	Copy the image file's path to the clipboard register (emacs <code>image-mode-copy-file-name-as-kill</code>).
<code>:doc-view-mode</code> , <code>:doc-view-minor-mode</code> , <code>:doc-view-toggle-display</code>	Render the current document's page in the terminal (emacs <code>doc-view-mode</code>).
<code>:doc-view-next-page</code> , <code>:doc-view-scroll-up-or-next-page</code>	Show the next page of the document (emacs <code>doc-view-next-page</code>).
<code>:doc-view-previous-page</code> , <code>:doc-view-scroll-down-or-previous-page</code>	Show the previous page of the document (emacs <code>doc-view-previous-page</code>).
<code>:doc-view-first-page</code>	Show the first page of the document (emacs <code>doc-view-first-page</code>).
<code>:doc-view-last-page</code>	Show the last page of the document (emacs <code>doc-view-last-page</code>).
<code>:doc-view-goto-page</code>	Show page N of the document (emacs <code>doc-view-goto-page</code>).
<code>:doc-view-enlarge</code>	Increase the document render resolution and redisplay (emacs <code>doc-view-enlarge</code>).
<code>:doc-view-shrink</code>	Decrease the document render resolution and redisplay (emacs <code>doc-view-shrink</code>).
<code>:doc-view-open-text</code>	Extract the document's text into a scratch buffer (emacs <code>doc-view-open-text</code>).
<code>:doc-view-search</code> , <code>:doc-view-search-backward</code>	Search the document's text for a pattern (emacs <code>doc-view-search</code>).
<code>:doc-view-clear-cache</code> , <code>:doc-view-kill-proc</code> , <code>:doc-view-kill-proc-and-buffer</code>	Forget the doc-view render state (emacs <code>doc-view-clear-cache</code>).
<code>:doc-view-set-slice</code>	Crop the displayed page to X Y WIDTH HEIGHT pixels (emacs <code>doc-view-set-slice</code>).
<code>:doc-view-reset-slice</code>	Drop the doc-view crop slice and show the full page (emacs <code>doc-view-reset-slice</code>).
<code>:append</code> , <code>:a</code>	Insert typed lines after the current line; end input with a line containing only '.' (vim <code>:append</code>).
<code>:insert</code> , <code>:i</code>	Insert typed lines before the current line; end input with a line containing only '.' (vim <code>:insert</code>).
<code>:change</code> , <code>:c</code>	Replace the current line with typed lines; end input with a line containing only '.' (vim <code>:change</code>).
<code>:doautocmd</code> , <code>:do</code> , <code>:doa</code>	Fire the autocommands registered for {event} on the current buffer (vim <code>:doautocmd</code>).
<code>:doautoall</code> , <code>:doautoa</code>	Fire the autocommands for {event} on every loaded buffer (vim <code>:doautoall</code>).
<code>:drop</code> , <code>:dr</code>	Jump to a buffer already editing {file}, else edit it (vim <code>:drop</code>).
<code>:lua</code>	Run a Lua snippet through the system lua interpreter and echo its output (vim <code>:lua</code>).
<code>:luafile</code> , <code>:luaf</code>	Run a Lua script file through the system lua interpreter (vim <code>:luafile</code>).
<code>:perl</code> , <code>:pe</code>	Run a Perl snippet through the system perl interpreter and echo its output (vim <code>:perl</code>).
<code>:perlfile</code> , <code>:perlf</code>	Run a Perl script file through the system perl interpreter (vim <code>:perlfile</code>).
<code>:python</code> , <code>:py</code>	Run a Python snippet through the system python interpreter and echo its output (vim <code>:python</code>).

Name	Description
<code>:pyfile, :pyf</code>	Run a Python script file through the system python interpreter (vim :pyfile).
<code>:py3, :python3</code>	Run a Python 3 snippet through the system python3 interpreter and echo its output (vim :py3 / :python3).
<code>:py3file, :py3f, :python3file</code>	Run a Python 3 script file through the system python3 interpreter (vim :py3file).
<code>:ruby, :rub</code>	Run a Ruby snippet through the system ruby interpreter and echo its output (vim :ruby).
<code>:rubyfile, :rubyf</code>	Run a Ruby script file through the system ruby interpreter (vim :rubyfile).
<code>:tabs</code>	List the tabpages and switch to the selected one.
<code>:cdo</code>	Run an ex-command on each quickfix entry.
<code>:cfdo</code>	Run an ex-command on the first quickfix entry of each file.
<code>:ldo</code>	Run an ex-command on each location list entry.
<code>:lfdo</code>	Run an ex-command on the first location entry of each file.
<code>:clist, :cl</code>	Show the quickfix list.
<code>:llist, :lli</code>	Show the location list.
<code>:colder, :col</code>	Go to an older quickfix list.
<code>:cnewer, :cnew</code>	Go to a newer quickfix list.
<code>:chistory, :chi</code>	Show the quickfix list history position.
<code>:lolder, :lol</code>	Go to an older location list (vim :lolder).
<code>:lnewer, :lnew</code>	Go to a newer location list (vim :lnewer).
<code>:lhistory, :lhi</code>	Show the location list history position (vim :lhistory).
<code>:lnfile, :lnf</code>	Jump to the first location-list entry in the next file (vim :lnfile).
<code>:LNfile, :LNf</code>	Jump to the last location-list entry in the previous file (vim :LNfile).
<code>:caddfile</code>	Append a file of error lines to the quickfix list.
<code>:laddbuffer</code>	Append the current buffer's error lines to the location list.
<code>:laddexpr</code>	Append the argument text's entries to the location list.
<code>:laddfile</code>	Append a file of error lines to the location list.
<code>:lpfile, :lpf</code>	Jump to the last location entry in the previous file.
<code>:shell-quote, :sh-quote</code>	Wrap the selection in safe shell single-quotes.
<code>:wrap-tag</code>	Wrap each selection in
<code>:csv-column, :csv-col</code>	Replace the selected CSV/TSV with just its Nth column (1-based).
<code>:code-fence, :fence</code>	Wrap the selection in a fenced Markdown code block with optional language.
<code>:md-table, :table-fmt</code>	Align the selected Markdown pipe table (pad columns, rebuild separator row).
<code>:json-query, :json-get</code>	Replace the selected JSON with the value at a dot-path (e.g. users.0.name).
<code>:json-flatten, :json-paths</code>	Flatten the selected JSON into greppable <code>path = value</code> lines.
<code>:json-to-csv, :json-csv</code>	Convert the selected JSON array of objects to CSV (sorted header union).

Name	Description
<code>:json-unflatten, :json-unpaths</code>	Rebuild nested JSON from <code>path = value</code> lines (inverse of <code>:json-flatten</code>).
<code>:toml-to-json, :toml-json</code>	Convert the selected TOML to pretty-printed JSON.
<code>:json-to-toml, :json-toml</code>	Convert the selected JSON to pretty-printed TOML.
<code>:json-sort, :json-sort-array</code>	Sort the selected JSON array (optionally by an object field: <code>:json-sort name</code>).
<code>:json-pick, :json-select</code>	Keep only the named keys in the selected JSON object/array (e.g. <code>:json-pick name age</code>).
<code>:json-omit, :json-drop</code>	Drop the named keys from the selected JSON object/array (e.g. <code>:json-omit password</code>).
<code>:json-unique, :json-uniq</code>	Remove duplicate elements from the selected JSON array (optionally by a field).
<code>:json-group-by, :json-group</code>	Group the selected JSON array of objects by a field (e.g. <code>:json-group-by city</code>).
<code>:extract, :matches</code>	Replace the selection with every regex match, one per line (group 1 if present).
<code>:filter, :keep-lines, :delete-non-matching-lines</code>	Keep only the selected lines matching a regex (Emacs <code>keep-lines</code> ; in-buffer <code>grep</code>).
<code>:reject, :remove-lines, :flush-lines, :delete-matching-lines</code>	Drop the selected lines matching a regex (Emacs <code>flush-lines</code> ; in-buffer <code>grep -v</code>).
<code>:count-matches, :count-regex, :how-many</code>	Report how many regex matches (and matching lines) are in the selection (Emacs <code>how-many</code>).
<code>:copy-matching-lines, :copy-lines-matching</code>	Copy lines matching a regex (region, or point to end) to the kill ring (Emacs <code>copy-matching-lines</code>).
<code>:kill-matching-lines, :kill-lines-matching</code>	Delete lines matching a regex (region, or point to end) and save them to the kill ring (Emacs <code>kill-matching-lines</code>).
<code>:uniq-count, :frequency</code>	Collapse the selected lines to <code>count line</code> , sorted by frequency (<code>uniq -c</code>).
<code>:stats, :describe</code>	Show count/sum/mean/min/max of the numbers in the selection (non-destructive).
<code>:seq, :sequence</code>	Insert an integer sequence, one per line: <code>:seq [step]</code> .
<code>:field, :cut</code>	Keep only the Nth whitespace field of each selected line (<code>awk '{print \$N}'</code>).
<code>:running-total, :cumsum</code>	Replace each numeric line with the cumulative total so far.
<code>:diff-lines, :deltas</code>	Replace each numeric line with its delta from the previous (inverse of <code>running-total</code>).
<code>:sum-column, :sumcol</code>	Sum the Nth whitespace field across the selected lines (non-destructive).
<code>:shuffle, :shuf</code>	Randomly reorder the selected lines (Fisher-Yates).
<code>:sample, :random-lines</code>	Keep N random lines from the selection, preserving order (<code>:sample 10</code>).
<code>:jsonl-to-json, :jsonl-json</code>	Convert the selected JSONL/NDJSON (one value per line) to a JSON array.
<code>:json-to-jsonl, :json-jsonl</code>	Convert the selected JSON array to JSONL (one compact value per line).
<code>:head, :first-lines</code>	Keep only the first N lines of the selection (<code>:head 10</code>).
<code>:tail, :last-lines</code>	Keep only the last N lines of the selection (<code>:tail 10</code>).
<code>:rev, :reverse-each-line</code>	Reverse the characters of each selected line independently (Unix <code>rev</code>).
<code>:json-table, :json-tbl</code>	Render the selected JSON array of objects as an aligned plain-text table.
<code>:hexdump, :xxd</code>	Render the selection as an xxd-style hex dump (offset, hex bytes, ASCII).

Name	Description
<code>:dedup, :unique-lines, :delete-duplicate-lines</code>	Remove all duplicate lines globally, keeping first occurrence and order. Emacs delete-duplicate-lines.
<code>:caesar, :shift-letters</code>	Caesar-shift the selection's letters by N (e.g. <code>:caesar 13</code> = ROT13; N may be negative).
<code>:base32-encode, :base32</code>	Base32-encode the selection (RFC 4648).
<code>:base32-decode, :unbase32</code>	Base32-decode the selection (RFC 4648).
<code>:crc32, :checksum</code>	Show the CRC32 (IEEE) checksum of the selection in hex and decimal (non-destructive).
<code>:rot47, :rot-47</code>	Apply ROT47 to the selection (rotates all printable ASCII; self-inverse).
<code>:morse-encode, :morse</code>	Encode the selection (A-Z, 0-9) to Morse code (words separated by /).
<code>:morse-decode, :unmorse</code>	Decode Morse code in the selection back to text.
<code>:human-bytes, :humanize-size</code>	Convert each numeric line (a byte count) to a human-readable size like 1.5 KiB.
<code>:ordinal, :ordinalize</code>	Convert each numeric line to its ordinal (1 → 1st, 22 → 22nd).
<code>:to-snake, :snake-case</code>	Convert the selected identifier to snake_case.
<code>:to-kebab, :kebab-case</code>	Convert the selected identifier to kebab-case.
<code>:to-camel, :camel-case</code>	Convert the selected identifier to camelCase.
<code>:to-pascal, :pascal-case</code>	Convert the selected identifier to PascalCase.
<code>:to-constant, :screaming-snake, :upper-snake</code>	Convert the selected identifier to CONSTANT_CASE.
<code>:to-binary, :text-to-binary</code>	Convert the selection to space-separated 8-bit binary.
<code>:from-binary, :binary-to-text</code>	Convert space-separated binary in the selection back to text.
<code>:natural-sort, :sort-natural</code>	Sort the selected lines in natural order (file2 before file10).
<code>:pad-right, :ljust</code>	Left-justify each selected line, padding with spaces to a minimum width.
<code>:pad-left, :rjust</code>	Right-justify each selected line, padding with spaces to a minimum width.
<code>:json-keys, :json-fields</code>	List the keys of the selected JSON object (or union across an array of objects).
<code>:json-type, :json-describe</code>	Show the JSON type and size of the selection in the status line (non-destructive).
<code>:after, :cut-after</code>	Keep the text after the first on each selected line.
<code>:before, :cut-before</code>	Keep the text before the first on each selected line.
<code>:swapcase, :invert-case</code>	Invert the case of each character in the selection (Hello → hELLO).
<code>:strip-invisible, :strip-zero-width</code>	Remove zero-width / invisible Unicode characters from the selection.
<code>:lines-to-json, :lines-to-array</code>	Wrap the selected lines into a JSON array of strings.
<code>:json-to-lines, :array-to-lines</code>	Unwrap a JSON array in the selection into one line per element.
<code>:checkbox-list, :task-list</code>	Turn the selected lines into a Markdown task list (- [] item).
<code>:unwrap-paragraphs, :unhardwrap</code>	Join hard-wrapped lines within each paragraph into single lines.
<code>:sql-in, :sql-in-list</code>	Build a SQL IN-list ('a', 'b', 'c') from the selected lines.
<code>:dec-to-hex, :to-hex-num</code>	Convert each decimal number line to hexadecimal.

Name	Description
<code>:hex-to-dec, :from-hex-num</code>	Convert each hexadecimal number line to decimal.
<code>:unicode-escape, :u-escape</code>	Escape non-ASCII characters in the selection as <code>\u{XXXX}</code> .
<code>:unicode-unescape, :u-unescape</code>	Decode <code>\u{XXXX}</code> and <code>\uXXXX</code> escapes in the selection back to characters.
<code>:sort-by-length, :sortlen</code>	Sort the selected lines by length (shortest first).
<code>:count-unique, :distinct-count</code>	Report the number of distinct vs total selected lines (non-destructive).
<code>:rotate-lines, :rotate</code>	Cyclically rotate the selected lines by N (negative rotates the other way).
<code>:unquote-lines, :strip-quotes-lines</code>	Remove surrounding quotes from each selected line independently.
<code>:quote-lines, :quote-each</code>	Wrap each selected line in double quotes (escaping <code> and </code> ").
<code>:repeat, :repeat-text</code>	Repeat the selected text N times (<code>:repeat 3</code>).
<code>:capitalize-lines, :capitalize</code>	Uppercase the first letter of each selected line.
<code>:remove-blank-lines, :remove-empty</code>	Remove all blank (whitespace-only) lines from the selection.
<code>:trim-lines, :trim</code>	Trim leading and trailing whitespace from each selected line.
<code>:kv-to-json, :env-to-json</code>	Convert <code>key=value / key: value</code> lines in the selection to a JSON object.
<code>:json-to-kv, :json-to-env</code>	Convert the selected JSON object to <code>key=value</code> lines.
<code>:json-pluck, :json-values-of</code>	Extract one field's value from each object in a JSON array, one per line.
<code>:to-html-list, :html-list</code>	Convert the selected lines into an HTML
<code>:from-html-list, :html-list-to-lines</code>	Extract
<code>:csv-to-html-table, :csv-to-html</code>	Convert the selected CSV/TSV (first row = headers) to an HTML
<code>:slugify-lines, :slug-lines</code>	Slugify each selected line independently (URL-friendly).
<code>:lines-to-csv-row, :join-csv</code>	Join the selected lines into one CSV row (RFC-4180 quoting).
<code>:csv-row-to-lines, :split-csv</code>	Split a CSV row in the selection into one field per line (quote-aware).
<code>:deslugify, :unslugify</code>	Turn a slug back into a Title Cased phrase (hyphens/underscores to spaces).
<code>:csv-to-tsv, :csv-tsv</code>	Convert the selected CSV to tab-separated values (quote-aware).
<code>:tsv-to-csv, :tsv-csv</code>	Convert the selected TSV to CSV (RFC-4180 quoting).
<code>:strip-line-numbers, :unnumber</code>	Remove a leading line number (and separator) from each selected line.
<code>:markdown-link, :md-link</code>	Wrap the selected text as a Markdown link <code>text</code> .
<code>:extract-urls, :urls</code>	Replace the selection with the http(s) URLs found in it, one per line.
<code>:extract-emails, :emails</code>	Replace the selection with the email addresses found in it, one per line.
<code>:extract-ips, :ips</code>	Replace the selection with the IPv4 addresses found in it, one per line.
<code>:extract-quoted, :quoted-strings</code>	Replace the selection with the contents of double-quoted strings, one per line.
<code>:extract-between, :between</code>	Extract substrings between and delimiters, one per line.
<code>:wrap-with, :surround-with</code>	Wrap the selection with the given text on both sides (<code>:wrap-with **</code>).

Name	Description
<code>:extract-numbers, :numbers</code>	Replace the selection with the numbers found in it, one per line.
<code>:json-validate, :json-check</code>	Report whether the selection is valid JSON (with error location) — non-destructive.
<code>:csv-validate, :csv-check</code>	Check all CSV rows have the same field count (non-destructive).
<code>:ordered-list, :numbered-list</code>	Turn the selected lines into a Markdown ordered list (1. 2. 3.).
<code>:strip-list-markers, :unlist</code>	Strip leading bullet/number/checkbox list markers from each selected line.
<code>:sort-words, :sort-fields</code>	Sort the whitespace-separated words within each selected line.
<code>:unique-words, :dedup-words</code>	Remove duplicate words within each selected line (first occurrence kept).
<code>:sum-fields, :row-sum</code>	Replace each line with the sum of its numeric fields (row total).
<code>:avg-fields, :row-avg</code>	Replace each line with the mean of its numeric fields.
<code>:max-fields, :row-max</code>	Replace each line with the maximum of its numeric fields.
<code>:min-fields, :row-min</code>	Replace each line with the minimum of its numeric fields.
<code>:range-fields, :row-range</code>	Replace each line with the range (max - min) of its numeric fields.
<code>:to-env-export, :export-vars</code>	Prefix each KEY=value line with <code>export</code> (turn a .env into shell exports).
<code>:strip-export, :unexport</code>	Remove a leading <code>export</code> from each selected line.
<code>:dos2unix, :crlf-to-lf</code>	Convert CRLF/CR line endings in the selection to LF.
<code>:unix2dos, :lf-to-crlf</code>	Convert LF line endings in the selection to CRLF.
<code>:percent-of-total, :percentages</code>	Replace each numeric line with its percentage of the column total.
<code>:running-max, :cummax</code>	Replace each numeric line with the running maximum so far.
<code>:running-min, :cummin</code>	Replace each numeric line with the running minimum so far.
<code>:to-fixed, :round-to</code>	Format each numeric line to N decimal places (:to-fixed 2).
<code>:clamp, :clip</code>	Clamp each numeric line to the [min, max] range (:clamp 0 100).
<code>:scale, :multiply-by</code>	Multiply each numeric line by a factor (:scale 1.5).
<code>:offset, :add-to-each</code>	Add N to each numeric line (:offset 10; negative subtracts).
<code>:abs, :absolute-value</code>	Replace each numeric line with its absolute value.
<code>:linkify, :auto-link</code>	Wrap bare URLs in the selection with Markdown link syntax <code>url</code> .
<code>:strip-markdown-links, :unlink</code>	Replace <code>text</code> Markdown links with just their text.
<code>:strip-emphasis, :strip-md-emphasis</code>	Remove Markdown bold/italic/code emphasis markers from the selection.
<code>:strip-html-comments, :strip-comments-html</code>	Remove HTML/Markdown comments from the selection.
<code>:remove-trailing-commas, :fix-trailing-commas</code>	Remove trailing commas before } or] (JSON5/JS to strict JSON).
<code>:add-trailing-commas, :trailing-commas</code>	Add trailing commas before } or] (cleaner JS/JSON5 diffs).
<code>:smart-quotes, :curly-quotes</code>	Convert straight quotes to typographic curly quotes (context-aware).
<code>:typographic-dashes, :em-dash</code>	Convert — to em dash, – to en dash, ... to ellipsis.

Name	Description
<code>:de-typography, :ascii-punctuation</code>	Normalize curly quotes/dashes/ellipsis back to ASCII punctuation.
<code>:to-ascii, :transliterate</code>	Transliterate accented Latin characters to ASCII (café → cafe).
<code>:nato, :phonetic</code>	Spell the selection in the NATO phonetic alphabet (A → Alfa).
<code>:transpose-grid, :transpose-ws</code>	Transpose a whitespace-separated grid (rows become columns).
<code>:repeat-lines, :duplicate-each</code>	Repeat each selected line N times (:repeat-lines 3).
<code>:rename-word, :rename-local</code>	Rename every whole-word occurrence of the symbol under the cursor in this buffer.
<code>:grep-word, :gw, :find-references</code>	Search the project for the whole word under the cursor (jumpable in Run).
<code>:todos, :project-todos, :fixme</code>	Scan the whole project for TODO/FIXME/HACK/XXX/BUG/NOTE markers (jumpable in Run).
<code>:registers, :reg, :display</code>	Show the contents of all registers.
<code>:yank-join</code>	Yank joined selections. A separator can be provided as first argument. Default value is newline.
<code>:clipboard-yank</code>	Yank main selection into system clipboard.
<code>:clipboard-yank-join</code>	Yank joined selections into system clipboard. A separator can be provided as first argument. Default value is newline.
<code>:primary-clipboard-yank</code>	Yank main selection into system primary clipboard.
<code>:primary-clipboard-yank-join</code>	Yank joined selections into system primary clipboard. A separator can be provided as first argument. Default value is newline.
<code>:clipboard-paste-after</code>	Paste system clipboard after selections.
<code>:clipboard-paste-before</code>	Paste system clipboard before selections.
<code>:clipboard-paste-replace</code>	Replace selections with content of system clipboard.
<code>:primary-clipboard-paste-after</code>	Paste primary clipboard after selections.
<code>:primary-clipboard-paste-before</code>	Paste primary clipboard before selections.
<code>:primary-clipboard-paste-replace</code>	Replace selections with content of system primary clipboard.
<code>:show-clipboard-provider</code>	Show clipboard provider name in status bar.
<code>:change-current-directory, :cd, :chdir, :lcd, :lchdir, :tcd, :tchdir</code>	Change the current working directory.
<code>:show-directory-stack</code>	Show the directory stack as a delimited string.
<code>:push-directory, :pushd</code>	Save and then change the current directory.
<code>:pop-directory, :popd</code>	Remove the top entry from the directory stack, and cd to the new top directory..
<code>:show-directory, :pwd</code>	Show the current working directory.
<code>:encoding</code>	Set encoding. Based on https://encoding.spec.whatwg.org .
<code>:character-info, :char, :ascii</code>	Get info about the character under the primary cursor.
<code>:reload, :rl</code>	Discard changes and reload from the source file.
<code>:reload-all, :rla</code>	Discard changes and reload all documents from the source files.
<code>:checktime, :checkt</code>	Reload loaded buffers that changed on disk (vim :checktime).

Name	Description
<code>:filetype, :filet</code>	Report the buffer's detected language / accept on
<code>:scriptnames, :scr</code>	List the sourced config files (vim :scriptnames).
<code>:mksession, :mks</code>	Write a session file (cwd + buffers) that :source restores (vim :mksession).
<code>:mkvimrc, :mkv</code>	Write the current runtime mappings to a vimrc file (vim :mkvimrc; mappings only).
<code>:mkexrc, :mk</code>	Write the current runtime mappings to an exrc file (vim :mkexrc; mappings only).
<code>:mkview, :mkvie</code>	Write the current window's view (cursor position) to a file (vim :mkview).
<code>:loadview, :lo</code>	Restore the current window's view by sourcing its :mkview file (vim :loadview).
<code>:swapname, :sw</code>	Show the current buffer's swap file name (vim :swapname).
<code>:preserve, :pre</code>	Flush the buffer to its swap file now (vim :preserve).
<code>:recover, :rec</code>	Replace the buffer with the contents of its swap file (vim :recover).
<code>:wundo, :wun</code>	Write the buffer's undo history to {file} (vim :wundo).
<code>:rundo, :rund</code>	Read the buffer's undo history from {file} (vim :rundo).
<code>:git-stage, :stage, :git-add</code>	Stage the current buffer's file (git add).
<code>:vc-root-version-diff, :vc-root-diff</code>	Unified diff of the whole working tree vs a revision, default HEAD (emacs vc-root-version-diff).
<code>:vc-revision-other-window</code>	Show a past revision of the current file, default HEAD (emacs vc-revision-other-window).
<code>:git-unstage, :unstage</code>	Unstage the current buffer's file (git reset HEAD).
<code>:stash, :git-stash</code>	git stash the working-tree changes (then reload open buffers).
<code>:stash-pop, :git-stash-pop</code>	git stash pop the most recent stash (then reload open buffers).
<code>:update, :u</code>	Write changes only if the file has been modified.
<code>:lsp-workspace-command</code>	Open workspace command picker
<code>:lsp-restart</code>	Restarts the given language servers, or all language servers that are used by the current file if no arguments are supplied
<code>:lsp-health, :lsp-status</code>	Show a health report of language servers: which are ready, initializing, or not running, plus each server's supported features
<code>:set, :se, :setg, :setglobal, :setl, :setlocal</code>	Set options with vim syntax (:set nu, :set nowrap, :set tw=80); no args lists all options.
<code>:map</code>	Map {lhs} to {rhs} in normal+select modes (Vim :map).
<code>:noremap</code>	Non-recursive :map in normal+select modes.
<code>:nmap</code>	Map {lhs} to {rhs} in normal mode (Vim :nmap).
<code>:nnoremap</code>	Non-recursive normal-mode map (Vim :nnoremap).
<code>:imap</code>	Map {lhs} to {rhs} in insert mode (Vim :imap).
<code>:inoremap</code>	Non-recursive insert-mode map (Vim :inoremap).
<code>:vmap</code>	Map {lhs} to {rhs} in select/visual mode (Vim :vmap).
<code>:vnoremap</code>	Non-recursive select/visual-mode map (Vim :vnoremap).
<code>:xmap</code>	Map {lhs} to {rhs} in visual mode (Vim :xmap).

Name	Description
<code>:xnoremap</code>	Non-recursive visual-mode map (Vim <code>:xnoremap</code>).
<code>:smap</code>	Map {lhs} to {rhs} in select mode (Vim <code>:smap</code>).
<code>:snoremap</code>	Non-recursive select-mode map (Vim <code>:snoremap</code>).
<code>:omap</code>	Map {lhs} to {rhs} in operator-pending mode (Vim <code>:omap</code>).
<code>:onoremap</code>	Non-recursive operator-pending map (Vim <code>:onoremap</code>).
<code>:unmap</code>	Remove a runtime {lhs} mapping (normal+select).
<code>:nunmap</code>	Remove a runtime normal-mode {lhs} mapping.
<code>:iunmap</code>	Remove a runtime insert-mode {lhs} mapping.
<code>:vunmap</code>	Remove a runtime select/visual-mode {lhs} mapping.
<code>:xunmap</code>	Remove a runtime visual-mode {lhs} mapping.
<code>:mapclear</code>	Clear runtime normal+select-mode mappings.
<code>:nmapclear</code>	Clear runtime normal-mode mappings.
<code>:imapclear</code>	Clear runtime insert-mode mappings.
<code>:vmapclear</code>	Clear runtime select/visual-mode mappings.
<code>:xmapclear</code>	Clear runtime visual-mode mappings (Vim <code>:xmapclear</code>).
<code>:smapclear</code>	Clear runtime select-mode mappings (Vim <code>:smapclear</code>).
<code>:omapclear</code>	Clear runtime operator-pending mappings (Vim <code>:omapclear</code>).
<code>:sunmap</code>	Remove a runtime select-mode {lhs} mapping (Vim <code>:sunmap</code>).
<code>:ounmap</code>	Remove a runtime operator-pending {lhs} mapping (Vim <code>:ounmap</code>).
<code>:normal, :norm</code>	Execute {commands} as normal-mode keystrokes (vim <code>:normal[!]</code>).
<code>:mark, :k</code>	Set mark {x} at the cursor position (vim <code>:mark / :k</code>).
<code>:buffer, :buf, :b</code>	Switch to the open buffer whose path contains {name} (vim <code>:buffer / :b</code>).
<code>:resize, :res</code>	Adjust the current window height (vim <code>:resize [+/-]{N}</code>).
<code>:let</code>	Set a vimscript variable via the embedded interpreter (<code>:let x = 42</code>).
<code>:Files</code>	Fuzzy-find files with fzf and open the selection (fzf.vim <code>:Files</code>).
<code>:GFiles, :GitFiles</code>	Fuzzy-find git-tracked files with fzf and open the pick (fzf.vim <code>:GFiles</code>).
<code>:Rg, :Ag, :RG</code>	Ripgrep the tree with fzf; open the pick at its line (fzf.vim <code>:Rg/:Ag</code>).
<code>:Todo, :TODO, :Todos</code>	TODO tool window: ripgrep TODO/FIXME/HACK/XXX across the tree, jump to the pick.
<code>:Locate</code>	locate(1) files with fzf and open the pick (fzf.vim <code>:Locate</code>).
<code>:BLines</code>	Fuzzy-search the current buffer's lines with fzf, jump to the pick (fzf.vim <code>:BLines</code>).
<code>:Lines</code>	Fuzzy-search lines across all open buffers with fzf, open the pick (fzf.vim <code>:Lines</code>).
<code>:History</code>	Fuzzy-pick a recently opened file with fzf and open it (fzf.vim <code>:History</code>).
<code>:History:</code>	Fuzzy-pick a past <code>:</code> command with fzf and run it (fzf.vim <code>:History:</code>).

Name	Description
<code>:History/</code>	Fuzzy-pick a past search with fzf and re-run it (fzf.vim :History/).
<code>:Filetypes</code>	Fuzzy-pick a language with fzf and set the buffer's filetype (fzf.vim :Filetypes).
<code>:Commits</code>	Fuzzy-pick a repo commit with fzf and show it (fzf.vim :Commits).
<code>:BCommits</code>	Fuzzy-pick a commit touching the current file with fzf and show it (fzf.vim :BCommits).
<code>:fzf-git-show</code>	(internal) show a <code><sha> subject</code> fzf pick via git show.
<code>:Jumps</code>	Fuzzy-pick a jumplist entry with fzf and open it (fzf.vim :Jumps).
<code>:RecentLocations, :recent-locations</code>	Recent Locations (JetBrains): jump ring newest-first, deduped, with context.
<code>:LocalHistory, :local-history</code>	Local History (JetBrains): pick a saved snapshot of this file and open it.
<code>:local-history-open</code>	(internal) open a <code>:LocalHistory</code> snapshot pick.
<code>:Windows</code>	Fuzzy-pick an open window with fzf and focus it (fzf.vim :Windows).
<code>:Marks</code>	Fuzzy-pick a mark in the current buffer with fzf and jump to it (fzf.vim :Marks).
<code>:Tags</code>	Fuzzy-pick a ctags tag across the tree with fzf and jump to it (fzf.vim :Tags).
<code>:BTags</code>	Fuzzy-pick a ctags tag in the current file with fzf and jump to it (fzf.vim :BTags).
<code>:Snippets</code>	Fuzzy-pick a snippet with fzf and insert its body (fzf.vim :Snippets).
<code>:Maps</code>	Fuzzy-browse the current keymaps with fzf (fzf.vim :Maps).
<code>:Helptags</code>	Fuzzy-pick a help tag with fzf and open the Help browser at it (fzf.vim :Helptags).
<code>:fzf-window</code>	(internal) focus the Nth window of a <code>:Windows</code> fzf pick.
<code>:fzf-snippet</code>	(internal) insert the body of a <code>:Snippets</code> fzf pick.
<code>:fzf-map</code>	(internal) echo a <code>:Maps</code> fzf pick.
<code>:fzf-helptag</code>	(internal) open the Help browser at a <code>:Helptags</code> fzf pick.
<code>:fzf-goto</code>	(internal) open a <code>file:line:col:text</code> fzf pick at the line.
<code>:fzf-line</code>	(internal) jump to the line of an <code>N: text</code> fzf pick.
<code>:Colors</code>	Fuzzy-pick a colorscheme with fzf (fzf.vim :Colors).
<code>:Buffers</code>	Fuzzy-pick an open buffer with fzf and switch to it (fzf.vim :Buffers).
<code>:Commands</code>	Fuzzy-pick a <code>:</code> command with fzf and run it (fzf.vim :Commands).
<code>:fold, :fo</code>	Create a fold over the selected/current lines (vim :fold).
<code>:foldopen, :foldo</code>	Open the fold under the cursor (vim :foldopen).
<code>:foldclose, :foldc</code>	Close the fold under the cursor (vim :foldclose).
<code>:G, :GG, :s-repeat</code>	Repeat the last <code>:substitute</code> on the current line (vim <code>:& / :&&</code>).
<code>:sleep, :sl</code>	Do nothing for {count} seconds (vim :sleep).
<code>:echomsg, :echom</code>	Echo an expression and save it in the message history (vim :echomsg).
<code>:eval, :ev</code>	Evaluate an expression and discard the result (vim :eval).

Name	Description
<code>:call, :cal</code>	Call a function (vim :call).
<code>:execute, :exe</code>	Execute the string result of an expression as an Ex command (vim :execute).
<code>:const, :cons</code>	Create a variable as a constant (vim :const).
<code>:unlet, :unl</code>	Delete a variable (vim :unlet).
<code>:ptnext, :ptn</code>	Show the next matching tag in a preview split (vim :ptnext).
<code>:ptprevious, :ptp, :ptNext, :ptN</code>	Show the previous matching tag in a preview split (vim :ptprevious / :ptNext).
<code>:ptfirst, :ptrewind, :ptr</code>	Show the first matching tag in a preview split (vim :ptfirst / :ptrewind).
<code>:ptlast, :ptl</code>	Show the last matching tag in a preview split (vim :ptlast).
<code>:pclose, :pc</code>	Close the preview window (vim :pclose).
<code>:startgreplace, :startg</code>	Start Virtual Replace mode (vim :startgreplace).
<code>:print, :p</code>	Display the selected lines (or current line) in a scratch buffer (vim :print).
<code>:number, :nu, :#</code>	Like :print, with line numbers (vim :number / :#).
<code>:list, :l</code>	Like :print, marking each line end with \$ (vim :list).
<code>:print-line-number, :=</code>	Echo the last line number of the buffer (vim :=).
<code>:version, :ver</code>	Show the zemacs version and compiled feature summary (vim :version).
<code>:intro, :int</code>	Show the introductory message (vim :intro).
<code>:redrawstatus</code>	Redraw the status line (vim :redrawstatus; approximated by a full redraw).
<code>:redrawtabline</code>	Redraw the tab line (vim :redrawtabline; approximated by a full redraw).
<code>:silent, :sil</code>	Run {cmd} silently (vim :silent[!]); message suppression is best-effort.
<code>:unsilent</code>	Run {cmd} with messages shown (vim :unsilent).
<code>:verbose, :verb</code>	Run {cmd} verbosely, optional leading count (vim :verbose).
<code>:noautocmd, :noa</code>	Run {cmd} without triggering autocommands (vim :noautocmd).
<code>:keepalt, :keepa</code>	Run {cmd} keeping the alternate file (vim :keepalt).
<code>:keepjumps, :keepj</code>	Run {cmd} without changing the jumplist (vim :keepjumps).
<code>:keepmarks, :kee</code>	Run {cmd} keeping marks (vim :keepmarks).
<code>:keeppatterns, :keepp</code>	Run {cmd} keeping the search pattern (vim :keeppatterns).
<code>:lockmarks, :loc</code>	Run {cmd} without adjusting marks (vim :lockmarks).
<code>:sandbox, :san</code>	Run {cmd} in the sandbox (vim :sandbox; best-effort).
<code>:confirm, :conf</code>	Run {cmd} confirming risky actions (vim :confirm; best-effort).
<code>:browse, :bro</code>	Run {cmd} (vim :browse; file dialog not applicable in a TUI).
<code>:noswapfile, :noswap</code>	Run {cmd} without a swapfile (vim :noswapfile).
<code>:hide, :hid</code>	Run {cmd} keeping the current buffer hidden (vim :hide).
<code>:vertical, :vert</code>	Run {cmd} with vertical split placement (vim :vertical; best-effort).

Name	Description
<code>:horizontal, :hor</code>	Run {cmd} with horizontal split placement (vim :horizontal).
<code>:aboveleft, :abo</code>	Run {cmd} placing a new split above/left (vim :aboveleft; best-effort).
<code>:belowright, :bel</code>	Run {cmd} placing a new split below/right (vim :belowright; best-effort).
<code>:leftabove, :lefta</code>	Run {cmd} placing a new split left/above (vim :leftabove; best-effort).
<code>:rightbelow, :rightb</code>	Run {cmd} placing a new split right/below (vim :rightbelow; best-effort).
<code>:topleft, :to</code>	Run {cmd} placing a new split at the top/left (vim :topleft; best-effort).
<code>:botright, :bo</code>	Run {cmd} placing a new split at the bottom/right (vim :botright; best-effort).
<code>:tab</code>	Run {cmd} opening its window in a new tab (vim :tab; best-effort placement).
<code>:lsp-stop</code>	Stops the given language servers, or all language servers that are used by the current file if no arguments are supplied
<code>:tree-sitter-scopes</code>	Display tree sitter scopes, primarily for theming and development.
<code>:tree-sitter-highlight-name, :Inspect</code>	Display the tree-sitter highlight capture under the cursor (neovim :Inspect).
<code>:tree-sitter-layers</code>	Display language names of tree-sitter injection layers under the cursor.
<code>:debug-start, :dbg</code>	Start a debug session from a given template with given parameters.
<code>:debug-remote, :dbg-tcp</code>	Connect to a debug adapter by TCP address and start a debugging session from a given template with given parameters.
<code>:debug-eval</code>	Evaluate expression in current debug context.
<code>:close, :clo</code>	Close the current window (vim :close). Refuses to close the last window.
<code>:only, :on</code>	Close all windows except the current one (vim :only).
<code>:vsplit, :vs</code>	Open the file in a vertical split.
<code>:vsplit-new, :vnew</code>	Open a scratch buffer in a vertical split.
<code>:hsplit, :hs, :sp, :split</code>	Open the file in a horizontal split.
<code>:hsplit-new, :hnew</code>	Open a scratch buffer in a horizontal split.
<code>:tutor</code>	Open the tutorial.
<code>:goto, :g</code>	Goto line number.
<code>:set-language, :lang, :setf, :setfiletype</code>	Set the language of current buffer (show current language if no value specified).
<code>:lpr-buffer, :print-buffer</code>	Print the whole buffer via the external lpr spooler (emacs lpr-buffer).
<code>:lpr-region, :print-region</code>	Print the selected region via the external lpr spooler (emacs lpr-region).
<code>:dictionary-search, :dictionary</code>	Look up a word (or the word at point) with the external dict client (emacs dictionary-search).
<code>:calendar-hebrew-list-yahrzeits, :list-yahrzeit-dates</code>	List a Hebrew death date's yahrzeit Gregorian dates over N years (emacs calendar-hebrew-list-yahrzeits).
<code>:eldoc-mode, :global-eldoc-mode, :turn-on-eldoc-mode</code>	Toggle automatic signature/parameter hints at point (emacs eldoc-mode).
<code>:normal-mode</code>	Re-detect the buffer's major mode from its file (emacs normal-mode).

Name	Description
<code>:text-mode, :fundamental-mode</code>	Switch the buffer to plain text with no code syntax (emacs text-mode/fundamental-mode).
<code>:set-option</code>	Set a config option at runtime. For example to disable smart case search, use <code>:set-option search.smart-case false</code> .
<code>:toggle-option, :toggle</code>	Toggle a config option at runtime. For example to toggle smart case search, use <code>:toggle search.smart-case</code> .
<code>:get-option, :get</code>	Get the current value of a config option.
<code>:move-line-down</code>	Move the current line down by one (drag down).
<code>:move-line-up</code>	Move the current line up by one (drag up).
<code>:cycle-case</code>	Cycle the case style of the symbol under the cursor.
<code>:change-case</code>	Change the symbol under the cursor to camel
<code>:left, :le</code>	Left-align line(s), setting leading indent to {n} (default 0) — vim :left.
<code>:right, :ri</code>	Right-align line(s) to width {n} (default 80) — vim :right.
<code>:center, :ce</code>	Center line(s) within width {n} (default 80) — vim :center.
<code>:undo</code>	Undo the last change (vim :undo).
<code>:redo, :red</code>	Redo the last undone change (vim :redo).
<code>:retab</code>	Replace tabs with spaces (tab-width per buffer) — vim :retab.
<code>:join, :j</code>	Join the current line(s) with the next, separated by a space (vim :j).
<code>:join!, :j!</code>	Join the current line(s) with the next, no separating space (vim :j!).
<code>:put, :pu</code>	Put (paste) a register's contents as new line(s) below the cursor (vim :put).
<code>:put!, :pu!</code>	Put (paste) a register's contents as new line(s) above the cursor (vim :put!).
<code>:iput, :ip</code>	Put a register's contents below the cursor, indenting to the current line (vim :iput).
<code>:execute-register, :@</code>	Execute a register's contents as Ex command line(s) (vim :@{reg}).
<code>:ijump, :ij</code>	Jump to the first line containing an identifier (vim :ijump).
<code>:djump, :dj</code>	Jump to the first #define of a macro (vim :djump).
<code>:isplit, :isp</code>	Split the window and jump to the first line containing an identifier (vim :isplit).
<code>:dsplit, :dsp</code>	Split the window and jump to the first #define of a macro (vim :dsplit).
<code>:ilist, :il</code>	List every line containing an identifier in a scratch buffer (vim :ilist).
<code>:digraphs, :dig</code>	List the digraph table in a scratch buffer (vim :digraphs).
<code>:z</code>	Print a window of lines from the cursor into a scratch buffer (vim :z).
<code>:checkpath, :checkp</code>	List the files #included by the current buffer in a scratch buffer (vim :checkpath).
<code>:exusage, :exu</code>	List the available Ex commands in a scratch buffer (vim :exusage).
<code>:viusage, :viu</code>	List the Normal-mode commands in a scratch buffer (vim :viusage).
<code>:dlist, :dli</code>	List every #define line of a macro in a scratch buffer (vim :dlist).

Name	Description
<code>:isearch, :is</code>	Echo the first line containing an identifier (vim :isearch).
<code>:dsearch, :ds</code>	Echo the first #define line of a macro (vim :dsearch).
<code>:delete-lines, :d, :del, :delete</code>	Delete the current line(s) into the unnamed register (vim :d).
<code>:dl</code>	Delete the current line(s) and echo the last deleted line in :list format (vim :dl).
<code>:yank-lines, :y, :ya, :yank</code>	Yank the current line(s) into the unnamed register (vim :y).
<code>:indent-lines</code>	Indent the current line(s) by one shiftwidth (vim :>).
<code>:dedent-lines</code>	Dedent the current line(s) by one shiftwidth (vim :<).
<code>:move-lines, :m</code>	Move the current line to after line {address}: :m{addr} (e.g. :m0, :m\$, :m.+2).
<code>:copy-lines, :t, :co, :copy</code>	Copy the current line to after line {address}: :t{addr} (e.g. :t0, :t\$).
<code>:global</code>	Run a command on matching lines: :g/pattern/d (delete). Also :g!/pat/d.
<code>:vglobal</code>	Run a command on non-matching lines: :v/pattern/d (delete).
<code>:substitute, :s</code>	Substitute: :s/pattern/replacement/[flags]. Also :%s/.../.../g for the whole file.
<code>:smagic</code>	Substitute forcing 'magic': :smagic/pattern/replacement/[flags] (vim :smagic).
<code>:snomagic</code>	Substitute forcing 'nomagic' (pattern literal): :snomagic/pattern/replacement/[flags] (vim :snomagic).
<code>:replace-word, :rw, :subword</code>	Global replace of the word under the cursor across the file: :replace-word bar → :%s/\bfoo\b/bar/g. Add i as a 2nd arg for case-insensitive.
<code>:Subvert, :S</code>	vim-abolish case-preserving substitute: :S/foo/bar/g rewrites foo/Foo/FOO → bar/Bar/BAR.
<code>:Thesaurus, :thesaurus</code>	Look up synonyms for the word under the cursor (or :Thesaurus word) and replace it.
<code>:split-line</code>	Split the current line at the cursor, keeping the cursor in place.
<code>:just-one-space</code>	Collapse spaces and tabs around the cursor to a single space.
<code>:delete-horizontal-space</code>	Delete all spaces and tabs around the cursor (emacs M-).
<code>:fixup-whitespace</code>	Collapse whitespace around the cursor to one space, or none by context (emacs fixup-whitespace).
<code>:cycle-spacing</code>	Cycle the whitespace around the cursor: one space, then none, then restore.
<code>:tabify</code>	Convert runs of spaces in the region to tabs at tab stops (emacs tabify).
<code>:untabify</code>	Expand tabs in the region to spaces at the buffer tab width (emacs untabify).
<code>:delete-blank-lines</code>	Collapse consecutive blank lines down to a single blank line.
<code>:uniquify-lines, :uniq</code>	Delete duplicate lines, keeping the first occurrence.
<code>:reverse, :reverse-lines, :reverse-region, :tac</code>	Reverse the order of the selected lines (or the whole buffer). Emacs reverse-region.
<code>:uuid, :guid</code>	Insert a random UUID v4 at each cursor (replaces any selection).
<code>:goto-offset, :goto-char</code>	Move the cursor to an absolute character offset.
<code>:goto-byte, :go, :gob</code>	Move the cursor to a 1-based byte offset (Vim :goto).
<code>:pad-numbers, :zero-pad</code>	Zero-pad every integer in the selection to digits.

Name	Description
<code>:increment-numbers, :incr-numbers</code>	Add N (default 1; negative to decrement) to every integer in the selection.
<code>:bases, :base-info</code>	Show the selected integer in decimal, hex, octal, and binary.
<code>:lorem, :lipsum</code>	Insert N words (default 30) of lorem-ipsun placeholder text.
<code>:date</code>	Insert the current UTC date (YYYY-MM-DD) at each cursor.
<code>:datetime, :now</code>	Insert the current UTC date and time (YYYY-MM-DD HH:MM:SS) at each cursor.
<code>:timestamp, :epoch</code>	Insert the current Unix epoch (seconds) at each cursor.
<code>:sum, :total</code>	Sum the numbers in the selection; reports sum/avg/min/max/count in the status line.
<code>:calc, :eval-math</code>	Evaluate an arithmetic expression (+ - * / % ^), or each selection in place.
<code>:join-with, :joinw</code>	Join the selected lines into one with a separator (default “,”).
<code>:split-on, :splito</code>	Split the selected line(s) on a separator (default “,”) into one item per line.
<code>:squeeze-blank-lines, :squeeze</code>	Collapse consecutive blank lines in the selection to one (cat -s).
<code>:dedup-adjacent, :uniq-adjacent</code>	Collapse consecutive duplicate lines in the selection (Unix uniq).
<code>:number-lines, :nl</code>	Prepend line numbers to the selected lines (optional start, default 1).
<code>:string-rectangle, :string-replace-rectangle</code>	Replace the selected rectangle's column span with a string on every line (emacs C-x r t).
<code>:string-insert-rectangle</code>	Insert a string at the rectangle's left column on every selected line, shifting text right (emacs string-insert-rectangle).
<code>:align, :tabularize</code>	Align the selected lines on a delimiter (default =) so it shares a column.
<code>:sort-by-field, :sortf</code>	Sort the selected lines by their Nth whitespace field (default 1).
<code>:sort-numeric-fields, :sortnf</code>	Sort the selected lines by the numeric value of their Nth whitespace field (default 1).
<code>:sort-columns, :sortc</code>	Sort the selected lines alphabetically by the character-column range [beg, end).
<code>:sort-paragraphs, :sortp</code>	Sort the paragraphs (blank-line separated blocks) of the selection alphabetically.
<code>:fill-individual-paragraphs</code>	Fill each paragraph of the selection separately, splitting on indentation changes and using each paragraph's indentation as its fill prefix (emacs fill-individual-paragraphs).
<code>:fill-nonuniform-paragraphs</code>	Fill each paragraph of the selection separately, splitting only on blank lines and using the smallest indentation of each paragraph as its fill prefix (emacs fill-nonuniform-paragraphs).
<code>:sort-pages</code>	Sort the ^L-delimited pages in the selection (or buffer) alphabetically (emacs sort-pages).
<code>:sort-lines, :sortl</code>	Sort the selected lines (or the whole buffer) — vim-style line sort.
<code>:highlight-regexp, :hi-lock</code>	Add a persistent highlight for the regexp (emacs highlight-regexp).
<code>:highlight-phrase</code>	Highlight the phrase, matching across whitespace/line breaks (emacs highlight-phrase).
<code>:highlight-lines-matching-regexp</code>	Highlight whole lines matching the regexp (emacs highlight-lines-matching-regexp).
<code>:unhighlight-regexp</code>	Remove the highlight for the regexp, or all highlights if none given (emacs unhighlight-regexp).
<code>:highlight-symbol-at-point, :hi-lock-face-symbol-at-point</code>	Highlight every whole-word occurrence of the symbol under the cursor (emacs highlight-symbol-at-point).

Name	Description
<code>:hi-lock-write-interactive-patterns</code>	Insert the active Hi-Lock patterns at point as commented Hi-lock: lines (emacs hi-lock-write-interactive-patterns).
<code>:hi-lock-find-patterns</code>	Activate Hi-Lock highlights from commented Hi-lock: lines at the top of the buffer (emacs hi-lock-find-patterns).
<code>:outline-hide-by-heading-regexp</code>	Fold the subtree of every heading whose line matches the regexp (emacs outline-hide-by-heading-regexp).
<code>:outline-show-by-heading-regexp</code>	Reveal the subtree of every heading whose line matches the regexp (emacs outline-show-by-heading-regexp).
<code>:set-fill-column</code>	Set the fill width to N, or the current cursor column if omitted (emacs set-fill-column).
<code>:customize</code> , <code>:customize-browse</code>	Open Preferences on the Settings tab (emacs customize / customize-browse).
<code>:customize-variable</code> , <code>:customize-option</code>	Open Settings pre-filtered to a variable name (emacs customize-variable).
<code>:customize-group</code>	Open Settings pre-filtered to a group name (emacs customize-group).
<code>:customize-apropos</code>	Open Settings pre-filtered to a regexp/substring (emacs customize-apropos).
<code>:customize-unsaved</code> , <code>:customize-changed</code> , <code>:customize-saved</code>	Open Settings showing only options changed from their default (emacs customize-unsaved).
<code>:customize-face</code> , <code>:customize-themes</code> , <code>:customize-create-theme</code>	Open the Color Scheme (theme/face editor) tab (emacs customize-face / customize-themes).
<code>:set-face-foreground</code>	Set a theme face's foreground color: <code>:set-face-foreground</code> (emacs set-face-foreground).
<code>:set-face-background</code>	Set a theme face's background color: <code>:set-face-background</code> (emacs set-face-background).
<code>:set-fringe-style</code>	Set the gutter (fringe) style: none clears it, default restores it (emacs set-fringe-style).
<code>:set-right-margin</code>	Re-fill the region to N columns narrower than text-width (emacs set-right-margin).
<code>:write-region</code>	Write the region (or whole buffer) to a file, overwriting it (emacs write-region).
<code>:append-to-file</code>	Append the region (or whole buffer) to the end of a file (emacs append-to-file).
<code>:set-justification-left</code>	Flush the region's lines to the left margin (emacs set-justification-left).
<code>:set-justification-right</code>	Right-justify the region's lines to the fill width (emacs set-justification-right).
<code>:set-justification-center</code>	Centre the region's lines within the fill width (emacs set-justification-center).
<code>:set-justification-full</code>	Justify the region's lines to both margins (emacs set-justification-full).
<code>:set-justification-none</code>	Turn justification off for the region (emacs set-justification-none).
<code>:set-left-margin</code>	Set the region's left margin to a column of spaces (emacs set-left-margin).
<code>:increase-left-margin</code>	Indent the region by standard-indent columns (emacs increase-left-margin).
<code>:decrease-left-margin</code>	Outdent the region by standard-indent columns (emacs decrease-left-margin).
<code>:append-to-buffer</code>	Insert the region at the end of another open buffer (emacs append-to-buffer).
<code>:prepend-to-buffer</code>	Insert the region at the start of another open buffer (emacs prepend-to-buffer).
<code>:copy-to-buffer</code>	Replace another open buffer's contents with the region (emacs copy-to-buffer).
<code>:rename-buffer</code>	Give the current buffer an explicit display name (emacs rename-buffer).

Name	Description
<code>:rename-uniquely</code>	Rename the current buffer to a unique name with a numeric suffix (emacs <code>rename-uniquely</code>).
<code>:desktop-save</code>	Save file-visiting buffers and point to a desktop file (emacs <code>desktop-save</code>).
<code>:desktop-read</code>	Reopen the buffers recorded in a desktop file (emacs <code>desktop-read</code>).
<code>:desktop-change-dir</code>	Switch to the desktop saved in another directory (emacs <code>desktop-change-dir</code>).
<code>:desktop-revert</code>	Re-read the last loaded desktop (emacs <code>desktop-revert</code>).
<code>:desktop-clear</code>	Close all buffers and erase the desktop file (emacs <code>desktop-clear</code>).
<code>:expand-region-abbrevs</code>	Expand every abbrev found in the region (emacs <code>expand-region-abbrevs</code>).
<code>:getenv</code>	Report the value of an environment variable (emacs <code>getenv</code>).
<code>:setenv</code>	Set (or, with no value, unset) an environment variable (emacs <code>setenv</code>).
<code>:apropos-command</code>	List commands whose name matches a regexp (emacs <code>apropos-command</code>).
<code>:apropos</code>	List commands (by name or doc) and config variables matching a regexp (emacs <code>apropos</code>).
<code>:apropos-documentation</code>	List commands whose documentation matches a regexp (emacs <code>apropos-documentation</code>).
<code>:apropos-variable</code>	List config variables whose name matches a regexp (emacs <code>apropos-variable</code>).
<code>:apropos-user-option</code>	List user-customizable options whose name matches a regexp (emacs <code>apropos-user-option</code>).
<code>:apropos-value</code>	List config variables whose value matches a regexp (emacs <code>apropos-value</code>).
<code>:goto-line-relative</code>	Go to a line counting from the narrowed region start (emacs <code>goto-line-relative</code>).
<code>:emacs-version</code>	Report the running editor version (emacs <code>emacs-version</code>).
<code>:add-name-to-file</code>	Hard-link the current buffer's file to a new name (emacs <code>add-name-to-file</code>).
<code>:browse-url</code>	Open a URL in the OS default browser (emacs <code>browse-url</code>).
<code>:set-visited-file-name</code>	Change the file the current buffer is visiting (emacs <code>set-visited-file-name</code>).
<code>:make-symbolic-link</code>	Create a symbolic link (emacs <code>make-symbolic-link</code>): <code>:</code> or for the current file.
<code>:set-file-modes</code>	Set the mode bits of a file, <code>chmod-style</code> , <code>MODE</code> as octal (emacs <code>set-file-modes</code>).
<code>:copy-directory</code>	Recursively copy a directory to a destination (emacs <code>copy-directory</code>).
<code>:insert-file-literally</code>	Insert the raw contents of a file at point (emacs <code>insert-file-literally</code>).
<code>:list-directory</code>	Show a listing of a directory in a scratch buffer (emacs <code>list-directory</code>).
<code>:write-abbrev-file</code>	Write every abbrev to the named file (emacs <code>write-abbrev-file</code>).
<code>:read-abbrev-file</code>	Read abbrevs from the named file, merging them into the table (emacs <code>read-abbrev-file</code>).
<code>:bookmark-write</code>	Write every bookmark to the named file (emacs <code>bookmark-write</code>).
<code>:bookmark-load</code>	Load bookmarks from the named file, merging them into the store (emacs <code>bookmark-load</code>).
<code>:multi-occur-in-matching-buffers, :multi-occur</code>	List matches for <code>SEARCH-REGEXP</code> across all buffers whose name matches <code>BUFFER-REGEXP</code> (emacs <code>multi-occur-in-matching-buffers</code>).

Name	Description
<code>:transpose-regions</code>	Swap two char ranges: START1 END1 START2 END2 (emacs transpose-regions).
<code>:transpose-words</code>	Transpose the word before the cursor with the word after it.
<code>:transpose-chars</code>	Transpose the two characters around the cursor.
<code>:duplicate-line, :dup</code>	Duplicate the current line below.
<code>:delete-trailing-whitespace, :dtw</code>	Delete trailing whitespace from every line in the buffer.
<code>:sort</code>	Sort ranges in selection.
<code>:reflow</code>	Hard-wrap the current selection of lines to a given width.
<code>:tree-sitter-subtree, :ts-subtree, :InspectTree</code>	Display the smallest tree-sitter subtree that spans the primary selection (neovim :InspectTree).
<code>:config-reload</code>	Refresh user config.
<code>:keymap</code>	Switch the active keymap preset: spacemacs, vim, helix, or emacs.
<code>:config-open</code>	Open the user config.toml file.
<code>:config-open-workspace</code>	Open the workspace config.toml file.
<code>:log-open</code>	Open the zemacs log file.
<code>:insert-output</code>	Run shell command, inserting output before each selection.
<code>:append-output</code>	Run shell command, appending output after each selection.
<code>:pipe, :\ </code>	Pipe each selection to the shell command.
<code>:pipe-to</code>	Pipe each selection to the shell command, ignoring output.
<code>:run-shell-command, :sh, :!</code>	Run a shell command
<code>:elisp, :eval-expression, :el</code>	Evaluate an Emacs Lisp expression against the editor (embedded elisprs).
<code>:vim, :viml, :vimscript</code>	Evaluate a Vimscript (VimL) expression via the embedded vimlrs interpreter.
<code>:awk, :awk-filter</code>	Filter the selection (or whole buffer) through an awk program (embedded awkrs).
<code>:zsh, :zshell</code>	Run a command in the embedded zsh shell (state persists); output shown in a popup.
<code>:stryke, :st</code>	Evaluate stryke (strykelang) source via the embedded interpreter (state persists).
<code>:zwire-host, :zh</code>	Send a raw JSON request to the zwire-host daemon; show the reply (e.g. {"cmd":"hostinfo"}).
<code>:zwire-sysinfo, :zsys</code>	Show live system stats (cpu/mem/load) from the shared zwire-host daemon.
<code>:zwire-hostinfo</code>	Show machine facts (os/arch/cpus/hostname) from the shared zwire-host daemon.
<code>:zwire-exec, :zx</code>	Run a command through the zwire-host daemon and insert its output at the cursor.
<code>:zwire-crawl, :zwc</code>	Recursively crawl the filesystem via zwire-host and insert matching paths at the cursor.
<code>:zwire-job, :zj</code>	Ship a long-running command to the zwire-host daemon; get notified on the status line when it finishes.
<code>:zwire-jobs, :jobs, :zjs</code>	List background zwire-host jobs still running, plus recent completions.
<code>:zwire-job-output, :zjo</code>	Insert a finished background job's output at the cursor (most recent, or a given id).
<code>:repl</code>	Open the embedded-language REPL (elisp/viml/stryke/awk/zsh); optional starting language.

Name	Description
<code>:reset-diff-change</code> , <code>:diffget</code> , <code>:diffg</code>	Reset the diff change at the cursor position.
<code>:clear-register</code>	Clear given register. If no argument is provided, clear all registers.
<code>:set-register</code>	Set contents of the given register.
<code>:redraw</code>	Clear and re-render the whole UI
<code>:move</code> , <code>:mv</code>	Move the current buffer and its corresponding file to a different path
<code>:move!</code> , <code>:mv!</code>	Move the current buffer and its corresponding file to a different path creating necessary subdirectories
<code>:delete-file</code> , <code>:remove-file</code>	Delete the current buffer's file from disk and close the buffer (vim-eunuch :Delete).
<code>:chmod-x</code> , <code>:chmodx</code> , <code>:make-executable</code>	Make the current file executable (chmod a+x). Unix only.
<code>:mkdir</code>	Create a directory and any missing parents; with no arg, the current file's parent.
<code>:yank-diagnostic</code>	Yank diagnostic(s) under primary cursor to register, or clipboard by default
<code>:read</code> , <code>:r</code>	Load a file into buffer
<code>:echo</code>	Prints the given arguments to the statusline.
<code>:echoerr</code> , <code>:echoe</code>	Prints the given arguments to the statusline as an error (vim :echoerr).
<code>:noop</code>	Does nothing.
<code>:workspace-trust</code>	Allow language servers and local config for the current workspace.
<code>:workspace-untrust</code>	Revoke the current workspace's trust grant or exclusion.
<code>:workspace-exclude</code>	Mark the current workspace as never-prompt. Never prompts for trust again.
<code>:stjump</code> , <code>:stj</code>	Jump to a tag in a new split, offering a picker when several tags match (vim :stjump).
<code>:ptjump</code> , <code>:ptj</code>	Like :tjump, showing the tag in the preview window — a split here (vim :ptjump).
<code>:stselect</code> , <code>:sts</code>	List matching tags in a picker; the chosen one opens in a new split (vim :stselect).
<code>:ptselect</code> , <code>:pts</code>	Like :tselect, showing the chosen tag in the preview window — a split here (vim :ptselect).
<code>:ppop</code> , <code>:pp</code>	Pop the tag stack into the preview window — a split here (vim :ppop).
<code>:ltag</code> , <code>:lt</code>	Jump to a tag and put every matching tag in the location list (vim :ltag).
<code>:psearch</code> , <code>:ps</code>	Show the first line matching an identifier in the preview window, keeping focus (vim :psearch).
<code>:perl-do</code> , <code>:perl-do</code>	Run a Perl snippet on every line (\$_ is the line, and replaces it) (vim :perl-do).
<code>:ruby-do</code> , <code>:ruby-do</code>	Run a Ruby snippet on every line (\$_ is the line, and replaces it) (vim :ruby-do).
<code>:luado</code> , <code>:luad</code>	Run a Lua chunk on every line (args line, linenr; the return value replaces the line) (vim :luado).
<code>:pydo</code> , <code>:pyd</code>	Run a Python function body on every line (args line, linenr; the return value replaces the line) (vim :pydo).
<code>:py3do</code> , <code>:py3d</code> , <code>:pyxdo</code> , <code>:pyxd</code>	Run a Python 3 function body on every line (args line, linenr) (vim :py3do / :pyxdo).
<code>:pyx</code> , <code>:pythonx</code>	Run a Python snippet through python3 and echo its output (vim :pyx / :pythonx).
<code>:pyxfile</code> , <code>:pyxfile</code>	Run a Python script file through python3 (vim :pyxfile).

Name	Description
<code>:command</code> , <code>:com</code>	Define a user command (<code>:command LI :lopen</code>), or list the ones defined (<code>vim :command</code>).
<code>:command!</code> , <code>:com!</code>	Define a user command, replacing an existing definition of that name (<code>vim :command!</code>).
<code>:delcommand</code> , <code>:delc</code>	Delete a user-defined command (<code>vim :delcommand</code>).
<code>:comclear</code> , <code>:comc</code>	Delete every user-defined command (<code>vim :comclear</code>).
<code>:folddopen</code> , <code>:foldd</code>	Run a command on every line that is not inside a closed fold (<code>vim :folddopen</code>).
<code>:folddoclosed</code> , <code>:folddoc</code>	Run a command on every line inside a closed fold (<code>vim :folddoclosed</code>).
<code>:breakadd</code> , <code>:breaka</code>	Set a debugger breakpoint: <code>:breakadd</code> here
<code>:breakdel</code> , <code>:breakd</code>	Delete a debugger breakpoint by <code>:breaklist</code> number, by position, or all with <code>*</code> (<code>vim :breakdel</code>).
<code>:breaklist</code> , <code>:breakl</code>	List the debugger breakpoints, numbered for <code>:breakdel</code> (<code>vim :breaklist</code>).
<code>:highlight</code> , <code>:hi</code>	List, show or set the theme's highlight groups (<code>:hi Comment guifg=#5c6370 gui=italic</code>) (<code>vim :highlight</code>).
<code>:syntax</code> , <code>:sy</code> , <code>:syn</code>	Turn the buffer's syntax highlighting on/off, or report the language (<code>vim :syntax</code>).
<code>:compiler</code> , <code>:comp</code>	Select the compiler for <code>:make</code> by setting <code>makeprg</code> (<code>:compiler cargo</code>) (<code>vim :compiler</code>).
<code>:checkhealth</code> , <code>:che</code>	Run the health checks (clipboard, language servers, grammars) and show the report (<code>nvim :checkhealth</code>).
<code>:helpclose</code> , <code>:helpc</code>	Close the help window (<code>vim :helpclose</code>).
<code>:options</code> , <code>:opt</code>	Open the options window — Preferences on the Settings tab (<code>vim :options</code>).
<code>:mode</code> , <code>:mod</code>	Redraw the screen (<code>vim :mode</code>).
<code>:startreplace</code> , <code>:starttr</code>	Start Replace mode (<code>vim :startreplace</code>).
<code>:tabfind</code> , <code>:tabf</code>	Find a file in the 'path' and edit it in a new tab page (<code>vim :tabfind</code>).
<code>:spellgood</code> , <code>:spe</code>	Add words to the known-good spell list (<code>vim :spellgood</code>).
<code>:~</code>	Repeat the last <code>:substitute</code> (<code>vim :~</code>).

Static Commands and Keybindings

Every static (bindable) command and the default keys bound to it, by mode.

Name	Description	Default keybinds
<code>no_op</code>	Do nothing	
<code>move_char_left</code>	Move left	normal: <code>h</code> , <code><left></code> , insert: <code><[-b></code> , <code><left></code>
<code>move_char_right</code>	Move right	normal: <code>l</code> , <code><right></code> , insert: <code><[-f></code> , <code><right></code>
<code>move_line_up</code>	Move up	normal: <code>gk</code> , <code>g<up></code>
<code>move_line_down</code>	Move down	normal: <code>gj</code> , <code>g<down></code>
<code>shift_line_up</code>	Move the line/selection up (JetBrains Move Statement Up)	normal: <code><A-up></code>
<code>shift_line_down</code>	Move the line/selection down (JetBrains Move Statement Down)	normal: <code><A-down></code>
<code>drag_line_down</code>	Drag the current line down (SPC x . j)	normal: <code><space>x.J</code> , <code><space>x.j</code> , <code><space>x.<down></code> , select: <code><space>x.J</code> , <code><space>x.j</code> , <code><space>x.<down></code>
<code>drag_line_up</code>	Drag the current line up (SPC x . k)	normal: <code><space>x.K</code> , <code><space>x.k</code> , <code><space>x.<up></code> , select: <code><space>x.K</code> , <code><space>x.k</code> , <code><space>x.<up></code>
<code>toggle_test_file</code>	Toggle between implementation and test file (SPC p a)	normal: <code><space>pa</code> , select: <code><space>pa</code>
<code>fold_comments</code>	Fold multi-line comment blocks (SPC c h)	normal: <code><space>ch</code> , select: <code><space>ch</code>
<code>move_visual_line_up</code>	Move up	normal: <code>k</code> , <code><up></code> , <code><[-p></code> , insert: <code><up></code> , <code><[-g>k</code> , <code><[-g><up></code> , <code><[-g><[-k></code>
<code>move_visual_line_down</code>	Move down	normal: <code>j</code> , <code><[-j></code> , <code><[-n></code> , <code><down></code> , insert: <code><[-g>j</code> , <code><down></code> , <code><[-g><[-j></code> , <code><[-g><down></code>
<code>extend_char_left</code>	Extend left	
<code>extend_char_right</code>	Extend right	
<code>extend_line_up</code>	Extend up	
<code>extend_line_down</code>	Extend down	
<code>extend_visual_line_up</code>	Extend up	
<code>extend_visual_line_down</code>	Extend down	
<code>copy_selection_on_next_line</code>	Copy selection on next line	
<code>copy_selection_on_prev_line</code>	Copy selection on previous line	
<code>column_selection</code>	Turn the selection into a rectangular column block (IntelliJ column selection)	
<code>visual_block_mode</code>	Enter/leave vim visual-block selection (CTRL-V)	normal: <code><[-v></code> , <code>g<[-h></code> , <code><[-x><space></code> , select: <code><[-v></code> , <code><[-x><space></code> , insert: <code><[-x><space></code>
<code>block_reproject</code>	Rebuild the visual-block rectangle from its anchor (internal motion helper)	

Name	Description	Default keybinds
<code>visual_line_mode</code>	Enter/leave vim visual-line selection (V)	normal: <code>V</code> , select: <code>V</code>
<code>line_reproject</code>	Rebuild the visual-line whole-line span from its anchor (internal motion helper)	
<code>block_dollar</code>	Visual-block: extend each row to its own line end (CTRL-V \$)	select: <code>\$</code> , <code><end></code>
<code>block_swap_corners</code>	Visual-block: move cursor to the opposite corner (o)	select: <code>o</code>
<code>block_swap_columns</code>	Visual-block: move cursor to the other column edge (O)	select: <code>O</code>
<code>block_insert</code>	Visual-block: insert at the left column on every row (I)	select: <code>I</code>
<code>block_append</code>	Visual-block: append at the right column, padding short rows (A)	select: <code>A</code>
<code>move_next_word_start</code>	Move to start of next word	normal: <code><A-f></code> , <code><C-right></code> , <code><S-right></code> , insert: <code><A-f></code> , <code><C-right></code> , <code><S-right></code>
<code>move_prev_word_start</code>	Move to start of previous word	normal: <code><A-b></code> , <code><C-left></code> , <code><S-left></code> , insert: <code><A-b></code> , <code><C-left></code> , <code><S-left></code>
<code>move_next_word_end</code>	Move to end of next word	
<code>move_prev_word_end</code>	Move to end of previous word	
<code>move_next_long_word_start</code>	Move to start of next long word	
<code>move_prev_long_word_start</code>	Move to start of previous long word	
<code>move_next_long_word_end</code>	Move to end of next long word	
<code>move_prev_long_word_end</code>	Move to end of previous long word	
<code>move_next_sub_word_start</code>	Move to start of next sub word	
<code>move_prev_sub_word_start</code>	Move to start of previous sub word	
<code>move_next_sub_word_end</code>	Move to end of next sub word	
<code>move_prev_sub_word_end</code>	Move to end of previous sub word	
<code>vim_move_next_word_start</code>	Move to start of next word (vim caret)	
<code>vim_move_prev_word_start</code>	Move to start of previous word (vim caret)	
<code>vim_move_next_word_end</code>	Move to end of next word (vim caret)	
<code>vim_move_prev_word_end</code>	Move to end of previous word (vim caret)	normal: <code>ge</code>
<code>vim_move_next_long_word_start</code>	Move to start of next long word (vim caret)	normal: <code>W</code>
<code>vim_move_prev_long_word_start</code>	Move to start of previous long word (vim caret)	normal: <code>B</code>
<code>vim_move_next_long_word_end</code>	Move to end of next long word (vim caret)	normal: <code>E</code>
<code>vim_move_prev_long_word_end</code>	Move to end of previous long word (vim caret)	normal: <code>gE</code>
<code>move_parent_node_end</code>	Move to end of the parent node	normal: <code><space>k\$</code> , select: <code><space>k\$</code>
<code>move_parent_node_start</code>	Move to beginning of the parent node	normal: <code><space>k0</code> , select: <code><space>k0</code>
<code>extend_next_word_start</code>	Extend to start of next word	
<code>extend_prev_word_start</code>	Extend to start of previous word	

Name	Description	Default keybinds
<code>extend_next_word_end</code>	Extend to end of next word	
<code>extend_prev_word_end</code>	Extend to end of previous word	
<code>extend_next_long_word_start</code>	Extend to start of next long word	
<code>extend_prev_long_word_start</code>	Extend to start of previous long word	
<code>extend_next_long_word_end</code>	Extend to end of next long word	
<code>extend_prev_long_word_end</code>	Extend to end of prev long word	
<code>extend_next_sub_word_start</code>	Extend to start of next sub word	
<code>extend_prev_sub_word_start</code>	Extend to start of previous sub word	
<code>extend_next_sub_word_end</code>	Extend to end of next sub word	
<code>extend_prev_sub_word_end</code>	Extend to end of prev sub word	
<code>extend_parent_node_end</code>	Extend to end of the parent node	
<code>extend_parent_node_start</code>	Extend to beginning of the parent node	
<code>find_till_char</code>	Move till next occurrence of char	
<code>find_next_char</code>	Move to next occurrence of char	
<code>extend_till_char</code>	Extend till next occurrence of char	
<code>extend_next_char</code>	Extend to next occurrence of char	
<code>till_prev_char</code>	Move till previous occurrence of char	
<code>find_prev_char</code>	Move to previous occurrence of char	
<code>sneak_forward</code>	Sneak: jump forward to a two-character sequence	
<code>sneak_backward</code>	Sneak: jump backward to a two-character sequence	
<code>sneak_or_substitute_char</code>	Sneak forward, or substitute char when vim-sneak is off	normal: <code>s</code>
<code>sneak_or_substitute_line</code>	Sneak backward, or substitute line when vim-sneak is off	normal: <code>S</code>
<code>extend_till_prev_char</code>	Extend till previous occurrence of char	
<code>extend_prev_char</code>	Extend to previous occurrence of char	
<code>repeat_last_motion</code>	Repeat last motion	
<code>repeat_find_char</code>	Repeat last find in same direction (:)	normal: <code>;</code> , select: <code>;</code>
<code>repeat_find_char_reverse</code>	Repeat last find in opposite direction (,)	normal: <code>,</code> , select: <code>,</code>
<code>replace</code>	Replace with new char	select: <code>r</code>
<code>switch_case</code>	Switch (toggle) case	select: <code>~</code>
<code>switch_case_forward</code>	Toggle case and advance cursor (vim ~)	normal: <code>~</code>
<code>switch_to_uppercase</code>	Switch to uppercase	normal: <code><space>xU</code> , <code><[-x]><[-u></code> , select: <code><space>xU</code> , <code><[-x]><[-u></code> , insert: <code><[-x]><[-u></code>
<code>switch_to_lowercase</code>	Switch to lowercase	normal: <code><space>xu</code> , <code><[-x]><[-l></code> , select: <code><space>xu</code> , <code><[-x]><[-l></code> , insert: <code><[-x]><[-l></code>

Name	Description	Default keybinds
<code>upcase_word</code>	Upper-case the word after point (emacs upcase-word, M-u)	
<code>downcase_word</code>	Lower-case the word after point (emacs downcase-word, M-l)	
<code>capitalize_word</code>	Capitalize the word after point (emacs capitalize-word, M-c)	
<code>capitalize_region</code>	Title-case every word in the region (emacs capitalize-region)	
<code>upcase_initials_region</code>	Upper-case the first letter of each word in the region (emacs upcase-initials-region)	
<code>page_up</code>	Move page up	normal: <A-v>, <[-b>, <[-x>[, <S-up>, <pageup>, <S-minus>, select: <[-x>[, insert: <A-v>, <[-x>[, <S-up>, <pageup>
<code>page_down</code>	Move page down	normal: <[-f>, <S-+>, <[-x>], <S-ret>, <S-down>, <pagedown>, select: <[-x>], insert: <[-x>], <S-down>, <pagedown>
<code>half_page_up</code>	Move half page up	
<code>half_page_down</code>	Move half page down	
<code>page_cursor_up</code>	Move page and cursor up	
<code>page_cursor_down</code>	Move page and cursor down	
<code>page_cursor_half_up</code>	Move page and cursor half up	normal: <[-u>
<code>page_cursor_half_down</code>	Move page and cursor half down	normal: <[-d>
<code>select_all</code>	Select whole document	normal: <[-x>h, <[-x><[-p>, select: <[-x>h, <[-x><[-p>, insert: <[-x>h, <[-x><[-p>
<code>select_regex</code>	Select all regex matches inside selections	normal: <[-x>wh, <[-x>wi, <[-x>wl, <[-x>wp, select: <[-x>wh, <[-x>wi, <[-x>wl, <[-x>wp, insert: <[-x>wh, <[-x>wi, <[-x>wl, <[-x>wp
<code>select_all_instances</code>	Select all occurrences of the current selection in the buffer	
<code>split_selection</code>	Split selections on regex matches	
<code>split_selection_on_newline</code>	Split selection on newlines	
<code>merge_selections</code>	Merge selections	
<code>merge_consecutive_selections</code>	Merge consecutive selections	
<code>search</code>	Search for regex pattern	normal: /, <[-s>, select: /
<code>rsearch</code>	Reverse search for regex pattern	normal: ?, select: ?
<code>search_next</code>	Select next search match	normal: <space>sH, select: <space>sH
<code>search_prev</code>	Select previous search match	
<code>extend_search_next</code>	Add next search match to selection	
<code>extend_search_prev</code>	Add previous search match to selection	
<code>select_gn_match</code>	vim gn: select the search match at/after the cursor (for cgn/dgn)	
<code>select_gn_match_prev</code>	vim gN: select the search match at/before the cursor	

Name	Description	Default keybinds
<code>search_next_vim</code>	vim n: repeat last search in its direction	normal: <code>n</code>
<code>search_prev_vim</code>	vim N: repeat last search in the opposite direction	normal: <code>N</code>
<code>extend_search_next_vim</code>	vim n (visual): extend to the repeated match	select: <code>n</code>
<code>extend_search_prev_vim</code>	vim N (visual): extend to the reverse match	select: <code>N</code>
<code>add_selection_to_next_match</code>	Add the next occurrence of the selection as a new cursor	
<code>select_all_occurrences</code>	Select every occurrence of the selection as a cursor (JetBrains Select All Occurrences)	normal: <code><space>xo</code> , select: <code><space>xo</code>
<code>search_selection</code>	Use current selection as search pattern	
<code>search_selection_detect_word_boundaries</code>	Use current selection as the search pattern, automatically wrapping with <code>\b</code> on word boundaries	
<code>make_search_word_bounded</code>	Modify current search to make it word bounded	
<code>global_search</code>	Global search in workspace folder	normal: <code><space>/</code> , <code><space>po</code> , <code><space>ps</code> , <code><space>sP</code> , <code><space>sb</code> , <code><space>sd</code> , <code><space>sf</code> , <code><space>sp</code> , <code><space>ss</code> , <code><space>saa</code> , <code><space>sab</code> , <code><space>sad</code> , <code><space>saf</code> , <code><space>sap</code> , <code><space>sgb</code> , <code><space>sgd</code> , <code><space>sgf</code> , <code><space>sgg</code> , <code><space>sgp</code> , <code><space>skb</code> , <code><space>skd</code> , <code><space>skf</code> , <code><space>skp</code> , <code><space>srb</code> , <code><space>urd</code> , <code><space>urf</code> , <code><space>urp</code> , <code><space>urr</code> , select: <code><space>/</code> , <code><space>po</code> , <code><space>ps</code> , <code><space>sP</code> , <code><space>sb</code> , <code><space>sd</code> , <code><space>sf</code> , <code><space>sp</code> , <code><space>ss</code> , <code><space>saa</code> , <code><space>sab</code> , <code><space>sad</code> , <code><space>saf</code> , <code><space>sap</code> , <code><space>sgb</code> , <code><space>sgd</code> , <code><space>sgf</code> , <code><space>sgg</code> , <code><space>sgp</code> , <code><space>skb</code> , <code><space>skd</code> , <code><space>skf</code> , <code><space>skp</code> , <code><space>srb</code> , <code><space>urd</code> , <code><space>urf</code> , <code><space>urp</code> , <code><space>urr</code>

Name	Description	Default keybinds
<code>global_search_symbol</code>	Global search seeded with the symbol under the cursor	normal: <code><space>*</code> , <code><space>sB</code> , <code><space>sD</code> , <code><space>sF</code> , <code><space>saA</code> , <code><space>saB</code> , <code><space>saD</code> , <code><space>saF</code> , <code><space>saP</code> , <code><space>sgB</code> , <code><space>sgF</code> , <code><space>sgG</code> , <code><space>skB</code> , <code><space>skD</code> , <code><space>skF</code> , <code><space>skP</code> , <code><space>srB</code> , <code><space>srD</code> , <code><space>srF</code> , <code><space>srP</code> , <code><space>srR</code> , select: <code><space>*</code> , <code><space>sB</code> , <code><space>sD</code> , <code><space>sF</code> , <code><space>saA</code> , <code><space>saB</code> , <code><space>saD</code> , <code><space>saF</code> , <code><space>saP</code> , <code><space>sgB</code> , <code><space>sgF</code> , <code><space>sgG</code> , <code><space>skB</code> , <code><space>skD</code> , <code><space>skF</code> , <code><space>skP</code> , <code><space>srB</code> , <code><space>srD</code> , <code><space>srF</code> , <code><space>srP</code> , <code><space>srR</code>
<code>clear_search_highlight</code>	Clear persistent search highlight (SPC s c)	normal: <code><C-x>wr</code> , <code><space>sc</code> , select: <code><C-x>wr</code> , <code><space>sc</code> , insert: <code><C-x>wr</code>
<code>regex_convert_form</code>	Convert the selected regex between PCRE and Emacs forms (SPC x r c)	normal: <code><space>xrc</code> , <code><space>xrep</code> , <code><space>xrpe</code> , select: <code><space>xrc</code> , <code><space>xrep</code> , <code><space>xrpe</code>
<code>regex_emacs_to_rx_replace</code>	Convert the selected Emacs regex to rx form (SPC x r e x)	normal: <code><space>xret</code> , <code><space>xrex</code> , select: <code><space>xret</code> , <code><space>xrex</code>
<code>regex_emacs_to_rx_explain</code>	Explain the selected Emacs regex as rx (SPC x r e /)	normal: <code><space>xre/</code> , select: <code><space>xre/</code>
<code>regex_pcre_to_rx_replace</code>	Convert the selected PCRE regex to rx form (SPC x r x)	normal: <code><space>xrt</code> , <code><space>xrx</code> , <code><space>xrpx</code> , select: <code><space>xrt</code> , <code><space>xrx</code> , <code><space>xrpx</code>
<code>regex_pcre_to_rx_explain</code>	Explain the selected PCRE regex as rx (SPC x r /)	normal: <code><space>xr/</code> , <code><space>xrp/</code> , select: <code><space>xr/</code> , <code><space>xrp/</code>
<code>justify_left</code>	Left-justify (fill) the region (SPC x j l)	normal: <code><space>xjl</code> , select: <code><space>xjl</code>
<code>justify_right</code>	Right-justify the region (SPC x j r)	normal: <code><space>xjr</code> , select: <code><space>xjr</code>
<code>justify_center</code>	Center-justify the region (SPC x j c)	normal: <code><space>xjc</code> , select: <code><space>xjc</code>
<code>justify_full</code>	Full-justify the region (SPC x j f)	normal: <code><space>xjf</code> , select: <code><space>xjf</code>
<code>justify_none</code>	Remove justification / left-fill (SPC x j n)	normal: <code><space>xjn</code> , select: <code><space>xjn</code>
<code>count_words_region</code>	Count occurrences per word in the selection (SPC x w c)	normal: <code><space>xwc</code> , select: <code><space>xwc</code>
<code>goto_next_close_paren</code>	Go forward to next closing paren (SPC k j)	normal: <code><space>kj</code> , select: <code><space>kj</code>
<code>goto_prev_open_paren</code>	Go backward to previous opening paren (SPC k k)	normal: <code><space>kk</code> , select: <code><space>kk</code>
<code>ediff_windows</code>	Diff the two front windows side by side (SPC D w w)	normal: <code><space>DwL</code> , <code><space>Dww</code> , select: <code><space>DwL</code> , <code><space>Dww</code>
<code>ediff_buffer</code>	Diff the current buffer against a picked buffer (SPC D b b)	normal: <code><space>Dbb</code> , select: <code><space>Dbb</code>
<code>compare_with_clipboard</code>	Diff the current buffer against the clipboard (JetBrains Compare with Clipboard)	normal: <code><space>Dc</code> , select: <code><space>Dc</code>
<code>transpose_paragraph</code>	Swap the current paragraph with the previous one (SPC x t p)	normal: <code><space>xtp</code> , select: <code><space>xtp</code>

Name	Description	Default keybinds
<code>transpose_line</code>	Swap the current line with the previous one (emacs transpose-lines, C-x C-t)	normal: <code><[-x><[-t></code> , select: <code><[-x><[-t></code> , insert: <code><[-x><[-t></code>
<code>move_element_right</code>	Swap the syntax node under the cursor with its next sibling (JetBrains Move Element Right)	normal: <code><space>x<gt></code> , select: <code><space>x<gt></code>
<code>move_element_left</code>	Swap the syntax node under the cursor with its previous sibling (JetBrains Move Element Left)	normal: <code><space>x<lt></code> , select: <code><space>x<lt></code>
<code>convert_indent_to_spaces</code>	Convert leading indentation to spaces (JetBrains Convert Indents to Spaces)	
<code>convert_indent_to_tabs</code>	Convert leading indentation to tabs (JetBrains Convert Indents to Tabs)	
<code>transpose_sexp</code>	Swap the current s-expression with the previous one (SPC x t e)	normal: <code><space>xte</code> , select: <code><space>xte</code>
<code>transpose_sentence</code>	Swap the current sentence with the previous one (SPC x t s)	normal: <code><space>xts</code> , select: <code><space>xts</code>
<code>make_3_windows</code>	Lay out three vertical windows (SPC w 3)	normal: <code><[-w>3</code> , <code><space>w3</code> , <code><space>u<space>w3</code> , select: <code><space>w3</code> , <code><space>u<space>w3</code>
<code>make_4_windows</code>	Lay out a 2x2 window grid (SPC w 4)	normal: <code><[-w>4</code> , <code><space>w4</code> , <code><space>u<space>w4</code> , select: <code><space>w4</code> , <code><space>u<space>w4</code>
<code>narrow_to_function</code>	Narrow the buffer to the enclosing function (SPC n f)	normal: <code><[-x>nd</code> , <code><space>nf</code> , select: <code><[-x>nd</code> , <code><space>nf</code> , insert: <code><[-x>nd</code>
<code>align_at_equals</code>	Align region at = (SPC x a =)	normal: <code><space>xa=</code> , select: <code><space>xa=</code>
<code>align_at_comma</code>	Align region at , (SPC x a ,)	normal: <code><space>xa,</code> , select: <code><space>xa,</code>
<code>align_at_colon</code>	Align region at : (SPC x a :)	normal: <code><space>xa:</code> , select: <code><space>xa:</code>
<code>align_at_semicolon</code>	Align region at ; (SPC x a ;)	normal: <code><space>xa;</code> , select: <code><space>xa;</code>
<code>align_at_ampersand</code>	Align region at & (SPC x a &)	normal: <code><space>xaf</code> , select: <code><space>xaf</code>
<code>align_at_lparen</code>	Align region at ((SPC x a (	normal: <code><space>xa(</code> , select: <code><space>xa(</code>
<code>align_at_rparen</code>	Align region at) (SPC x a)	normal: <code><space>xa)</code> , select: <code><space>xa)</code>
<code>align_at_lbracket</code>	Align region at [(SPC x a [	normal: <code><space>xa[</code> , select: <code><space>xa[</code>
<code>align_at_rbracket</code>	Align region at] (SPC x a])	normal: <code><space>xa]</code> , select: <code><space>xa]</code>
<code>align_at_lbrace</code>	Align region at { (SPC x a {	normal: <code><space>xa{</code> , select: <code><space>xa{</code>
<code>align_at_rbrace</code>	Align region at } (SPC x a }	normal: <code><space>xa}</code> , select: <code><space>xa}</code>
<code>align_at_dot</code>	Align region at . (SPC x a .)	normal: <code><space>xa.</code> , select: <code><space>xa.</code>
<code>align_at_arithmetic</code>	Align region at arithmetic operators (SPC x a m)	normal: <code><space>xam</code> , select: <code><space>xam</code>
<code>align_at_regex</code>	Align region at a user-specified regexp (SPC x a r)	normal: <code><space>xar</code> , select: <code><space>xar</code>
<code>align_current</code>	Auto-align the region into columns, per blank-line section (emacs align-current)	
<code>align_entire</code>	Auto-align the whole region into columns as one section (emacs align-entire)	
<code>align_left_at_char</code>	Left-align region at a typed delimiter (SPC x a l)	normal: <code><space>xal</code> , select: <code><space>xal</code>

Name	Description	Default keybinds
<code>align_right_at_char</code>	Right-align region at a typed delimiter (SPC x a L)	normal: <code><space>xaL</code> , select: <code><space>xaL</code>
<code>buffer_to_window_1</code>	Move current buffer to window 1 (SPC b . 1)	normal: <code><space>b.1</code> , select: <code><space>b.1</code>
<code>buffer_to_window_2</code>	Move current buffer to window 2 (SPC b . 2)	normal: <code><space>b.2</code> , select: <code><space>b.2</code>
<code>buffer_to_window_3</code>	Move current buffer to window 3 (SPC b . 3)	normal: <code><space>b.3</code> , select: <code><space>b.3</code>
<code>buffer_to_window_4</code>	Move current buffer to window 4 (SPC b . 4)	normal: <code><space>b.4</code> , select: <code><space>b.4</code>
<code>buffer_to_window_5</code>	Move current buffer to window 5 (SPC b . 5)	normal: <code><space>b.5</code> , select: <code><space>b.5</code>
<code>buffer_to_window_6</code>	Move current buffer to window 6 (SPC b . 6)	normal: <code><space>b.6</code> , select: <code><space>b.6</code>
<code>buffer_to_window_7</code>	Move current buffer to window 7 (SPC b . 7)	normal: <code><space>b.7</code> , select: <code><space>b.7</code>
<code>buffer_to_window_8</code>	Move current buffer to window 8 (SPC b . 8)	normal: <code><space>b.8</code> , select: <code><space>b.8</code>
<code>buffer_to_window_9</code>	Move current buffer to window 9 (SPC b . 9)	normal: <code><space>b.9</code> , select: <code><space>b.9</code>
<code>goto_window_1</code>	Go to window 1 (SPC 1)	normal: <code><C-w>.1, <C-w>[1, <C-w>{1, <space>1, <space>w.1, <space>w[1, <space>w{1, <space>b.<C-1>, select: <space>1, <space>w.1, <space>w[1, <space>w{1, <space>b.<C-1></code>
<code>goto_window_2</code>	Go to window 2 (SPC 2)	normal: <code><C-w>.2, <C-w>[2, <C-w>{2, <space>2, <space>w.2, <space>w[2, <space>w{2, <space>b.<C-2>, select: <space>2, <space>w.2, <space>w[2, <space>w{2, <space>b.<C-2></code>
<code>goto_window_3</code>	Go to window 3 (SPC 3)	normal: <code><C-w>.3, <C-w>[3, <C-w>{3, <space>3, <space>w.3, <space>w[3, <space>w{3, <space>b.<C-3>, select: <space>3, <space>w.3, <space>w[3, <space>w{3, <space>b.<C-3></code>
<code>goto_window_4</code>	Go to window 4 (SPC 4)	normal: <code><C-w>.4, <C-w>[4, <C-w>{4, <space>4, <space>w.4, <space>w[4, <space>w{4, <space>b.<C-4>, select: <space>4, <space>w.4, <space>w[4, <space>w{4, <space>b.<C-4></code>
<code>goto_window_5</code>	Go to window 5 (SPC 5)	normal: <code><C-w>.5, <C-w>[5, <C-w>{5, <space>5, <space>w.5, <space>w[5, <space>w{5, <space>b.<C-5>, select: <space>5, <space>w.5, <space>w[5, <space>w{5, <space>b.<C-5></code>
<code>goto_window_6</code>	Go to window 6 (SPC 6)	normal: <code><C-w>.6, <C-w>[6, <C-w>{6, <space>6, <space>w.6, <space>w[6, <space>w{6, <space>b.<C-6>, select: <space>6, <space>w.6, <space>w[6, <space>w{6, <space>b.<C-6></code>

Name	Description	Default keybinds
<code>goto_window_7</code>	Go to window 7 (SPC 7)	normal: <code><C-w>.7, <C-w>[7, <C-w>{7, <space>7, <space>w.7, <space>w[7, <space>w{7, <space>b.<C-7>, select: <code><space>7, <space>w.7, <space>w[7, <space>w{7, <space>b.<C-7></code></code>
<code>goto_window_8</code>	Go to window 8 (SPC 8)	normal: <code><C-w>.8, <C-w>[8, <C-w>{8, <space>8, <space>w.8, <space>w[8, <space>w{8, <space>b.<C-8>, select: <code><space>8, <space>w.8, <space>w[8, <space>w{8, <space>b.<C-8></code></code>
<code>goto_window_9</code>	Go to window 9 (SPC 9)	normal: <code><C-w>.9, <C-w>[9, <C-w>{9, <space>9, <space>w.9, <space>w[9, <space>w{9, <space>b.<C-9>, select: <code><space>9, <space>w.9, <space>w[9, <space>w{9, <space>b.<C-9></code></code>
<code>delete_window_and_buffer</code>	Close window and kill its buffer (SPC w . x)	normal: <code><C-w>.x, <C-w>[x, <C-w>{x, <space>w.x, <space>w[x, <space>w{x, <space>u<space>bD, <space>u<space>wD, select: <code><space>w.x, <space>w[x, <space>w{x, <space>u<space>bD, <space>u<space>wD</code></code>
<code>eval_elisp_region</code>	Evaluate the selection as elisp (SPC m e r)	normal: <code><space>mer, select: <code><space>mer</code></code>
<code>eval_elisp_buffer</code>	Evaluate the buffer as elisp (SPC m e b)	normal: <code><space>meb, select: <code><space>meb</code></code>
<code>eval_elisp_line</code>	Evaluate the current line as elisp (SPC m e e)	normal: <code><C-x><C-e>, <space>me\$, <space>mee, <space>meL, select: <code><C-x><C-e>, <space>me\$, <space>mee, <space>meL, insert: <code><C-x><C-e></code></code></code>
<code>eval_elisp_defun</code>	Evaluate the enclosing form as elisp (SPC m e f)	normal: <code><space>mec, <space>mef, select: <code><space>mec, <space>mef</code></code>
<code>eval_print_last_sexp</code>	Evaluate the sexp before point and insert its value (emacs eval-print-last-sexp)	
<code>compare_windows</code>	Compare this window with the next, moving both points to the first difference (emacs compare-windows)	
<code>layout_create</code>	Create a new window-layout from the current windows (SPC l l)	normal: <code><C-x>rf, <C-x>rw, <space>Fn, <space>ll, <space>lww, select: <code><C-x>rf, <C-x>rw, <space>Fn, <space>ll, <space>lww, insert: <code><C-x>rf, <C-x>rw</code></code></code>
<code>layout_next</code>	Switch to the next layout (SPC l n)	normal: <code><space>ln, <space>lwL, <space>lwn, <space>l<C-l>, select: <code><space>ln, <space>lwL, <space>lwn, <space>l<C-l></code></code>
<code>layout_prev</code>	Switch to the previous layout (SPC l p)	normal: <code><space>lN, <space>lp, <space>lwN, <space>lwh, <space>lwp, <space>l<C-h>, select: <code><space>lN, <space>lp, <space>lwN, <space>lwh, <space>lwp, <space>l<C-h></code></code>
<code>layout_last</code>	Switch to the last-used layout (SPC l TAB)	normal: <code><space>l<tab>, <space>lw<tab>, select: <code><space>l<tab>, <space>lw<tab></code></code>

Name	Description	Default keybinds
<code>layout_default</code>	Switch to the default (first) layout (SPC I h)	normal: <code><space>lh</code> , select: <code><space>lh</code>
<code>layout_delete</code>	Delete the current layout, keeping its buffers (SPC I d)	normal: <code><space>ld</code> , <code><space>lx</code> , <code><space>ld</code> , <code><space>lx</code> , <code><space>lwd</code> , select: <code><space>ld</code> , <code><space>lx</code> , <code><space>ld</code> , <code><space>lx</code> , <code><space>lwd</code>
<code>layout_save</code>	Save layouts to disk (SPC I s)	normal: <code><space>l5</code> , select: <code><space>l5</code>
<code>layout_rename</code>	Rename the current layout (SPC I R)	normal: <code><space>lR</code> , <code><space>lwR</code> , select: <code><space>lR</code> , <code><space>lwR</code>
<code>layout_load</code>	Load layouts from disk (SPC I L)	normal: <code><space>ll</code> , <code><space>lo</code> , select: <code><space>ll</code> , <code><space>lo</code>
<code>layout_goto_1</code>	Switch to layout 1 (SPC I 1)	normal: <code><space>l1</code> , <code><space>lw1</code> , <code><space>l<C-1></code> , <code><space>lw<C-1></code> , select: <code><space>l1</code> , <code><space>lw1</code> , <code><space>l<C-1></code> , <code><space>lw<C-1></code>
<code>layout_goto_2</code>	Switch to layout 2 (SPC I 2)	normal: <code><space>l2</code> , <code><space>lw2</code> , <code><space>l<C-2></code> , <code><space>lw<C-2></code> , select: <code><space>l2</code> , <code><space>lw2</code> , <code><space>l<C-2></code> , <code><space>lw<C-2></code>
<code>layout_goto_3</code>	Switch to layout 3 (SPC I 3)	normal: <code><space>l3</code> , <code><space>lw3</code> , <code><space>l<C-3></code> , <code><space>lw<C-3></code> , select: <code><space>l3</code> , <code><space>lw3</code> , <code><space>l<C-3></code> , <code><space>lw<C-3></code>
<code>layout_goto_4</code>	Switch to layout 4 (SPC I 4)	normal: <code><space>l4</code> , <code><space>lw4</code> , <code><space>l<C-4></code> , <code><space>lw<C-4></code> , select: <code><space>l4</code> , <code><space>lw4</code> , <code><space>l<C-4></code> , <code><space>lw<C-4></code>
<code>layout_goto_5</code>	Switch to layout 5 (SPC I 5)	normal: <code><space>l5</code> , <code><space>lw5</code> , <code><space>l<C-5></code> , <code><space>lw<C-5></code> , select: <code><space>l5</code> , <code><space>lw5</code> , <code><space>l<C-5></code> , <code><space>lw<C-5></code>
<code>layout_goto_6</code>	Switch to layout 6 (SPC I 6)	normal: <code><space>l6</code> , <code><space>lw6</code> , <code><space>l<C-6></code> , <code><space>lw<C-6></code> , select: <code><space>l6</code> , <code><space>lw6</code> , <code><space>l<C-6></code> , <code><space>lw<C-6></code>
<code>layout_goto_7</code>	Switch to layout 7 (SPC I 7)	normal: <code><space>l7</code> , <code><space>lw7</code> , <code><space>l<C-7></code> , <code><space>lw<C-7></code> , select: <code><space>l7</code> , <code><space>lw7</code> , <code><space>l<C-7></code> , <code><space>lw<C-7></code>
<code>layout_goto_8</code>	Switch to layout 8 (SPC I 8)	normal: <code><space>l8</code> , <code><space>lw8</code> , <code><space>l<C-8></code> , <code><space>lw<C-8></code> , select: <code><space>l8</code> , <code><space>lw8</code> , <code><space>l<C-8></code> , <code><space>lw<C-8></code>
<code>layout_goto_9</code>	Switch to layout 9 (SPC I 9)	normal: <code><space>l9</code> , <code><space>lw9</code> , <code><space>l<C-9></code> , <code><space>lw<C-9></code> , select: <code><space>l9</code> , <code><space>lw9</code> , <code><space>l<C-9></code> , <code><space>lw<C-9></code>
<code>toggle_modeline_position</code>	Toggle cursor position in the mode line (SPC t m p)	normal: <code><space>tmp</code> , select: <code><space>tmp</code>
<code>toggle_modeline_vcs</code>	Toggle version-control info in the mode line (SPC t m v)	normal: <code><space>tmv</code> , select: <code><space>tmv</code>
<code>toggle_centered_cursor</code>	Keep the cursor vertically centered (SPC t -)	normal: <code><space>t<minus></code> , select: <code><space>t<minus></code>
<code>toggle_hl_line</code>	Highlight the current line (emacs hl-line-mode / global-hl-line-mode)	
<code>toggle_electric_pair</code>	Auto-insert matching close delimiters (emacs electric-pair-mode)	
<code>toggle_auto_revert</code>	Reload buffers when their file changes on disk (emacs auto-revert-mode)	

Name	Description	Default keybinds
<code>set_fill_prefix</code>	Set the fill-prefix from line start to point (emacs set-fill-prefix)	
<code>set_goal_column</code>	Make vertical motion stick to the current column (emacs set-goal-column)	
<code>toggle_fill_column</code>	Toggle a fill-column ruler (SPC t f)	normal: <code><[>x>f</code> , <code><space>tf</code> , select: <code><[>-x>f</code> , <code><space>tf</code> , insert: <code><[>-x>f</code>
<code>toggle_long_line_marker</code>	Toggle an 80th-column ruler (SPC t 8)	normal: <code><space>t8</code> , <code><space>t<[>-8></code> , select: <code><space>t8</code> , <code><space>t<[>-8></code>
<code>toggle_soft_wrap</code>	Toggle soft-wrap of long lines (IntelliJ View > Soft-Wrap)	normal: <code><[>-x>xt</code> , select: <code><[>-x>xt</code> , insert: <code><[>-x>xt</code>
<code>toggle_whitespace_render</code>	Toggle rendering of whitespace characters (IntelliJ View > Show Whitespaces)	
<code>toggle_line_numbers</code>	Toggle the line-numbers gutter (IntelliJ View > Show Line Numbers)	
<code>toggle_indent_guides</code>	Toggle indentation guides (IntelliJ View > Show Indent Guides)	
<code>toggle_inlay_hints</code>	Toggle display of LSP inlay hints (IntelliJ View > Inlay Hints)	
<code>toggle_auto_highlight</code>	Toggle automatic symbol-under-cursor highlight (SPC t h a)	normal: <code><[>-x>w.</code> , <code><[>-x>wb</code> , <code><space>tha</code> , select: <code><[>-x>w.</code> , <code><[>-x>wb</code> , <code><space>tha</code> , insert: <code><[>-x>w.</code> , <code><[>-x>wb</code>
<code>toggle_syntax_highlighting</code>	Toggle syntax highlighting for the current buffer (SPC t h s)	normal: <code><space>ths</code> , select: <code><space>ths</code>
<code>toggle_diagnostics</code>	Toggle diagnostics display / flycheck (SPC t s)	normal: <code><space>ts</code> , select: <code><space>ts</code>
<code>ediff_file</code>	Diff a prompted file against the current buffer (SPC D f f)	normal: <code><space>Dff</code> , select: <code><space>Dff</code>
<code>ediff_3_files</code>	3-way diff of three prompted files, read-only (SPC D f 3)	normal: <code><space>Df3</code> , select: <code><space>Df3</code>
<code>ediff_regions</code>	Ediff two regions linewise: mark A, then diff B (SPC D r l)	normal: <code><space>Dr l</code> , select: <code><space>Dr l</code>
<code>ediff_merge_file</code>	Merge a picked file into the current buffer (editable, SPC D m f f)	normal: <code><space>Dmff</code> , select: <code><space>Dmff</code>
<code>ediff_3_buffers</code>	3-way diff of three open buffers, read-only (SPC D b 3)	normal: <code><space>Db3</code> , select: <code><space>Db3</code>
<code>kill_buffers_by_regex</code>	Kill all buffers whose name matches a regex (SPC b M)	normal: <code><space>b<[>-D></code> , select: <code><space>b<[>-D></code>
<code>narrow_to_page</code>	Narrow the buffer to the current page (SPC n p)	normal: <code><[>-x>np</code> , <code><space>np</code> , select: <code><[>-x>np</code> , <code><space>np</code> , insert: <code><[>-x>np</code>
<code>copy_file</code>	Copy the current file to a prompted destination (SPC f c)	normal: <code><space>fc</code> , select: <code><space>fc</code>
<code>find_file_replace_buffer</code>	Open a file and replace the current buffer with it (SPC f A)	normal: <code><space>fA</code> , select: <code><space>fA</code>
<code>open_file_literally</code>	Open a file with no syntax/language (fundamental mode, SPC f l)	normal: <code><space>fl</code> , select: <code><space>fl</code>
<code>locate_file</code>	Locate a file via system locate/mdfind and open it (SPC f L)	normal: <code><space>fL</code> , select: <code><space>fL</code>
<code>edit_project_config</code>	Edit the project-local .zemacs/config.toml (SPC p e)	normal: <code><space>pe</code> , select: <code><space>pe</code>
<code>man_page_search</code>	Search man pages via apropos and view the selected page (SPC h m)	normal: <code><[>-h>S</code> , <code><space>hm</code> , select: <code><[>-h>S</code> , <code><space>hm</code> , insert: <code><[>-h>S</code>
<code>info_search</code>	Search GNU info manuals (apropos) and view the selected node (SPC h i)	normal: <code><[>-h>i</code> , <code><[>-h>4i</code> , <code><space>hi</code> , select: <code><[>-h>i</code> , <code><[>-h>4i</code> , <code><space>hi</code> , insert: <code><[>-h>i</code> , <code><[>-h>4i</code>
<code>diagnostics_verify_setup</code>	Report the buffer's diagnostics/LSP setup (SPC e v)	normal: <code><space>ev</code> , select: <code><space>ev</code>
<code>clear_diagnostics</code>	Clear all diagnostics for the current buffer (SPC e c)	normal: <code><space>ec</code> , select: <code><space>ec</code>

Name	Description	Default keybinds
<code>ai_chat</code>	Ask the AI provider about the selection/buffer (SPC a i)	normal: <code><space>ai</code> , select: <code><space>ai</code>
<code>ai_chat_panel</code>	Open the streaming AI chat drawer (SPC a p)	normal: <code><space>ap</code> , select: <code><space>ap</code>
<code>ai_model_picker</code>	Pick the AI model at runtime (SPC a m)	
<code>toggle_ai_privacy</code>	Toggle AI privacy mode (SPC a P)	normal: <code><space>aP</code> , select: <code><space>aP</code>
<code>ai_apply_block</code>	Apply the last AI code block to the selection (SPC a y)	normal: <code><space>ay</code> , select: <code><space>ay</code>
<code>ai_add_file_context</code>	Add a file as @context for the next AI chat (SPC a @)	normal: <code><space>a@</code> , select: <code><space>a@</code>
<code>ai_codebase_context</code>	Add codebase-search results as @context (SPC a b)	normal: <code><space>ab</code> , select: <code><space>ab</code>
<code>ai_symbol_context</code>	Add the symbol-under-cursor's definitions as @context (SPC a s)	normal: <code><space>as</code> , select: <code><space>as</code>
<code>ai_terminal_command</code>	Generate a shell command from natural language (SPC a k)	normal: <code><space>ak</code> , select: <code><space>ak</code>
<code>ai_inline_edit</code>	AI inline edit/generate on the selection (SPC a e)	normal: <code><space>ae</code> , select: <code><space>ae</code>
<code>ai_inline_edit_preview</code>	AI inline edit with a diff preview (SPC a E)	normal: <code><space>aE</code> , select: <code><space>aE</code>
<code>ai_accept_edit</code>	Accept the pending AI inline-edit preview (SPC a .)	normal: <code><space>a.</code> , select: <code><space>a.</code>
<code>ai_explain</code>	Explain the selected code with AI (SPC a x)	normal: <code><space>ax</code> , select: <code><space>ax</code>
<code>ai_generate_tests</code>	Generate tests for the selection with AI (SPC a u)	normal: <code><space>au</code> , select: <code><space>au</code>
<code>ai_commit_message</code>	Generate a git commit message with AI (SPC a c)	
<code>ai_agent</code>	Run the autonomous AI agent on a task (SPC a a)	normal: <code><space>aa</code> , select: <code><space>aa</code>
<code>ai_agent_review</code>	Toggle agent review (dry-run) mode — propose changes without applying (SPC a R)	normal: <code><space>aR</code> , select: <code><space>aR</code>
<code>ai_complete</code>	AI code completion at the cursor (SPC a TAB)	normal: <code><space>a<tab></code> , select: <code><space>a<tab></code>
<code>ai_docs_context</code>	@docs keyword-search over the docs directory for AI context (SPC a D)	normal: <code><space>aD</code> , select: <code><space>aD</code>
<code>ai_web_context</code>	@web live web-search results as AI context (SPC a w)	normal: <code><space>aw</code> , select: <code><space>aw</code>
<code>toggle_ai_autocomplete</code>	Toggle real-time AI ghost-text autocomplete (SPC a g)	normal: <code><space>ag</code> , select: <code><space>ag</code>
<code>ghost_text_accept</code>	Accept the AI ghost-text suggestion, else Tab (insert mode)	insert: <code><tab></code>
<code>ghost_text_accept_word</code>	Accept the next word of the AI ghost-text suggestion (partial accept)	insert: <code><A-right></code>
<code>ai_revert_agent</code>	Revert the workspace to the last agent checkpoint (SPC a z)	normal: <code><space>az</code> , select: <code><space>az</code>
<code>ai_fix</code>	AI-fix the diagnostic(s) on the current line (SPC a F)	normal: <code><space>aF</code> , select: <code><space>aF</code>
<code>describe_diagnostics_checker</code>	Describe the buffer's checkers/language servers (SPC e h)	normal: <code><space>eh</code> , select: <code><space>eh</code>
<code>describe_text_properties</code>	Describe the tree-sitter node stack at the cursor (SPC h d t)	normal: <code><space>hdt</code> , select: <code><space>hdt</code>
<code>copy_system_info</code>	Copy system info (version/OS/arch) to the clipboard (SPC h d s)	normal: <code><space>hds</code> , select: <code><space>hds</code>
<code>copy_last_keys</code>	Copy the most recently pressed keys to the clipboard (SPC h d l)	normal: <code><space>hdl</code> , select: <code><space>hdl</code>

Name	Description	Default keybinds
<code>ace_window</code>	Jump to a window by its number, prompted (ace-window, SPC w . a)	normal: <code><[-w>.a</code> , <code><[-w>[a</code> , <code><[-w>{a</code> , <code><space>w.a</code> , <code><space>w[a</code> , <code><space>w{a</code> , select: <code><space>w.a</code> , <code><space>w[a</code> , <code><space>w{a</code>
<code>browse_news</code>	Browse zemacs release notes / NEWS (SPC h n)	normal: <code><[-h>n</code> , <code><space>hn</code> , <code><[-h><[-n></code> , select: <code><[-h>n</code> , <code><space>hn</code> , <code><[-h><[-n></code> , insert: <code><[-h>n</code> , <code><[-h><[-n></code>
<code>browse_faq</code>	Browse the zemacs FAQ (SPC h f)	normal: <code><space>hf</code> , <code><[-h><[-p></code> , select: <code><space>hf</code> , <code><[-h><[-p></code> , insert: <code><[-h><[-p></code>
<code>layer_search</code>	Search zemacs capability areas / layers (SPC h l)	normal: <code><space>hl</code> , select: <code><space>hl</code>
<code>show_environment</code>	Show the editor's environment variables (SPC f e e)	normal: <code><space>fee</code> , select: <code><space>fee</code>
<code>reimport_shell_env</code>	Re-import the shell environment into the editor (SPC f e C-e)	normal: <code><space>feE</code> , <code><space>fe<[-e></code> , select: <code><space>feE</code> , <code><space>fe<[-e></code>
<code>goto_buffer_window</code>	Focus the window already showing a chosen buffer (SPC b w)	normal: <code><space>bW</code> , select: <code><space>bW</code>
<code>git_file_dispatch</code>	Magit-style file operations dispatch for the current file (SPC g f m)	normal: <code><[-x>vG</code> , <code><[-x>vi</code> , <code><[-x>vu</code> , <code><space>gfm</code> , select: <code><[-x>vG</code> , <code><[-x>vi</code> , <code><[-x>vu</code> , <code><space>gfm</code> , insert: <code><[-x>vG</code> , <code><[-x>vi</code> , <code><[-x>vu</code>
<code>describe_current_modes</code>	Describe the current editor/buffer modes (SPC h d m)	normal: <code><[-h>m</code> , <code><space>hdm</code> , select: <code><[-h>m</code> , <code><space>hdm</code> , insert: <code><[-h>m</code>
<code>describe_command</code>	Describe a command — its doc and key bindings (C-h f)	normal: <code><[-h>a</code> , <code><[-h>d</code> , <code><[-h>f</code> , <code><[-h>o</code> , <code><[-h>x</code> , select: <code><[-h>a</code> , <code><[-h>d</code> , <code><[-h>f</code> , <code><[-h>o</code> , <code><[-h>x</code> , insert: <code><[-h>a</code> , <code><[-h>d</code> , <code><[-h>f</code> , <code><[-h>o</code> , <code><[-h>x</code>
<code>where_is</code>	Show the keys a command is bound to (C-h w)	normal: <code><[-h>w</code> , select: <code><[-h>w</code> , insert: <code><[-h>w</code>
<code>describe_key</code>	Describe a key — pick a binding, show its command and doc (C-h k)	normal: <code><[-h>c</code> , <code><[-h>k</code> , select: <code><[-h>c</code> , <code><[-h>k</code> , insert: <code><[-h>c</code> , <code><[-h>k</code>
<code>describe_bindings</code>	List every key binding of the current mode (C-h b)	normal: <code><[-h>b</code> , select: <code><[-h>b</code> , insert: <code><[-h>b</code>
<code>describe_coding_system</code>	Describe the buffer's coding system / encoding (C-h C)	normal: <code><[-h>C</code> , select: <code><[-h>C</code> , insert: <code><[-h>C</code>
<code>describe_language_environment</code>	Describe the language environment / locale (C-h L)	normal: <code><[-h>L</code> , select: <code><[-h>L</code> , insert: <code><[-h>L</code>
<code>describe_syntax</code>	Describe the buffer's syntax / tree-sitter status (C-h s)	normal: <code><[-h>s</code> , select: <code><[-h>s</code> , insert: <code><[-h>s</code>
<code>view_lossage</code>	Show the recently pressed keys (C-h l)	normal: <code><[-h>l</code> , select: <code><[-h>l</code> , insert: <code><[-h>l</code>
<code>describe_char</code>	Describe the character after point — code, Unicode block, category (emacs describe-char, C-u C-x =)	
<code>view_hello_file</code>	Show a multi-script greeting sample (emacs view-hello-file, C-h h)	normal: <code><[-h>h</code> , select: <code><[-h>h</code> , insert: <code><[-h>h</code>
<code>view_echo_area_messages</code>	Show the last echo-area message (emacs view-echo-area-messages, C-h e)	normal: <code>g<[t></code> , <code><[-h>e</code> , select: <code><[-h>e</code> , insert: <code><[-h>e</code>
<code>describe_copying</code>	Show zemacs's copying license, the GPL (emacs describe-copying, C-h C-c)	normal: <code><[-h><[-c></code> , select: <code><[-h><[-c></code> , insert: <code><[-h><[-c></code>
<code>describe_distribution</code>	How to get zemacs / GNU software (emacs describe-distribution, C-h C-d)	normal: <code><[-h><[-d></code> , select: <code><[-h><[-d></code> , insert: <code><[-h><[-d></code>
<code>describe_gnu_project</code>	Open the GNU project page (emacs describe-gnu-project, C-h g)	normal: <code><[-h>g</code> , select: <code><[-h>g</code> , insert: <code><[-h>g</code>

Name	Description	Default keybinds
<code>describe_no_warranty</code>	Show the GPL no-warranty sections (emacs describe-no-warranty, C-h C-w)	normal: <code><[-h><[-w></code> , select: <code><[-h><[-w></code> , insert: <code><[-h><[-w></code>
<code>view_emacs_faq</code>	Open the GNU Emacs FAQ (emacs view-emacs-faq, C-h C-f)	normal: <code><[-h><[-f></code> , select: <code><[-h><[-f></code>
<code>view_emacs_todo</code>	Open the Emacs TODO list (emacs view-emacs-todo)	
<code>view_emacs_problems</code>	Open the Emacs known-problems file (emacs view-emacs-problems)	
<code>view_emacs_debugging</code>	Open the Emacs debugging manual (emacs view-emacs-debugging)	
<code>view_order_manuals</code>	Open where to get the GNU manuals (emacs view-order-manuals)	
<code>view_external_packages</code>	Open GNU ELPA / external packages (emacs view-external-packages)	
<code>describe_keymap</code>	List every binding of the current mode's keymap (emacs describe-keymap)	
<code>describe_prefix_bindings</code>	List the sub-bindings of a prefix (emacs describe-prefix-bindings)	
<code>describe_categories</code>	List the character categories zemacs recognises (emacs describe-categories)	
<code>list_character_sets</code>	List the Unicode blocks zemacs knows (emacs list-character-sets)	
<code>list_charset_chars</code>	List the printable characters of each Unicode block (emacs list-charset-chars)	
<code>list_coding_systems</code>	List the coding systems / encodings zemacs supports (emacs list-coding-systems)	
<code>describe_language_package</code>	Describe the language-support config for the buffer (SPC h d p)	normal: <code><space>hdp</code> , select: <code><space>hdp</code>
<code>package_search</code>	Search configured language packages and describe one (SPC h p)	normal: <code><[-h>P</code> , <code><[-h>p</code> , <code><space>hp</code> , <code><[-h><[-e></code> , select: <code><[-h>P</code> , <code><[-h>p</code> , <code><space>hp</code> , <code><[-h><[-e></code> , insert: <code><[-h>P</code> , <code><[-h>p</code> , <code><[-h><[-e></code>
<code>config_variable_search</code>	Search editor config variables, copy path on select (SPC h .)	normal: <code><[-h>v</code> , <code><space>h.</code> , select: <code><[-h>v</code> , <code><space>h.</code> , insert: <code><[-h>v</code>
<code>clone_indirect_buffer</code>	Clone the current buffer into a shared-document split (SPC b N i)	normal: <code><space>bNI</code> , <code><space>bNi</code> , select: <code><space>bNI</code> , <code><space>bNi</code>
<code>clone_indirect_from_buffer</code>	Open an existing buffer in a shared-document split (SPC b N C-i)	normal: <code><space>bN<[-i></code> , select: <code><space>bN<[-i></code>
<code>open_junk_file</code>	Open a fresh timestamped junk file (SPC f J)	normal: <code><space>fJ</code> , select: <code><space>fJ</code>
<code>open_hex</code>	Open the current file in the hex editor (SPC f h, hexl)	normal: <code><space>fh</code> , select: <code><space>fh</code>
<code>open_file_external</code>	Open the current file with the OS default program (SPC f o)	normal: <code><space>fo</code> , select: <code><space>fo</code>
<code>git_init</code>	Initialize a new git repository (SPC g i)	normal: <code><space>gi</code> , select: <code><space>gi</code>
<code>view_file_at_rev</code>	View the current file at a branch/commit (SPC g f f)	normal: <code><[-x>v-</code> , <code><space>gff</code> , select: <code><[-x>v-</code> , <code><space>gff</code> , insert: <code><[-x>v-</code>
<code>extend_line</code>	Select current line, if already selected, extend to another line based on the anchor	normal: <code><space>kV</code> , select: <code><space>kV</code>
<code>extend_line_below</code>	Select current line, if already selected, extend to next line	
<code>extend_line_above</code>	Select current line, if already selected, extend to previous line	

Name	Description	Default keybinds
<code>select_line_above</code>	Select current line, if already selected, extend or shrink line above based on the anchor	
<code>select_line_below</code>	Select current line, if already selected, extend or shrink line below based on the anchor	
<code>extend_to_line_bounds</code>	Extend selection to line bounds	
<code>extend_chars_right_vim</code>	Extend count graphemes right, line-bounded (dl/cl/yl)	
<code>extend_chars_left_vim</code>	Extend count graphemes left, line-bounded (dh/ch/yh)	
<code>extend_line_below_linewise</code>	Extend whole lines down for a linewise operator (dj/cj/yj)	
<code>extend_line_above_linewise</code>	Extend whole lines up for a linewise operator (dk/ck/yk)	
<code>extend_next_paragraph</code>	Extend to next paragraph for an operator (d/c/y)	
<code>extend_prev_paragraph</code>	Extend to previous paragraph for an operator (d{/c{/y{)	
<code>select_paragraph_forward_vim</code>	vim }: paragraph operator target with linewise promotion	
<code>select_paragraph_backward_vim</code>	vim {: paragraph operator target with linewise promotion	
<code>select_paragraph_forward_vim_linewise</code>	vim V}: force-linewise paragraph operator target	
<code>select_paragraph_backward_vim_linewise</code>	vim V{: force-linewise paragraph operator target	
<code>shrink_to_line_bounds</code>	Shrink selection to line bounds	
<code>delete_selection</code>	Delete selection	
<code>delete_selection_linewise</code>	Delete selection (vim linewise, EOF-aware)	
<code>delete_selection_noyank</code>	Delete selection without yanking	
<code>change_selection</code>	Change selection	
<code>change_selection_noyank</code>	Change selection without yanking	
<code>collapse_selection</code>	Collapse selection into single cursor	
<code>flip_selections</code>	Flip selection cursor and anchor	normal: <[-x><[-x>, select: <[-x><[-x>, insert: <[-x><[-x>
<code>ensure_selections_forward</code>	Ensure all selections face forward	
<code>insert_mode</code>	Insert before selection	normal: i, <ins>, <space>ki, select: <space>ki
<code>append_mode</code>	Append after selection	normal: a
<code>replace_mode</code>	Enter Replace mode (overtyping)	normal: R, gR
<code>command_mode</code>	Enter command mode	normal: gQ, <space>:, <space>k:, select: :, <space>:, <space>k:

Name	Description	Default keybinds
<code>file_picker</code>	Open file picker	normal: <code><space>ff</code> , <code><space>pf</code> , <code><space>ph</code> , <code><space>pp</code> , <code><[-x]<[-f></code> , <code><[-x]<[-r></code> , <code><[-x]<[-v></code> , select: <code><space>ff</code> , <code><space>pf</code> , <code><space>ph</code> , <code><space>pp</code> , <code><[-x]<[-f></code> , <code><[-x]<[-r></code> , <code><[-x]<[-v></code> , insert: <code><[-x]<[-f></code> , <code><[-x]<[-r></code> , <code><[-x]<[-v></code>
<code>file_picker_in_current_buffer_directory</code>	Open file picker at current buffer's directory	normal: <code><[-x]<[-j></code> , select: <code><[-x]<[-j></code> , insert: <code><[-x]<[-j></code>
<code>file_picker_in_current_directory</code>	Open file picker at current working directory	normal: <code><[-x>d</code> , <code><[-x]<[-d></code> , select: <code><[-x>d</code> , <code><[-x]<[-d></code> , insert: <code><[-x>d</code> , <code><[-x]<[-d></code>
<code>file_explorer</code>	Open file explorer in workspace root	normal: <code><space>ad</code> , <code><space>af</code> , <code><space>ft</code> , <code><space>pd</code> , <code><space>pt</code> , <code><space>atrd</code> , <code><space>attr</code> , select: <code><space>ad</code> , <code><space>af</code> , <code><space>ft</code> , <code><space>pd</code> , <code><space>pt</code> , <code><space>atrd</code> , <code><space>attr</code>
<code>file_explorer_in_current_buffer_directory</code>	Open file explorer at current buffer's directory	normal: <code><space>fd</code> , <code><space>fj</code> , <code><space>jd</code> , <code><space>jd</code> , select: <code><space>fd</code> , <code><space>fj</code> , <code><space>jd</code> , <code><space>jd</code>
<code>file_explorer_in_current_directory</code>	Open file explorer at current working directory	
<code>buffer_menu</code>	Open the Buffer Menu (emacs buffer-menu / C-x C-b)	normal: <code><space>bM</code> , select: <code><space>bM</code>
<code>list_buffers</code>	List open buffers in the Buffer Menu (emacs list-buffers)	
<code>calendar</code>	Open the Calendar month grid (emacs calendar)	
<code>diary</code>	Show today's diary entries (emacs diary)	
<code>diary_view_entries</code>	Show diary entries for the current date (emacs diary-view-entries)	
<code>diary_show_all_entries</code>	Open the diary file (emacs diary-show-all-entries)	
<code>diary_insert_entry</code>	Add a diary entry for today (emacs diary-insert-entry)	
<code>diary_insert_weekly_entry</code>	Add a weekly diary entry for today (emacs diary-insert-weekly-entry)	
<code>diary_mark_entries</code>	Mark calendar dates that have diary entries (emacs diary-mark-entries)	
<code>diary_list_entries</code>	List diary entries for the current date (emacs diary-list-entries)	
<code>diary_fancy_display</code>	Show the day's diary entries in fancy format (emacs diary-fancy-display)	
<code>diary_simple_display</code>	Show the day's diary entries in simple format (emacs diary-simple-display)	
<code>diary_sort_entries</code>	Sort the day's diary entries by time (emacs diary-sort-entries)	
<code>diary_include_other_diary_files</code>	Include entries from #include'd diary files (emacs diary-include-other-diary-files)	
<code>diary_mark_included_diary_files</code>	Mark dates from #include'd diary files (emacs diary-mark-included-diary-files)	

Name	Description	Default keybinds
<code>icalendar_export_file</code>	Export the diary file to ~/diary.ics (emacs icalendar-export-file)	
<code>icalendar_export_region</code>	Export the selected diary region to ~/diary.ics (emacs icalendar-export-region)	
<code>icalendar_import_file</code>	Import ~/diary.ics into the diary file (emacs icalendar-import-file)	
<code>icalendar_import_buffer</code>	Import the current buffer's iCalendar into the diary (emacs icalendar-import-buffer)	
<code>diary_print_entries</code>	Print the day's diary entries (emacs diary-print-entries)	
<code>diary_day_of_year</code>	Report today's day-of-year and days remaining (emacs diary-day-of-year)	
<code>diary_hebrew_date</code>	Today's Hebrew calendar date (emacs diary-hebrew-date)	
<code>diary_islamic_date</code>	Today's Islamic calendar date (emacs diary-islamic-date)	
<code>diary_french_date</code>	Today's French Revolutionary date (emacs diary-french-date)	
<code>diary_bahai_date</code>	Today's Baha'i calendar date (emacs diary-bahai-date)	
<code>diary_coptic_date</code>	Today's Coptic calendar date (emacs diary-coptic-date)	
<code>diary_ethiopic_date</code>	Today's Ethiopic calendar date (emacs diary-ethiopic-date)	
<code>diary_astro_day_number</code>	Today's astronomical (Julian) day number (emacs diary-astro-day-number)	
<code>diary_hebrew_omer</code>	Report today's Omer count, if any (emacs diary-hebrew-omer)	
<code>diary_hebrew_rosh_hodesh</code>	Report if today is Rosh Hodesh (emacs diary-hebrew-rosh-hodesh)	
<code>diary_hebrew_birthday</code>	Today's Hebrew date for a birthday entry (emacs diary-hebrew-birthday)	
<code>diary_hebrew_yahrzeit</code>	Report any yahrzeit falling today (emacs diary-hebrew-yahrzeit)	
<code>diary_insert_monthly_entry</code>	Add a monthly diary entry for today (emacs diary-insert-monthly-entry)	
<code>diary_insert_yearly_entry</code>	Add a yearly diary entry for today (emacs diary-insert-yearly-entry)	
<code>diary_insert_anniversary_entry</code>	Add a diary-anniversary entry for today (emacs diary-insert-anniversary-entry)	
<code>diary_insert_block_entry</code>	Add a diary-block entry for today (emacs diary-insert-block-entry)	
<code>diary_insert_cyclic_entry</code>	Add a diary-cyclic entry for today (emacs diary-insert-cyclic-entry)	
<code>diary_hebrew_insert_entry</code>	Add a Hebrew-date diary entry for today (emacs diary-hebrew-insert-entry)	
<code>diary_hebrew_insert_monthly_entry</code>	Add a monthly Hebrew diary entry (emacs diary-hebrew-insert-monthly-entry)	
<code>diary_hebrew_insert_yearly_entry</code>	Add a yearly Hebrew diary entry (emacs diary-hebrew-insert-yearly-entry)	
<code>diary_hebrew_insert_anniversary_entry</code>	Add a Hebrew anniversary diary entry (emacs diary-hebrew-insert-anniversary-entry)	
<code>diary_islamic_insert_entry</code>	Add an Islamic-date diary entry for today (emacs diary-islamic-insert-entry)	

Name	Description	Default keybinds
<code>diary_islamic_insert_monthly_entry</code>	Add a monthly Islamic diary entry (emacs diary-islamic-insert-monthly-entry)	
<code>diary_islamic_insert_yearly_entry</code>	Add a yearly Islamic diary entry (emacs diary-islamic-insert-yearly-entry)	
<code>diary_islamic_insert_anniversary_entry</code>	Add an Islamic anniversary diary entry (emacs diary-islamic-insert-anniversary-entry)	
<code>diary_bahai_insert_entry</code>	Add a Baha'i-date diary entry for today (emacs diary-bahai-insert-entry)	
<code>diary_bahai_insert_monthly_entry</code>	Add a monthly Baha'i diary entry (emacs diary-bahai-insert-monthly-entry)	
<code>diary_bahai_insert_yearly_entry</code>	Add a yearly Baha'i diary entry (emacs diary-bahai-insert-yearly-entry)	
<code>diary_bahai_insert_anniversary_entry</code>	Add a Baha'i anniversary diary entry (emacs diary-bahai-insert-anniversary-entry)	
<code>appt_add</code>	Add an appointment reminder (emacs appt-add)	
<code>appt_delete</code>	Delete appointment reminders (emacs appt-delete)	
<code>appt_activate</code>	Toggle appointment checking (emacs appt-activate)	
<code>calendar_print_other_dates</code>	Report today's date in all other calendars (emacs calendar-print-other-dates)	
<code>calendar_julian_print_date</code>	Today's Julian (Roman) calendar date (emacs calendar-julian-print-date)	
<code>calendar_iso_print_date</code>	Today's ISO 8601 week date (emacs calendar-iso-print-date)	
<code>calendar_hebrew_print_date</code>	Today's Hebrew calendar date (emacs calendar-hebrew-print-date)	
<code>calendar_islamic_print_date</code>	Today's Islamic calendar date (emacs calendar-islamic-print-date)	
<code>calendar_persian_print_date</code>	Today's Persian calendar date (emacs calendar-persian-print-date)	
<code>calendar_coptic_print_date</code>	Today's Coptic calendar date (emacs calendar-coptic-print-date)	
<code>calendar_ethiopic_print_date</code>	Today's Ethiopic calendar date (emacs calendar-ethiopic-print-date)	
<code>calendar_french_print_date</code>	Today's French Revolutionary date (emacs calendar-french-print-date)	
<code>calendar_bahai_print_date</code>	Today's Baha'i date, approx (emacs calendar-bahai-print-date)	
<code>calendar_astro_print_day_number</code>	Astronomical (Julian) day number (emacs calendar-astro-print-day-number)	
<code>calendar_mayan_print_date</code>	Today's Mayan date (emacs calendar-mayan-print-date)	
<code>calendar_day_of_year</code>	Day-of-year of today (emacs calendar-day-of-year)	
<code>calendar_goto_day_of_year</code>	Echo the date for a day-of-year (emacs calendar-goto-day-of-year)	
<code>calendar_count_days_region</code>	Count days between two dates (emacs calendar-count-days-region)	
<code>calendar_list_holidays</code>	List this year's holidays (emacs calendar-list-holidays)	

Name	Description	Default keybinds
<code>holidays</code>	List this year's holidays (emacs holidays)	
<code>holiday_list</code>	List this year's holidays (emacs holiday-list)	
<code>list_holidays</code>	List this year's holidays (emacs list-holidays)	
<code>calendar_lunar_phases</code>	This month's moon phases, approx (emacs calendar-lunar-phases)	
<code>calendar_sunrise_sunset</code>	Sunrise/sunset today, approx (emacs calendar-sunrise-sunset)	
<code>calendar_other_month</code>	Open the Calendar at another month (emacs calendar-other-month)	
<code>calendar_set_date_style</code>	Cycle american/european/iso date style (emacs calendar-set-date-style)	
<code>calendar_hebrew_goto_date</code>	Echo Gregorian for a Hebrew date (emacs calendar-hebrew-goto-date)	
<code>calendar_islamic_goto_date</code>	Echo Gregorian for an Islamic date (emacs calendar-islamic-goto-date)	
<code>calendar_julian_goto_date</code>	Echo Gregorian for a Julian date (emacs calendar-julian-goto-date)	
<code>calendar_iso_goto_week</code>	Echo Gregorian for an ISO week date (emacs calendar-iso-goto-week)	
<code>calendar_persian_goto_date</code>	Echo Gregorian for a Persian date (emacs calendar-persian-goto-date)	
<code>calendar_coptic_goto_date</code>	Echo Gregorian for a Coptic date (emacs calendar-coptic-goto-date)	
<code>calendar_ethiopic_goto_date</code>	Echo Gregorian for an Ethiopic date (emacs calendar-ethiopic-goto-date)	
<code>calendar_french_goto_date</code>	Echo Gregorian for a French Revolutionary date (emacs calendar-french-goto-date)	
<code>calendar_bahai_goto_date</code>	Echo Gregorian for a Baha'i date (emacs calendar-bahai-goto-date)	
<code>calendar_astro_goto_day_number</code>	Echo Gregorian for an astro day number (emacs calendar-astro-goto-day-number)	
<code>calendar_mayan_goto_long_count</code>	Echo Gregorian for a Mayan long count (emacs calendar-mayan-goto-long-count)	
<code>calc_dispatch</code>	Open the RPN Calc stack calculator (emacs calc / C-x *)	normal: <code><space>ac</code> , select: <code><space>ac</code>
<code>occur</code>	List lines matching a regexp in an <i>Occur</i> overlay (emacs occur / M-s o)	
<code>isearch_forward_word</code>	Incremental whole-word search forward (emacs isearch-forward-word)	
<code>isearch_forward_symbol</code>	Incremental whole-symbol search forward (emacs isearch-forward-symbol)	
<code>isearch_forward_thing_at_point</code>	Search for the symbol/word at point (emacs isearch-forward-thing-at-point)	
<code>isearch_forward_symbol_at_point</code>	Search for the symbol at point (emacs isearch-forward-symbol-at-point)	
<code>isearch_toggle_regexp</code>	Toggle regexp matching for the current search (emacs isearch-toggle-regexp)	
<code>isearch_toggle_word</code>	Toggle whole-word matching for the current search (emacs isearch-toggle-word)	

Name	Description	Default keybinds
<code>isearch_toggle_symbol</code>	Toggle whole-symbol matching for the current search (emacs <code>isearch-toggle-symbol</code>)	
<code>isearch_toggle_case_fold</code>	Toggle case-folding for the current search (emacs <code>isearch-toggle-case-fold</code>)	
<code>isearch_toggle_lax_whitespace</code>	Toggle lax-whitespace matching for the current search (emacs <code>isearch-toggle-lax-whitespace</code>)	
<code>isearch_toggle_char_fold</code>	Toggle character folding (inert: no fold table) (emacs <code>isearch-toggle-char-fold</code>)	
<code>isearch_toggle_invisible</code>	Toggle invisible-text matching (inert) (emacs <code>isearch-toggle-invisible</code>)	
<code>isearch_toggle_input_method</code>	Input-method toggle (unsupported in <code>zemacs</code>) (emacs <code>isearch-toggle-input-method</code>)	
<code>isearch_yank_char</code>	Extend the search with the next buffer char (emacs <code>isearch-yank-char</code>)	
<code>isearch_yank_word_or_char</code>	Extend the search with the next word or char (emacs <code>isearch-yank-word-or-char</code>)	
<code>isearch_yank_symbol_or_char</code>	Extend the search with the next symbol or char (emacs <code>isearch-yank-symbol-or-char</code>)	
<code>isearch_yank_word</code>	Extend the search with the next word (emacs <code>isearch-yank-word</code>)	
<code>isearch_yank_symbol</code>	Extend the search with the next symbol (emacs <code>isearch-yank-symbol</code>)	
<code>isearch_yank_line</code>	Extend the search to end of line (emacs <code>isearch-yank-line</code>)	
<code>isearch_yank_until_char</code>	Extend the search up to a given char (emacs <code>isearch-yank-until-char</code>)	
<code>isearch_yank_kill</code>	Extend the search with the kill-ring top (emacs <code>isearch-yank-kill</code>)	
<code>isearch_yank_pop</code>	Extend the search with a kill-ring entry (emacs <code>isearch-yank-pop</code>)	
<code>isearch_yank_x_selection</code>	Extend the search with the clipboard selection (emacs <code>isearch-yank-x-selection</code>)	
<code>isearch_del_char</code>	Shorten the search string by one char (emacs <code>isearch-del-char</code>)	
<code>isearch_delete_char</code>	Shorten the search string by one char (emacs <code>isearch-delete-char</code>)	
<code>isearch_edit_string</code>	Edit the search string in a prompt (emacs <code>isearch-edit-string</code>)	
<code>isearch_ring_advance</code>	Cycle to an older search-ring entry (emacs <code>isearch-ring-advance</code>)	
<code>isearch_ring_retreat</code>	Cycle to a newer search-ring entry (emacs <code>isearch-ring-retreat</code>)	
<code>isearch_exit</code>	End the current incremental search (emacs <code>isearch-exit</code>)	
<code>isearch_abort</code>	Abort the search, return to origin (emacs <code>isearch-abort</code>)	
<code>isearch_cancel</code>	Cancel the search, return to origin (emacs <code>isearch-cancel</code>)	
<code>isearch_quote_char</code>	Add a literal char to the search string (emacs <code>isearch-quote-char</code>)	
<code>isearch_complete</code>	Complete the search string from history (emacs <code>isearch-complete</code>)	

Name	Description	Default keybinds
<code>isearch_char_by_name</code>	Add a char by digraph mnemonic to the search (emacs isearch-char-by-name)	
<code>isearch_emoji_by_name</code>	Add a char by digraph mnemonic to the search (emacs isearch-emoji-by-name)	
<code>isearch_occur</code>	Run occur with the current search pattern (emacs isearch-occur)	
<code>isearch_query_replace</code>	Query-replace the current search pattern (emacs isearch-query-replace)	
<code>isearch_query_replace_regexp</code>	Query-replace the current search regexp (emacs isearch-query-replace-regexp)	
<code>isearch_highlight_regexp</code>	Highlight matches of the current search (emacs isearch-highlight-regexp)	
<code>isearch_highlight_lines_matching_regexp</code>	List/highlight lines matching the search (emacs isearch-highlight-lines-matching-regexp)	
<code>rmail</code>	Open the Rmail mail reader on ~/RMAIL (emacs rmail)	normal: <code><space>ar</code> , select: <code><space>ar</code>
<code>dired</code>	Open the Dired directory editor (emacs C-x d)	
<code>dired_jump</code>	Open Dired on the current buffer's directory (emacs C-x C-j)	
<code>dired_other_window</code>	Open Dired (overlay; emacs dired-other-window C-x 4 d)	
<code>dired_jump_other_window</code>	Open Dired on the buffer's dir (overlay; emacs dired-jump-other-window)	
<code>dired_at_point</code>	Open Dired on the file name at point (emacs dired-at-point)	
<code>tex_insert_braces</code>	TeX: insert a {} brace pair (emacs tex-insert-braces)	

Name	Description	Default keybinds
tex_insert_quote	TeX: insert or ' ' smart quotes (emacs tex-insert-quote) `tex_terminate_paragraph` TeX: end the paragraph (emacs tex-terminate-paragraph) `latex_insert_block` LaTeX: insert a \begin{...}\end{...} block (emacs latex-insert-block) `latex_close_block` LaTeX: close the innermost open environment (emacs latex-close-block) `tex_validate` TeX: check {}/\$/begin-end balance (emacs tex-validate-region) `tex_mode` TeX: enter TeX editing mode (emacs tex-mode) `latex_mode` LaTeX: enter LaTeX editing mode (emacs latex-mode) `latex_electric_env_pair_mode` LaTeX: toggle electric \begin/\end pairing (emacs latex-electric-env-pair-mode) `tex_file` TeX: run LaTeX on the current file (emacs tex-file) `tex_buffer` TeX: compile the current buffer (emacs tex-buffer) `tex_region` TeX: compile the current file (emacs tex-region) `tex_compile` TeX: run LaTeX on the current file (emacs tex-compile) `tex_bibtex_file` TeX: run BibTeX on the current file (emacs tex-bibtex-file) `tex_view` TeX: open the compiled PDF (emacs tex-view) `tex_print` TeX: print the compiled PDF via lpr (emacs tex-print) `tex_kill_job` TeX: kill the running TeX job (emacs tex-kill-job) `tex_recenter_output_buffer` TeX: recenter the TeX output (emacs tex-recenter-output-buffer) `sgml_tag` SGML: wrap region/point in a <tag>...</tag> (emacs sgml-tag) `sgml_close_tag` SGML: close the innermost open element (emacs sgml-close-tag) `sgml_delete_tag` SGML: delete the enclosing tag pair, keeping content (emacs sgml-delete-tag) `sgml_skip_tag_forward` SGML: move past a balanced tag group (emacs sgml-skip-tag-forward) `sgml_skip_tag_backward` SGML: move past a balanced tag group (emacs sgml-skip-tag-backward)	normal: [`

Name	Description	Default keybinds
<code>goto_next_mark</code>	Goto next lowercase mark (<code>()</code>) normal: <code>` `] `goto_prev_mark_line` Goto previous lowercase mark, line start (<code>[]</code>) normal: <code>[`goto_next_mark_line` Goto next lowercase mark, line start (<code>[]</code>) normal: <code>] `yank_file_path` Yank current file path to clipboard normal: <code>fyC</code>, <code>fyY</code>, <code>fyy</code>, select: <code>fyC</code>, <code>fyY</code>, <code>fyy</code> <code>`yank_file_name`</code> Yank current file name to clipboard normal: <code>fyN</code>, <code>fyn</code>, select: <code>fyN</code>, <code>fyn</code> <code>`yank_file_path_with_line`</code> Yank current file path:line to clipboard normal: <code>fyL</code>, <code>fyl</code>, select: <code>fyL</code>, <code>fyl</code> <code>`yank_file_path_with_line_col`</code> Yank current file path:line:col to clipboard normal: <code>fyC</code>, select: <code>fyC</code> <code>`yank_file_dir`</code> Yank current file's directory to clipboard normal: <code>fyD</code>, <code>fyd</code>, select: <code>fyD</code>, <code>fyd</code> <code>`copy_remote_url`</code> Copy web permalink (host/blob/<sha>/path#Ln) for current line normal: <code>glC</code>, <code>gll</code>, <code>glP</code>, select: <code>glC</code>, <code>gll</code>, <code>glP</code> <code>`open_remote_url`</code> Open current line's web permalink in the browser normal: <code>glc</code>, <code>gll</code>, <code>glp</code>, select: <code>glc</code>, <code>gll</code>, <code>glp</code> <code>`open_url_under_cursor`</code> Open the URL under the cursor in the browser <code>`goto_address_mode`</code> Report that URLs are always openable at point (emacs goto-address-mode) <code>`duplicate_selection_down`</code> Duplicate current line(s) downward <code>`duplicate_selection_up`</code> Duplicate current line(s) upward <code>`move_text_line_down`</code> Move current line(s) down past the next line <code>`move_text_line_up`</code> Move current line(s) up past the previous line <code>`count_selection`</code> Count chars/words/lines in selection normal: <code>xc</code>, select: <code>xc</code> <code>`match_brackets`</code> Goto matching bracket normal: <code>k%</code>, select: <code>k%</code> <code>`match_brackets_or_goto_percent`</code> Goto matching bracket, or {count} percent through the file normal: <code>%</code>, select: <code>%</code> <code>`surround_add`</code> Surround add ⁹⁶ <code>`surround_replace`</code> Surround replace <code>`surround_delete`</code> Surround delete </code></code></code>	normal: <code>`</code>

Name	Description	Default keybinds
<code>goto_mark_line</code>	Goto mark line ('{a-z/A-Z/0-9},' for last jump)	normal: <code>'</code>

Name	Description	Default keybinds
goto_mark_nojump	Goto mark exact without changing jumplist (g) normal: ``g `goto_mark_line_nojump` Goto mark line without changing jumplist (g') normal: g' `repeat_substitute` Repeat last :substitute (&) normal: & `repeat_substitute_global` Repeat last :substitute on whole file (g&) normal: g& `vim_record_macro` Record macro into register (q{reg}) normal: q `vim_replay_macro` Replay macro from register (@{reg}) normal: @ `save_visual_selection` Save the visual selection (for gv) `reselect_visual` Reselect the last visual area (gv) normal: gv , select: gv `mark_insert_exit` Record the insert-exit position (for gi) `insert_at_last_insert` Insert at the last insert position (gi) normal: gi `goto_next_function` Goto next function normal:][,]m `goto_prev_function` Goto previous function normal: [] , [m `goto_next_class` Goto next type definition `goto_prev_class` Goto previous type definition `goto_next_parameter` Goto next parameter `goto_prev_parameter` Goto previous parameter `goto_next_comment` Goto next comment normal:]* ,]/ `goto_prev_comment` Goto previous comment normal: [* , [/ `goto_next_test` Goto next test `goto_prev_test` Goto previous test `goto_next_xml_element` Goto next (X)HTML element `goto_prev_xml_element` Goto previous (X)HTML element `goto_next_entry` Goto next pairing `goto_prev_entry` Goto previous pairing `goto_next_paragraph` Goto next paragraph normal: } ,]] , select: } `goto_prev_paragraph` Goto previous paragraph normal: { , [[, select: { `move_sentence_forward` Move to next sentence normal:) , select:) `move_sentence_backward` Move to previous sentence normal: (, select: (`dap_launch` Launch debug target	Spelling suggestions for the word being typed (i_CTRL-X s)

Language Support

Language	Syntax Highlighting	Treesitter Textobjects	Auto Indent	Code Navigation Tags	Rainbow Brackets	Default language servers
ada	✓	✓				ada_ language_ server
adl	✓	✓	✓			
agda	✓					
alloy	✓					
amber	✓	✓	✓	✓	✓	amber-lsp
astro	✓					astro-ls
awk	✓	✓				awk- language- server
bash	✓	✓	✓	✓	✓	zshrs
basic	✓	✓	✓	✓		
bass	✓					bass
beancount	✓					beancount- language- server
bibtex	✓					texlab
bicep	✓					bicep- langserver
bitbake	✓					bitbake- language- server
blade	✓	✓			✓	
blueprint	✓					blueprint- compiler
bovex						
c	✓	✓	✓	✓	✓	clangd
c-sharp	✓	✓	✓	✓	✓	roslyn- language- server, OmniSharp, csharp-ls
c3	✓					c3-lsp
cabal						haskell- language- server- wrapper

Language	Syntax Highlighting	Treesitter Textobjects	Auto Indent	Code Navigation Tags	Rainbow Brackets	Default language servers
caddyfile	✓	✓	✓			
cairo	✓	✓	✓			cairo-language-server
capnp	✓		✓			
cel	✓					
chuck	✓					
circom	✓					circom-lsp
clarity	✓					clarinet
clojure	✓	✓	✓	✓	✓	clojure-lsp
cmake	✓	✓	✓		✓	neocmakelsp, cmake-language-server
codeql	✓	✓				codeql
comment	✓					
common-lisp	✓	✓		✓	✓	cl-lsp
concerto	✓	✓	✓			
cpon	✓		✓			
cpp	✓	✓	✓	✓	✓	clangd
cross-config	✓	✓			✓	taplo, tombi
crystal	✓	✓	✓	✓	✓	crystalline, ameba-ls
css	✓	✓	✓		✓	vscode-css-language-server
csv	✓					
cue	✓					cue
cylc	✓	✓	✓			
cython	✓		✓	✓		
d	✓	✓	✓			serve-d
dart	✓	✓	✓	✓	✓	dart
dbml	✓					
debian	✓					debian-lsp
devicetree	✓					dts-lsp
dhall	✓	✓				dhall-lsp-server
diff	✓					
djot	✓					
docker-bake	✓	✓	✓	✓	✓	docker-language-server

Language	Syntax Highlighting	Treesitter Textobjects	Auto Indent	Code Navigation Tags	Rainbow Brackets	Default language servers
docker-compose	✓	✓	✓			<code>docker-compose-language-server</code> , <code>yaml-language-server</code> , <code>docker-language-server</code>
dockerfile	✓	✓				<code>docker-language-server</code> , <code>docker-language-server</code>
dot	✓					<code>dot-language-server</code>
doxyfile drools	✓	✓	✓	✓		<code>drools-lsp</code>
dtd	✓					
dune	✓					
dunstrc	✓					
earthfile	✓	✓	✓			<code>earthlyls</code>
ebnf	✓					<code>ebnfer</code>
edoc	✓					
eex	✓					
eiffel	✓	✓	✓			<code>eiffel-language-server</code>
ejs	✓					
elisp	✓			✓		
elixir	✓	✓	✓	✓	✓	<code>elixir-ls</code> , <code>expert</code>
elm	✓	✓		✓	✓	<code>elm-language-server</code>
elvish	✓					<code>elvish</code>
embedded-perl	✓					
env	✓	✓				
erb	✓					
erlang	✓	✓	✓	✓	✓	<code>erlang_ls</code> , <code>elp</code>
esdl	✓					
fennel	✓				✓	<code>fennel-ls</code>
fga	✓	✓	✓			
fidl	✓					
fish	✓	✓	✓			<code>fish-lsp</code>
flatbuffers	✓					

Language	Syntax Highlighting	Treesitter Textobjects	Auto Indent	Code Navigation Tags	Rainbow Brackets	Default language servers
forth	✓					forth-lsp
fortran	✓		✓			fortls
freebasic	✓	✓	✓	✓		
fsharp	✓				✓	fsautocomplete
gas	✓	✓				asm-lsp
gdscrip	✓	✓	✓	✓	✓	
gemin	✓					
gherkin	✓					
ghostty	✓					
git-attributes	✓					
git-cliff-config	✓	✓			✓	taplo, tombi
git-commit	✓	✓				commit-lsp
git-config	✓	✓		✓		
git-ignore	✓					
git-notes	✓					
git-rebase	✓					
github-action	✓	✓	✓		✓	actions-language-server, yamllanguage-server, zizmor
gitlab-ci	✓	✓	✓	✓	✓	yamllanguage-server, gitlab-ci-ls
gjs	✓	✓	✓	✓		typescript-language-server, vscode-eslint-language-server, ember-language-server
gleam	✓	✓			✓	gleam
glimmer	✓					ember-language-server
gls	✓	✓	✓	✓	✓	gls_analyzer, glsld
gn	✓					
gnuplot	✓					

Language	Syntax Highlighting	Treesitter Textobjects	Auto Indent	Code Navigation Tags	Rainbow Brackets	Default language servers
go	✓	✓	✓	✓	✓	gopls, golangci-lint- langserver
go-format-string	✓				✓	
godot-resource	✓	✓		✓		
gomod	✓					
gotmpl	✓					
gowork	✓					
gpr	✓					
graphql	✓	✓			✓	ada_ language_ server
gren	✓	✓				graphql-lsp
groovy	✓				✓	
gts	✓	✓	✓	✓		
						typescript- language- server, vscode- eslint- language- server, ember- language- server
hare	✓	✓				hare-lsp
haskell	✓	✓		✓	✓	haskell- language- server- wrapper
haskell-literate	✓					haskell- language- server- wrapper
haskell-persistent	✓					
haxe	✓			✓		
hcl	✓	✓	✓		✓	terraform- ls
hdl	✓					hdl
heex	✓	✓				elixir-ls, expert
helm	✓					helm_ls
hocon	✓	✓	✓			
hoon	✓					
hosts	✓					

Language	Syntax Highlighting	Treesitter Textobjects	Auto Indent	Code Navigation Tags	Rainbow Brackets	Default language servers
html	✓	✓	✓		✓	vscode-html-language-server , superhtml
htmldjango	✓					djllsp , vscode-html-language-server , superhtml
hurl	✓	✓	✓			
hy	✓					hyuga
hyprrlang	✓		✓	✓		hyprrls
idris						idris2-lsp
iex	✓					
ini	✓			✓		
ink	✓					
inko	✓	✓	✓	✓		
janet	✓		✓		✓	
java	✓	✓	✓	✓	✓	
javascript	✓	✓	✓	✓	✓	jdtls
						typescript-language-server
jinja	✓					
jjconfig	✓	✓	✓			taplo , tombi
jjdescription	✓					
jjrevset	✓					
jjtemplate	✓					
jq	✓	✓				jq-lsp
jsdoc	✓					
json	✓	✓	✓		✓	vscode-json-language-server
json-ld	✓	✓	✓		✓	vscode-json-language-server
json5	✓	✓	✓		✓	
jsonc	✓	✓	✓		✓	vscode-json-language-server
jsonnet	✓					jsonnet-language-server

Language	Syntax Highlighting	Treesitter Textobjects	Auto Indent	Code Navigation Tags	Rainbow Brackets	Default language servers
jsx	✓	✓	✓	✓	✓	typescript-language-server
julia	✓	✓	✓	✓	✓	julia
just	✓	✓	✓	✓		just-lsp
kcl	✓					kcl-language-server
kconfig	✓		✓			
kdl	✓	✓	✓	✓	✓	
klog	✓					
koka	✓		✓			koka
kotlin	✓	✓	✓	✓	✓	kotlin-language-server
koto	✓	✓	✓		✓	koto-ls
latex	✓	✓	✓			texlab
ld	✓		✓			
ldif	✓					
lean	✓					lake
ledger	✓					
less	✓		✓		✓	vscode-css-language-server
llvm	✓	✓	✓			
llvm-mir	✓	✓	✓			
llvm-mir-yaml	✓		✓			
log						
lpf	✓					
lua	✓	✓	✓	✓	✓	lua-language-server
lua-format-string	✓					
luap	✓					
luau	✓	✓	✓			luau-lsp
mail	✓	✓			✓	
make	✓		✓			
markdoc	✓					markdoc-ls
markdown	✓			✓		marksman , markdown-oxide , rumdl
markdown-rustdoc	✓					
markdown.inline	✓					
matlab	✓	✓	✓			
mermaid	✓					

Language	Syntax Highlighting	Treesitter Textobjects	Auto Indent	Code Navigation Tags	Rainbow Brackets	Default language servers
meson	✓		✓			mesonlsp
mint						mint
miseconfig	✓	✓	✓			taplo, tombi
mojo	✓	✓	✓			pixi
move	✓		✓			
msbuild	✓		✓			
nasm	✓	✓				asm-lsp
nearley	✓				✓	
nestedtext	✓	✓	✓			
nginx	✓					
nickel	✓		✓			nls
nim	✓	✓	✓		✓	nimlangserver
nix	✓	✓	✓	✓	✓	nil, nixd
nu	✓	✓	✓			nu, nu-lint
nunjucks	✓					
ocaml	✓	✓	✓		✓	ocaml lsp
ocaml-interface	✓					ocaml lsp
odin	✓	✓	✓			ols
ohm	✓	✓	✓			
opencl	✓	✓	✓			clangd
openscad	✓					openscad-lsp
org	✓					
pascal	✓	✓				pasls
passwd	✓					
pem	✓					
penrose	✓	✓				
perl	✓	✓	✓	✓	✓	perlnavigator
pest	✓	✓	✓			pest-language-server
php	✓	✓	✓	✓	✓	intelephense
php-only	✓			✓		
picat	✓	✓	✓	✓	✓	
pip-requirements	✓					
pkgbuild	✓	✓	✓			termux-language-server, bash-language-server
pkc						
pkl	✓		✓			pkl-lsp

Language	Syntax Highlighting	Treesitter Textobjects	Auto Indent	Code Navigation Tags	Rainbow Brackets	Default language servers
po	✓	✓				
pod	✓					
ponylang	✓	✓	✓			pony-lsp
powershell	✓				✓	
prisma	✓	✓				prisma-language-server
prolog	✓		✓			swipl
properties	✓	✓				
protobuf	✓	✓	✓	✓		buf, pb, protols
proverif	✓					
prql	✓					
ptx	✓		✓			
pug	✓					
purescript	✓	✓				purescript-language-server
python	✓	✓	✓	✓	✓	ty, ruff, jedi-language-server, pylsp, zuban
qml	✓	✓	✓			qmls
qmv						
quarto	✓		✓			
quint	✓					quint-language-server
r	✓	✓	✓	✓	✓	R
racket	✓		✓		✓	racket
regex	✓				✓	
rego	✓					regols
rescript	✓	✓				rescript-language-server
ripple	✓			✓	✓	ripple-language-server
rmarkdown	✓		✓			R
robot	✓		✓			robotcode, robotframework_ls
robots.txt	✓	✓		✓		
ron	✓		✓	✓	✓	ron-lsp
rpmspec	✓					

Language	Syntax Highlighting	Treesitter Textobjects	Auto Indent	Code Navigation Tags	Rainbow Brackets	Default language servers
rshtml	✓	✓				rshtml-analyzer, vscode-html-language-server, superhtml
rst	✓			✓		
ruby	✓	✓	✓	✓	✓	ruby-lsp, solargraph
rust	✓	✓	✓	✓	✓	rust-analyzer
rust-format-args	✓					
rust-format-args-macro	✓	✓	✓		✓	
sage	✓	✓				
scala	✓	✓	✓	✓	✓	metals
scfg	✓					
scheme	✓	✓	✓	✓	✓	
scss	✓		✓		✓	vscode-css-language-server
shellcheckrc	✓	✓				
slang	✓	✓	✓			slangd
slint	✓	✓	✓			slint-lsp
slisp	✓			✓		
smali	✓		✓			smalisp
smithy	✓					cs
sml	✓					
snakemake	✓		✓			pylsp
solidity	✓	✓			✓	solc
sourcepawn	✓	✓				sourcepawn-studio
spade	✓		✓		✓	swim
spicedb	✓			✓		
sql	✓	✓			✓	
ssh_client_config	✓					
starlark	✓	✓	✓		✓	starpls, buck2
strace	✓					
strictdoc	✓			✓		
stryke	✓					stryke
styx	✓	✓	✓		✓	styx
supercollider	✓					

Language	Syntax Highlighting	Treesitter Textobjects	Auto Indent	Code Navigation Tags	Rainbow Brackets	Default language servers
svelte	✓	✓	✓	✓	✓	svelte-server
sway	✓	✓	✓			forc
swift	✓	✓	✓	✓	✓	sourcekit-lsp
systemd	✓			✓		systemd-lsp
systemverilog	✓					svlangserver, verible-verilog-ls
t32	✓					
tablegen	✓	✓	✓			
tact	✓	✓	✓			
task	✓					
tcl	✓		✓			
teal	✓					teal-language-server
templ	✓					templ
tera	✓					
textproto	✓	✓	✓			
tfvars	✓		✓			terraform-ls
thrift	✓					
tilt	✓	✓	✓		✓	tilt
tlaplus	✓					
todotxt	✓					
tolk	✓		✓			
toml	✓	✓	✓	✓	✓	taplo, tomls
tql	✓		✓			
tsq	✓				✓	ts_query_ls
tsx	✓	✓	✓	✓	✓	typescript-language-server
twig	✓					
typescript	✓	✓	✓	✓	✓	typescript-language-server
typespec	✓	✓	✓			tsp-server
typst	✓	✓		✓		tinymist
ungrammar	✓					
unison	✓	✓	✓	✓	✓	
uxntal	✓					
v	✓	✓	✓			v-analyzer
vala	✓	✓				vala-language-server
vento	✓					

Language	Syntax Highlighting	Treesitter Textobjects	Auto Indent	Code Navigation Tags	Rainbow Brackets	Default language servers
verilog	✓	✓				verible-verilog-ls
vhdl	✓					vhdl_ls
vhs	✓					
vim	✓				✓	
vue	✓	✓	✓			vue-language-server
wast	✓					
wat	✓					wat_server
webc	✓					
werk	✓					
wesl	✓	✓				
wgsl	✓	✓	✓	✓	✓	wgsl-analyzer
wikitext	✓					wikitext-lsp
wit	✓		✓			
woodpecker-ci	✓	✓	✓	✓	✓	yaml-language-server
wren	✓	✓	✓			
xit	✓					
xml	✓	✓	✓		✓	
xtc	✓					
yaml	✓	✓	✓		✓	yaml-language-server, ansible-language-server
yara	✓					yls
yuck	✓		✓		✓	
zig	✓	✓	✓	✓	✓	zls